

NewtonVirtual

Fundamentos de Inteligência Artificial

Adriano Donizete Pila

Fundamentos de Inteligência Artificial



Adriano Donizete Pila

Pila, Adriano

Fundamentos de Inteligência Artificial. Sorocaba/SP, 2020. 187 f. Ed. 1.

Validador: Iremar Nunes de Lima.

Instituto Cultural Newton Paiva Ferreira Ltda. | ED+ Content Hub, 2020.

Assuntos:

1. Inteligência Artificial;
2. Ciência de Dados;
3. Machine Learning.

Formato: digital.

Recurso: PDF e HTML.

Requisitos do sistema operacional:

- Windows 8.1 ou superior;
- Mac OSX 10.6 ou superior;
- Linux - ChromeOS.

Configurações técnicas:

- 2GB de memória RAM;
- 2.5GHz de processador;
- 10GB de espaço em disco.

Navegadores:

- Google Chrome - Versão mais atualizada;
- Mozilla Firefox - Versão mais atualizada.

Dispositivos móveis:

- iOS 10 ou superior;
- Android 5 ou superior.

Modo de acesso: área restrita - Ambiente Virtual de Aprendizagem.

Todos os direitos desta edição são reservados ao Centro Universitário Facens.

Rodovia Senador José Ermírio de Moraes, 1425, km 1,5 - Sorocaba/SP

CEP: 18.085-784 | tel.: 55 15 3238 1188

Nenhuma parte desta publicação poderá ser reproduzida por qualquer meio ou forma sem autorização.

A violação dos direitos autorais é crime estabelecido pela Lei n.º 9.610/98 e punido pelo artigo 184 do Código Penal.

Todas as imagens, vetores e ilustrações são creditados ao Shutterstock Inc., salvo quando indicada a referência.

Conteúdo

Unidade 1

Conceitos em Ciência de dados e IA..... 6

Unidade 2

Linguagem Python..... 24

Unidade 3

Análise exploratória de dados 46

Unidade 4

Aprendizagem automática..... 76

Unidade 5

Pré-processamento..... 96

Unidade 6

Seleção de modelo 120

Unidade 7

Regressão 139

Unidade 8

Outras aplicações de machine learning 157

PALAVRAS DO AUTOR

Olá, estudante! Nos últimos anos, os dados se tornaram um insumo essencial para impulsionar o crescimento organizacional, permitindo que elas, as empresas, se diferenciem dos demais e mantenham uma vantagem competitiva. Porém, essa vantagem só será obtida se a organização souber utilizar as ferramentas corretas de Data Science para armazenamento, tratamento e, principalmente, análise desses dados.

Nesta disciplina, você irá adquirir os conhecimentos necessários que permitirão entender e identificar os principais conceitos relacionados à Data Science e à Inteligência Artificial, fundamentais para quem deseja trabalhar com o tratamento de dados. Você será apresentado à linguagem Python, uma poderosa linguagem de programação utilizada em diversos tipos de aplicações, principalmente em Data Science. Durante a leitura, você conhecerá a sintaxe dessa linguagem bem como bibliotecas como NumPy, que possui uma vasta coleção de funções matemáticas para tratamento de arrays multidimensionais.

Essa disciplina é fundamental para a formação avançada do discente no uso e implementação de técnicas, algoritmos e linguagens associadas à área de Data Science e Inteligência Artificial. A atualidade da disciplina justifica-se pelo aumento da necessidade de analisar dados, principalmente utilizando técnicas de Inteligência Artificial.

Espero que os conhecimentos dessa disciplina sejam significativos para você.

Seja bem-vindo e bons estudos!

Unidade 1

Conceitos em Ciência de dados e IA

Objetivos de aprendizagem:

Apresentar as ramificações da área de IA e Ciência de dados de modo a destacar os principais pontos de convergência entre elas.

Tópicos de estudo:

- IA;
- Ciência de dados (Data Science);
- Business Analytics;
- Engenharia de dados.

Iniciando os estudos:

No início deste século, se você falasse sobre Inteligência Artificial, haveria uma boa chance de você ser ridicularizado. Muitos lembrariam das máquinas dos filmes de ficção científica como *Jornada nas estrelas* (de 1979) ou do personagem HAL 9000 do filme *2001: uma odisseia no espaço*.

Atualmente, trata-se de palavras mais em alta e com maior poderio de impulsionamento para negócios e indústrias. O termo Inteligência Artificial (IA) é um ponto crucial na transformação digital que está a ocorrer, à medida que as organizações se posicionam para capitalizar a quantidade crescente de dados gerados e coletados.

Mas te pergunto: o que levou a essa mudança de pensamentos? Ou seja, se antes falar de IA era uma “maluquice”, hoje não falar dela lhe trará a mesma alcunha. Bem, em parte, isso se deve à própria revolução do Big Data.

O massivo volume de dados tem nos levado a pesquisas intensas sobre como esses dados podem ser adquiridos, processados e analisados. Como veremos, as máquinas são muito mais adequadas para essas atividades do que os seres humanos.

1 IA

Muitos autores dedicam algumas páginas de livros para discutir o significado de Inteligência Artificial e, em muitos casos, são definições diferentes sobre uma mesma área de estudo. Apesar disso, elas convergem para um mesmo ponto. Começaremos a analisar a palavra inteligência. O que vem a ser inteligência? Inteligência é um conjunto de habilidades distintas e que não se relacionam ou é uma característica inerente a um ser? O ato de raciocinar torna o ser mais ou menos inteligente?

Reflita

Nossa espécie é classificada pela inteligência, somos da espécie homo (humano) sapiens (sábia). Logo, somos uma espécie capaz de apresentar sabedoria a partir de um dado conhecimento. Aliás, conhecimento esse adquirido durante um processo de aprendizado em que acertamos e erramos até saber distinguir o correto do errado. Dessa forma, com base no raciocínio e na experiência, somos capazes de tomar decisões.

Perguntas como essas justificam o fato de não haver definição única sobre **IA**. Uma das definições que mais se aproximam das dos demais pesquisadores da área é feita por Lima *et al.* (2014). Os autores afirmam que IA trata de ações dos computadores que, se fossem também realizadas por seres humanos, seriam consideradas inteligentes.

Existem, é claro, alguns pontos a serem questionados. Por exemplo, um computador é capaz de realizar cálculos mais rapidamente do que um ser humano. Mas isso não demonstra inteligência, afinal trata-se de uma máquina calculando. Um computador é capaz de armazenar uma infinidade de dados em sua memória, em ordem de grandeza maior do que a capacidade de memória de um ser humano, porém, como exposto por Jourdain (1997), isso não é um sinal de inteligência.

1.1 TESTE DE TURING

Uma vez que estamos falando de inteligência e inteligência artificial, nada melhor do que apresentar o famoso teste de Turing.

Alan Turing foi um cientista britânico e pioneiro em ciência da computação que, durante a Segunda Guerra Mundial, desenvolveu uma máquina que ajudou a quebrar o código alemão Enigma. Ele também lançou as bases para a computação moderna e criou teorias sobre a Inteligência Artificial.

Aprofunde-se

Saiba mais sobre o teste de Turing com o filme *O jogo da imitação*.

Título: O Jogo da Imitação.

Ano: 2014.

Sinopse: Durante a Segunda Guerra Mundial, o governo britânico monta uma equipe que tem por objetivo quebrar o Enigma, o famoso código que os alemães usam para enviar mensagens aos submarinos. Um de seus integrantes é Alan Turing (Benedict Cumberbatch), um matemático de 27 anos estritamente lógico e focado no trabalho que tem problemas de relacionamento com praticamente todos a sua volta. Entretanto, para o sucesso do projeto, Turing terá que aprender a trabalhar em equipe e, para isso, tem Joan Clarke (Keira Knightley) como sua grande incentivadora. Fonte: disponível em: <http://www.adorocinema.com/filmes/filme-198371/> Acesso em: 21 jul. 2020.



No ano de 1950, Alan Turing propôs um experimento para aferir a inteligência de um computador. Esse teste ficou conhecido como O jogo da imitação ou teste de Turing e, apesar de existir há um longo tempo, é até hoje utilizado para avaliar o quanto um sistema é realmente inteligente.

Assista

Acesse na plataforma o vídeo: O Jogo da Imitação (Teste de Turing)

Porém, a partir do argumento do Quarto chinês, Searle (1980) apresenta dois conceitos fundamentais que regem a IA:

- **IA fraca:** as populares Siri do iPhone e Alexa da Amazon podem ser chamadas de IA fracas. Essa categorização está enraizada na diferença entre a programação supervisionada e a não supervisionada, pois a assistência ativada por voz geralmente tem uma resposta programada.

O que elas fazem é perceber ou 'procurar' coisas semelhantes às que elas já sabem e classificá-las de acordo com este "conhecimento".

Esse é um recurso semelhante ao humano, mas é basicamente aí que as semelhanças terminam, já que as IAs fracas são simplesmente simulações. Se você solicitar à Siri que ligue o ar-condicionado, ela entenderá palavras-chave como "ligado" e "ar-condicionado" para que ela responda ativando o ar-condicionado. Ou seja, ela apenas responde ao que foi solicitado, mesmo comportamento do homem no Quarto chinês: ele não sabia o que estava escrevendo, apenas replicava algo mediante a um dicionário.

- **IA forte:** apresentada em muitos filmes, a IA forte funciona mais próxima ao cérebro humano. Elas não classificam e usam cluster e associação para processar dados. Isso significa que não há uma resposta programada para suas palavras-chave ou solicitações, como pode ser visto em IAs fracas, e os resultados de sua programação e funções são amplamente imprevisíveis.

Por exemplo, uma máquina pode ouvir “O dia está muito quente” e relacionar essas palavras com “ar-condicionado”. Logo, quando o sistema ouvir as palavras “O dia está muito quente”, automaticamente a IA ligará o ar-condicionado na temperatura tida como ideal. Observe que o sistema inteligente detecta o que o usuário falou, mas ele não estava programado (como na IA Fraca) a receber esta frase. Então, em IA Forte, se o usuário disser: “o dia está quente”, “estou com muito calor” ou qualquer outra frase, sendo ela nova ou não para o dispositivo (ar-condicionado), ele regulará a temperatura.

1.2 RAMOS DE PESQUISA EM IA

É importante entender alguns dos vários campos de estudo da IA para que possamos escolher a estrutura adequada para resolver um determinado problema do mundo real.

1.2.1 SIMBÓLICO

A Inteligência Artificial simbólica visa modelar a mente com representações (metáforas linguísticas). Uma representação é definida como um símbolo interno na mente que reflete uma realidade externa por meio de associação, convenção ou semelhança (JOURDAIN, 1997). Por exemplo: “Vermelho é uma cor” e “todas as cores podem ser vistas”. Um processo dedutivo que opera na relação citada pode retornar a expressão “Vermelho pode ser visto”.

1.2.2 CONEXIONISTA

O ramo conexionista da Inteligência Artificial busca modelar a inteligência simulando as redes neurais em nossos cérebros (metáfora cerebral). Essas redes neurais computacionais são projetadas para construir caminhos entre entrada e saída por meio de uma série de unidades interconectadas, como ilustrado na figura 1, uma rede neural sem treinamento (imagem da esquerda) e caminhos distintos com maiores valores que ‘emergem’ da ativação repetida (imagem da direita, após treinamento).

Por exemplo, supondo que vamos criar um reconhecedor facial que seja capaz de reconhecer um indivíduo X. Deste modo, precisamos treinar a rede neural considerando uma classe positiva e outra nega-

tiva. A classe positiva terá fotos do indivíduo X. Então, como entrada, temos algumas fotos desta pessoa, quanto mais fotos, mais memória e informações sobre esta pessoa o nosso sistema terá. Como saída, teremos um sistema que sabe quem é o indivíduo X, mas ainda não é capaz de reconhecê-lo. Para tanto, precisamos criar uma classe negativa, ou seja, o que não é o indivíduo X.

O desenvolvimento de uma classe de treinamento negativa visa indicar para a rede neural o que (e quem) não é o indivíduo X, quais características o indivíduo X não possui. Então, como entrada, vamos colocar fotos do que não é o indivíduo X. Como saída, teremos uma rede neural que reconhece (agora sim) o indivíduo X.

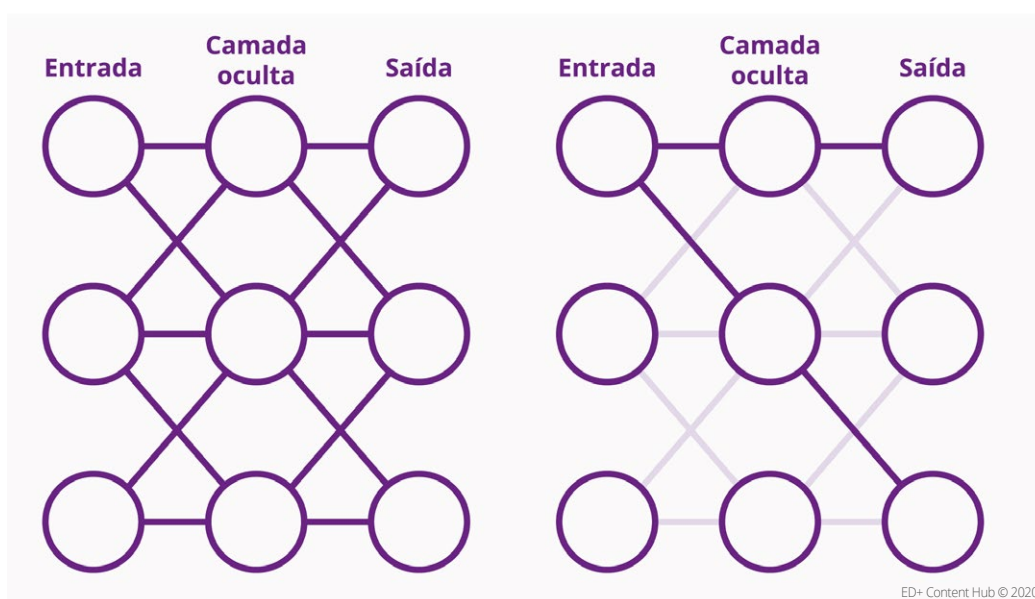


Figura 1 - Redes Neurais. Ao lado esquerdo, uma rede neural sem treinamento; ao lado direito, a mesma rede neural após treinamento.
Fonte: elaborado pelo autor.

O surgimento de caminhos distintos na rede neural imita o processo de aprendizado de um cérebro em que o mesmo recebe repetidos padrões de ativação para aumentar a probabilidade de que esses caminhos disparem novamente ao receber uma entrada sensorial semelhante.

No caso do reconhecedor citado anteriormente, como a rede neural recebeu tanto informações positivas quanto negativas sobre o indivíduo X, ou seja, repetidos padrões de ativação, a probabilidade do indivíduo X ser reconhecido quando a rede neural receber uma entrada sensorial é muito maior, se comparada a uma rede que não recebeu este tipo de treinamento.

1.2.3 ESTATÍSTICO/PROBABILÍSTICO

O Machine Learning (**ML**) é um campo interdisciplinar que usa estatísticas, probabilidade e algoritmos para aprender com os dados e fornecer informações que podem ser usadas para criar aplicativos inteligentes. Podemos também definir ML como a ciência que estuda como os computadores podem atuar sem estarem programados previamente.

2 CIÊNCIA DE DADOS (DATA SCIENCE)

A evolução constante da tecnologia e o enorme volume de dados produzidos diariamente levaram à alta demanda de cientistas de dados no mundo todo. A coleta de dados é importante pois permite que as organizações determinem e, assim, influenciem as tendências e tomadas de decisão em um setor específico, atrelada obviamente a outras técnicas ou ações.

A **DS** (Data Science) inclui muitas tecnologias inovadoras, como a Inteligência Artificial (IA), a Internet das Coisas (**IoT**) e a deep learning, para citar algumas. O progresso e os desenvolvimentos tecnológicos na Data Science aumentaram seu impacto em todos os setores.

Aprofunde-se

Conheça um pouco mais sobre a Data Science nesta rápida e interessante explicação.

Título: Data Science

Acesso em: 26/05/2020.

Disponível em: <https://youtu.be/c6fdZmTwhWo>

Podemos definir DS como uma ferramenta multidisciplinar que extrai informações de dados estruturados e não estruturados usando métodos, processos, algoritmos e sistemas científicos. Em linguagem técnica, a DS unifica o negócio com a estatística, análise de dados e aprendizado de máquina para entender e analisar fenômenos reais por meio de dados (figura 2), ajudando os gerentes a desenvolver estratégias para tomadas de decisões.

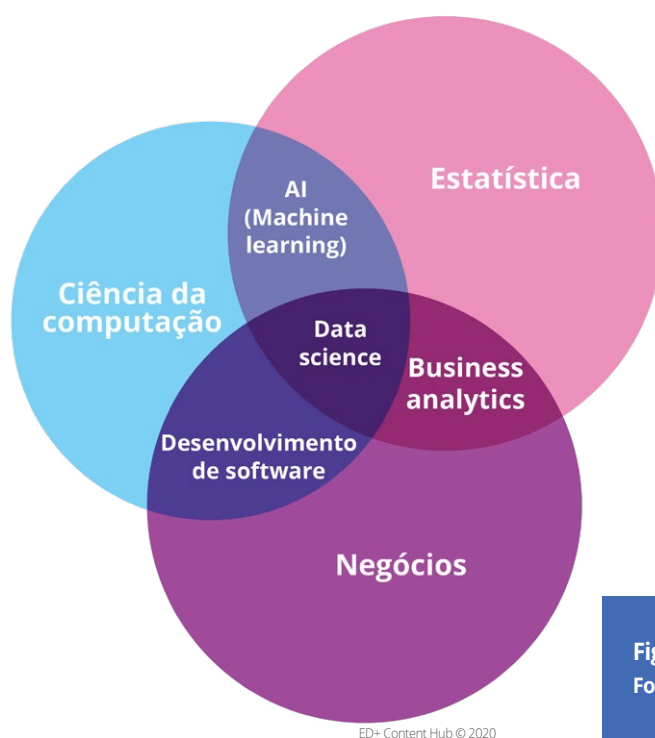


Figura 2 - Data Science.
Fonte: adaptado de Conway (2010).

Deste modo, a DS não pode ser considerada uma ferramenta completamente técnica, pois emprega técnicas e teorias de campos como matemática e estatística, juntamente com ciência da computação e ciência da informação.

Assista

Acesse na plataforma o vídeo: Estatística – Rumo à Data Science

Os três principais componentes envolvidos em DS são a organização (onde o planejamento e a execução do armazenamento físico ocorre), o empacotamento (onde as estatísticas são aplicadas e é desenvolvida a visualização dos dados) e a entrega de dados (este componente garante que o resultado final seja entregue às pessoas em questão). A DS analisa os dados e os resultados são usados para tirar conclusões e tomar decisões sobre eles.

2.1 APLICAÇÕES

A crescente demanda por cientistas de dados fornece uma ideia sobre o escopo da Data Science no futuro. Aqui estão algumas das principais indústrias com alta demanda por cientistas de dados.

2.1.1 LOJAS VIRTUAIS E COMÉRCIO ELETRÔNICO

A implementação da análise de dados ajudará as organizações a prever compras, lucros, perdas e até incentivar os clientes para que comprem coisas por meio da análise de seus perfis de consumo.

2.1.2 INDÚSTRIAS

A Data Science é usada na indústria por vários motivos e o principal deles é com o objetivo de afetar a produtividade, minimizar os riscos e aumentar o lucro.

2.1.3 BANCO E FINANÇAS

A análise de dados está ajudando as instituições financeiras a se envolverem com os clientes de maneira mais significativa, entendendo seus padrões transacionais. Os dados das transações disponíveis para os bancos são usados no gerenciamento de riscos e fraudes. O advento da Data Science levou a um melhor gerenciamento das informações pessoais de cada cliente.

2.1.4 SETOR DE TRANSPORTE

O setor de transporte cria quantidades sem precedentes de dados diariamente. A maioria dos dados do setor é capturada por meio de sistemas de contagem de passageiros, sistemas de localização de veículos, sistemas de bilhetagem e cobrança. O uso de DS contém o potencial sem precedentes de obter informações sobre o planejamento e o gerenciamento de redes de transporte.

Reflita

A Netflix utiliza um sistema inteligente de DS para nos recomendar filmes. Desse modo, quando começamos a assistir um filme e desistimos, a Netflix entende que aquele gênero não é do nosso gosto, logo é uma categoria que dificilmente ela voltará a nos indicar (com base em outros dados). Porém, se assistimos muita comédia, por exemplo, a tendência é que indicações de filmes e séries desse gênero tenham grandes recomendações em nossa página principal.

As vantagens para as empresas desses ramos (e de outros não listados) vão desde vender produtos e serviços para você até entregar a informação que você precisa e que mais lhe interessa.

3 BUSINESS ANALYTICS

Business Analytics (BA) é uma solução de gerenciamento de dados e subconjunto de Business Intelligence (BI), que se refere ao uso de metodologias como mineração de dados, análise preditiva e análise estatística (análise de regressão) para analisar e transformar dados em informações úteis, identificar e antecipar tendências e resultados e, finalmente, tornar as decisões de negócios inteligentes e orientadas a dados.

Aprofunde-se

Conheça a mineração de dados, um dos principais componentes de Business Analytics.

Título do livro: Sistemas de informação gerenciais

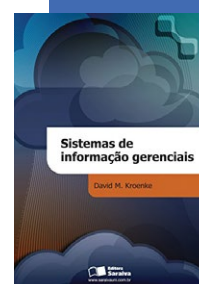
Autor: David M. Kroenke.

Local: São Paulo.

Editora: Saraiva

Data de publicação: 06/10/2017.

ASIN: B076C24TP7



3.1 COMPONENTES

Os principais componentes de **BA** são:

- **Agregação de dados (data aggregation):** antes da análise, os dados devem primeiro ser coletados, organizados e filtrados por meio de dados voluntários ou registros transacionais.
- **Mineração de dados (data mining):** a mineração de dados, por meio de grandes conjuntos, estatísticas e aprendizado de máquina, é utilizada para identificar tendências e estabelecer relacionamentos com o objetivo de encontrar padrões anteriormente ocultos. São seis estágios: compreensão do negócio, compreensão dos dados, preparação dos dados, modelagem dos dados, avaliação e implantação.
- **Mineração de texto (text mining):** explora e organiza conjuntos de dados de texto não estruturados para fins de análise qualitativa e quantitativa.
- **Análise preditiva:** essa análise usa uma variedade de técnicas estatísticas para criar modelos preditivos, que extraem informações de conjuntos de dados, identificam padrões e fornecem uma pontuação preditiva para uma série de resultados organizacionais.
- **Visualização de dados (data visualization):** fornece representações visuais como tabelas e gráficos para análise de dados fácil e rápida.

3.2 FERRAMENTAS

As ferramentas de BA incluem muitas metodologias e soluções de código aberto que podem ser aproveitadas para ajudar os analistas a executar tarefas e gerar relatórios de fácil entendimento até mesmo para leigos.

As ferramentas de gerenciamento de requisitos ajudam a garantir que as organizações possam identificar, documentar, verificar e atender às necessidades e expectativas de seus dados demográficos e clientes existentes. Os requisitos podem ser gerados por clientes, parceiros ou partes interessadas. Muitas empresas simplesmente usam o Microsoft Excel em nível de negócios.

Ferramentas de Business Analytics

As ferramentas de relatório podem ser obtidas em plataformas de código aberto para permitir que os analistas de negócios gerenciem seus processos e apresentem suas descobertas de maneira relevante e compreensível. A seguir, veja algumas das opções de ferramentas de código aberto mais flexíveis e fáceis de usar.



BIRT

O BIRT é uma ferramenta básica de análise de código aberto para relatórios, dashboards e visualizações, mas requer conhecimento prático de Java, scripts e formatação.



OmniSci

Anteriormente chamado de MapD, o OmniSci é uma ferramenta de análise de alto desempenho, ideal para analistas de negócios e cientistas de dados, com Big Data capaz de consultar painéis interativos e um mecanismo SQL projetado para gerenciar cargas de trabalho extremamente grandes.



Apache Zeppelin

Zeppelin by Apache

Essa opção é anunciada como uma ferramenta multiuso para visualização, análise e descoberta de dados e gerencia prontamente dados de tecnologias relacionadas ao Apache, incluindo Spark, Hadoop Hive, PostgreSQL e Python.



SpagoBI

O SpagoBI é uma ferramenta popular com recursos que inclui relatórios básicos, painéis e ferramentas de gerenciamento de dados. O dashboard permite que os usuários criem gráficos 3D para relatórios em alto nível.



ED+ ContentHub © 2020

Infográfico 1 - Business Analytics.
Fonte: elaborado pelo autor.

Além dessas listadas no infográfico, podemos citar outras ferramentas como Matomo e Metabase. Matomo é uma alternativa de código aberto ao Google Analytics, com um alto nível de personalização e recursos expansivos.

Assista

Acesse na plataforma o vídeo: Big Data, mineração de dados e Business Intelligence: comparando tecnologias

De modo semelhante, Metabase é uma ferramenta de código aberto que permite aos usuários fazerem perguntas inserindo dados de entrada e exibe respostas em tabelas detalhadas ou gráficos de barras, o que objetiva facilitar a leitura dos relatórios.

Saiba mais

Cientistas, analistas e engenheiros de dados trabalham juntos no processo de análise de dados para coletar, integrar e preparar dados para o desenvolvimento, teste e revisão de modelos analíticos, garantindo resultados precisos.

3.3 BUSINESS ANALYTICS VS DATA ANALYTICS

A análise de dados ou Data Analytics é um termo abrangente que se refere à ciência de analisar dados brutos para transformá-los em informações úteis a partir das quais tendências e métricas podem ser reveladas. Embora Business Analytics e Data Analytics tenham como objetivo melhorar a eficiência operacional, BA é especificamente orientada para os usos e DA tem um foco mais amplo, englobando os relatórios de Business Intelligence (**BI**) e os relatórios de **OLAP** (Online Analytical Processing).



Figura 3 - Os relatórios de BI permitem analisar, visualmente, o comportamento dos dados.

O OLAP é uma categoria de ferramenta de software que permite aos usuários analisar informações oriundas de vários sistemas de banco de dados, informações estas que estão centralizadas em um data warehouse, com alto nível de sumarização. É uma tecnologia que permite aos analistas extrair e visualizar dados de negócios de diferentes pontos de vista.

4 ENGENHARIA DE DADOS

Os dados se tornaram um dos recursos mais valiosos do mundo graças à rápida transformação digital das indústrias globais. Na era digital em que estamos – a quarta Revolução Industrial –, os dados dos clientes são o novo petróleo, dando um tremendo poder econômico às empresas que os produzem e os controlam, e informações como vantagem crucial quando se trata de prosperar no campo de batalha competitivo global (SILVA; BARBOSA; CORDOVA, 2018). Porém, essa vantagem só será obtida se a empresa souber utilizar as ferramentas corretas para armazenamento, tratamento e análise para a tomada de decisão.

Aprofunde-se

Este artigo aborda as ferramentas e os modos de processamento do ETL. Compreender cada etapa do ETL é fundamental para o profissional que utilizará ferramentas analíticas para a tomada de decisão.

Título do artigo: O processo **ETL** em sistemas Data Warehouse atuais

Link: <http://repositorium.sdum.uminho.pt/handle/1822/11435>

Acesso em: 26/05/2020.

Para entender a engenharia de dados em termos simples, vamos aos bancos de dados. Dentro de uma grande organização, geralmente existem muitos tipos diferentes de software de gerenciamento de operações: **ERP**, **CRM**, sistemas de produção, sistemas legados e muito mais. E, portanto, também existem muitos bancos de dados diferentes. À medida que o número de fontes de dados se multiplica, a dispersão dos dados em vários formatos impede a organização de ter uma visão completa e clara do estado de seus negócios.

Logo, surge a necessidade de realizar a integração dos dados em um sistema de armazenamento unificado, em que os mesmos são coletados, reformatados e serão disponibilizados para uso em um Data Warehouse (**DW**). Deste modo, tanto cientistas de dados como engenheiros de BI podem se conectar ao Data Warehouse, acessar os dados que necessitam e começar a obter informações valiosas sobre eles.

Refleta

Considere que, em um sistema ERP, o peso dos produtos está em quilogramas. Por outro lado, o sistema de controle de estoque apresenta os pesos em gramas. Desse modo, se tivermos o produto melancia, ele pode estar cadastrado no ERP com o valor 5, indicando 5kg e no sistema de controle de estoque com o valor 5000, indicando 5000g. Apesar de serem as mesmas informações, os dados estão diferentes. Quando esses dados forem enviados para o Data Warehouse, eles terão de ser tratados para que haja apenas uma unidade de medida.

Enquanto o Data Warehouse diz respeito ao armazenamento de dados, o pipeline ETL (figura 4) e o pipeline de dados garantem o consumo e o manuseio deles. Esses pipelines são compostos por um conjunto de ferramentas e processos para executar a integração de dados.

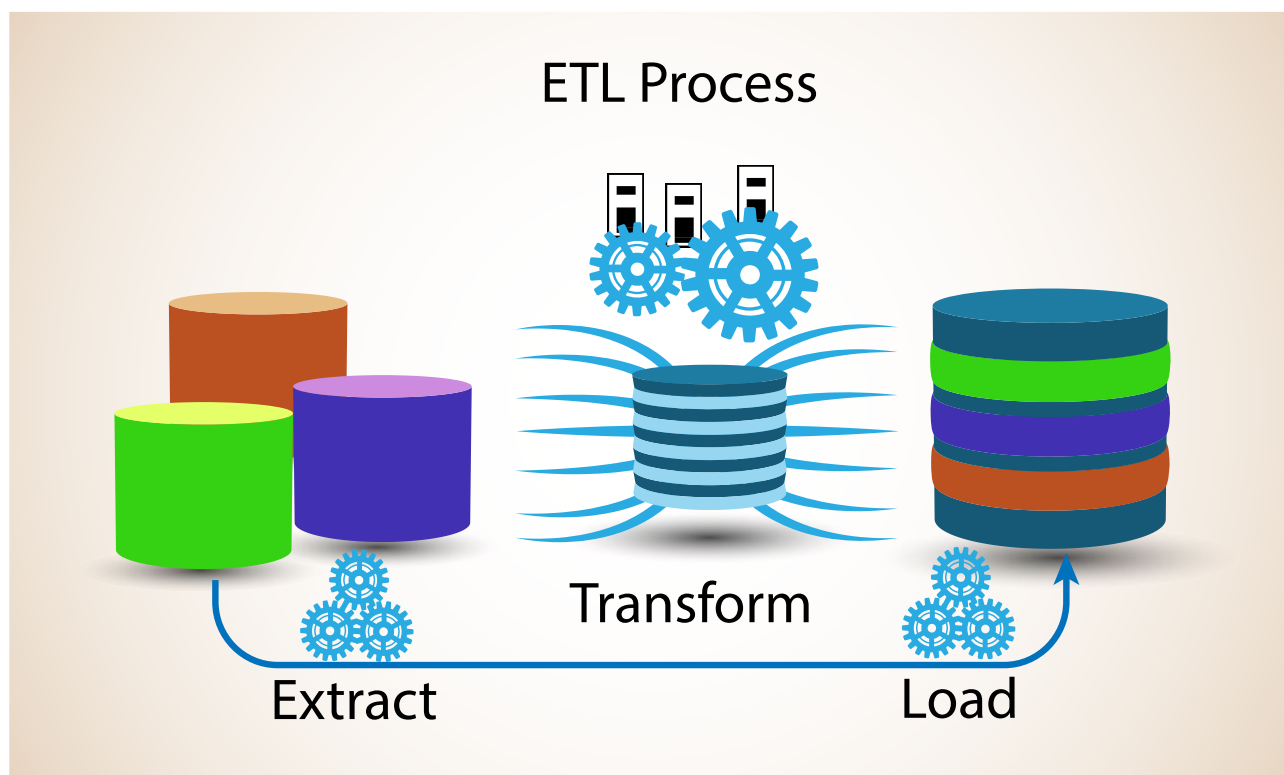


Figura 4 - Processo ETL.

A construção de pipelines é a principal responsabilidade da engenharia de dados. Requer habilidades avançadas de programação para projetar um programa para troca de dados contínua e automatizada.

Assista

[Acesse na plataforma o vídeo: Processo ETL \(Extract, Transform, Load\)](#)

Configurar um fluxo de dados seguro e confiável é uma tarefa desafiadora. Há muitas coisas que podem dar errado durante o transporte de dados: os dados podem ser corrompidos ou as fontes de dados podem entrar em conflito, gerando dados duplicados ou incorretos. A inserção de dados em um único local requer planejamento e testes cuidadosos para filtrar dados indesejados, eliminando duplicados e tipos de dados incompatíveis, ofuscando informações confidenciais sem perder dados críticos.

CONSIDERAÇÕES FINAIS

Nesta unidade, comecei apresentando a você o conceito de Inteligência Artificial. Você pôde conhecer o teste de Turing e o argumento do Quarto chinês, experimentos esses que serviram como base para a evolução da IA.

Em seguida, você conheceu o termo Data Science, um conjunto de ferramentas que abrange a estatística, a computação, entre outras áreas. Com essa multidisciplinaridade, é possível extrair informações de dados estruturados e não estruturados usando métodos, processos, algoritmos e sistemas científicos.

Você pôde compreender que Business Analytics é um processo de gerenciamento de dados que, entre seus componentes, consta a mineração de dados, a análise preditiva e a visualização de dados.

Por fim, você conheceu a engenharia de dados, cujo principal objetivo se concentra em transformar dados em um processo ETL.

GLOSSÁRIO

BA: Business Analytics.

BI: Business Intelligence.

CRM: Customer Relationship Management.

DS: Data Science.

DW: Data Warehouse.

ERP: Enterprise Resource Planning.

ETL: Extract, Transform, Load.

IA: Inteligência Artificial.

IoT: Internet of Things.

ML: Machine Learning.

OLAP: Online Analytical Processing.

REFERÊNCIAS

CASTRO, Leandro Nunes de; FERRARI, Daniel Gomes. **Introdução à mineração de dados: conceitos básicos, algoritmos e aplicações**. Saraiva, 2016. ISBN Digital: 9788547201005.

CONWAY, Drew. **Data Science Venn Diagram**, 2010. Disponível em: <http://drewconway.com/zia/2013/3/26/the-data-science-venn-diagram> Acesso em: 2 set. 2020.

FACELI, Katti *et al.* **Inteligência Artificial: uma abordagem de aprendizado de máquina**. LTC, 2015.

JOURDAIN, Robert. **Música, Cérebro e Êxtase**. São Paulo: Objetiva, 1997.

LIMA, Isaías *et al.* **Inteligência Artificial**. Rio de Janeiro: Campus, 2014.

SEARLE, J. R. **Minds, brains, and programs**. Disponível em: <https://www.cambridge.org/core/journals/behavioral-and-brain-sciences/article/minds-brains-and-programs/DC644B47A4299C637C89772FACC2706A> Acesso em: 26 mai. 2020.

SILVA, K.; BARBOSA, C.; CORDOVA, R. **Sistemas de informações gerenciais**. Porto Alegre: SAGAH, 2018

Unidade 2

Linguagem Python

Objetivos de aprendizagem:

Conhecer a linguagem Python, a utilização básica das variáveis, estruturas de condição e repetição, bem como as estruturas de dados e a manipulação de matrizes em NumPy.

Tópicos de estudo:

- Python e variáveis;
- Estrutura de condição e repetição;
- Estrutura de dados;
- Manipulação de matrizes em NumPy.

Iniciando os estudos:

Sem dúvidas é possível afirmar que a Inteligência Artificial (IA) é a tecnologia de tendências do futuro. Você já deve ter se deparado em algum momento com algum aplicativo desenvolvido utilizando IA. Desse modo, muitas empresas e pesquisadores estão se interessando por essa poderosa ferramenta. Entretanto, a principal questão que surge aqui é: em que linguagem de programação esses aplicativos de IA podem ser desenvolvidos? Existem várias linguagens de programação como Lisp, Prolog, C++, C#, Java e Python que podem ser usadas para o desenvolvimento de aplicativos de IA. Dentre as diversas linguagens, a linguagem de programação Python ganha uma enorme popularidade devido à sintaxe simples, uso de código enxuto (menos código, mais funcionalidades) e importantes bibliotecas embutidas como NumPy, SciPy e SimpleAI.

Nesta unidade, você conhecerá a linguagem Python, sua sintaxe e semântica, bem como recursos e bibliotecas que vão acompanhar você no decorrer de sua carreira profissional.

1 PYTHON E VARIÁVEIS

O Python é uma poderosa linguagem de script de alto nível, interpretada e multiparadigmática. Isso significa que ele possui características de diversos paradigmas de programação por ser imperativo, funcional, estruturado e orientado a objetos.

Dizer que Python é uma linguagem interpretada significa que o seu código-fonte é processado em tempo de execução pelo interpretador, ou seja, diferente de linguagens como C ou C# que são compiladas. No Python, você não precisará compilar seu programa antes de executá-lo.

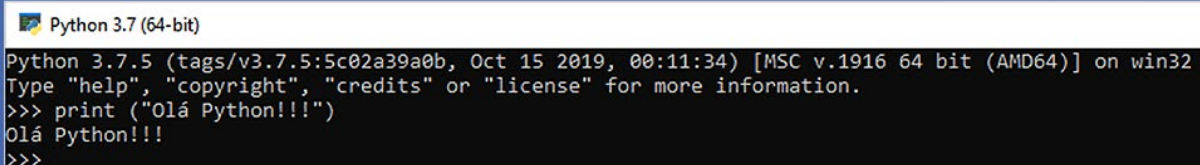
Python, assim como outras linguagens como C# e Java, é orientado a objeto. Logo, é possível criar classes encapsulando atributos e métodos dentro de um objeto.

Assista

[Acesse na plataforma o vídeo: Primeiro programa em Python](#)

Neste primeiro momento, sempre que formos desenvolver algum código em Python nós vamos executá-lo no terminal do interpretador dessa linguagem. Logo mais, você poderá utilizar os recursos de editores de texto e outras funcionalidades que visam agilizar o desenvolvimento.

Vamos então criar nosso primeiro programa em linguagem Python, especificamente usando a programação em modo interativo. Abra o interpretador e digite uma pequena saudação como o código da figura 1.



```
Python 3.7 (64-bit)
Python 3.7.5 (tags/v3.7.5:5c02a39a0b, Oct 15 2019, 00:11:34) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Olá Python!!!")
Olá Python!!!
>>>
```

Figura 1 - Primeiro programa em Python.

Fonte: elaborado pelo autor.

Simples, não é? Não precisamos declarar nenhum tipo de biblioteca ou outro recurso extra, simplesmente demos o comando **print ("Olá Python")**. Observe atentamente o espaço entre o comando print e os parênteses. A linguagem Python trabalha diretamente com o uso de indentação. Você verá, ao longo

desta unidade, que um espaço pode fazer uma profunda diferença no código (e, às vezes, fazer você perder alguns minutos procurando erros).

Vejamos o uso do Python como uma calculadora. Vamos fazer uma operação matemática utilizando seus operadores aritméticos que são: adição (+), subtração (-), multiplicação (*), divisão (/), módulo (%), expoente (**) e divisão de piso (//). Veja um exemplo na figura 2.

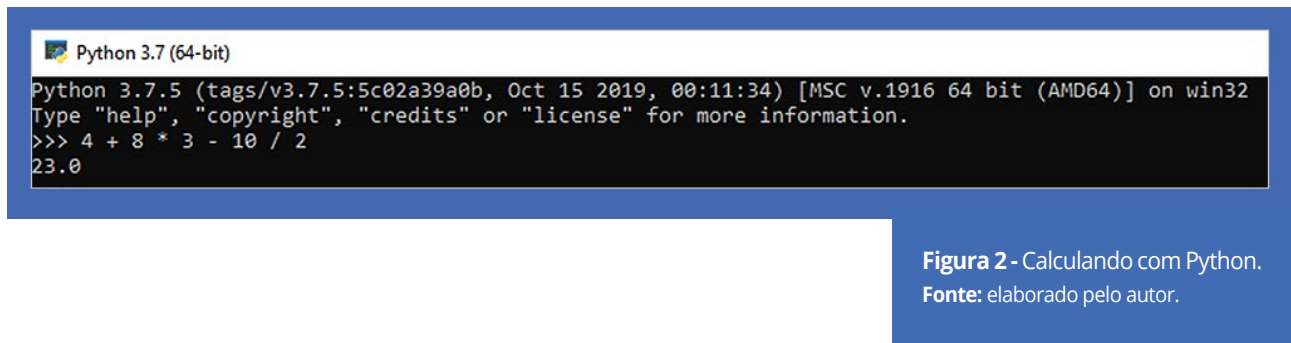


Figura 2 - Calculando com Python.

Fonte: elaborado pelo autor.

A precedência de operação começa na multiplicação. Em seguida, é a vez da divisão, para depois realizar a soma e, por fim, a subtração.

Assim como em outras linguagens, é interessante guardarmos na memória do computador os valores com os quais estamos trabalhando. O principal recurso que nos permite manipular a memória do computador e guardar valores são as variáveis.

Diferentemente de outras linguagens em que precisamos declarar o tipo da variável, em Python essa declaração não é explícita. O operando à esquerda do operador = é o nome da variável e o operando à direita do operador = é o valor armazenado na variável. Veja o exemplo da figura 3.

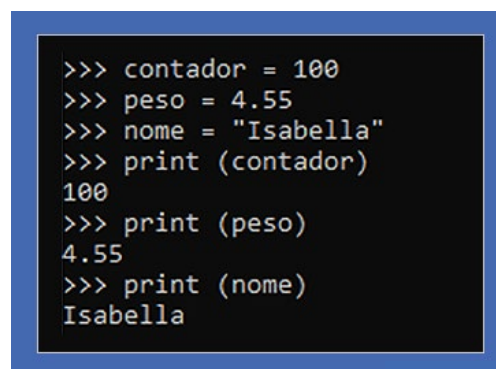


Figura 3 - Uso de variáveis.

Fonte: elaborado pelo autor.

Na primeira linha, foi declarada uma variável chamada *contador* e a ela foi atribuído o valor 100, o que a torna uma variável do tipo inteiro. Em seguida, foi declarada uma variável chamada *peso* a qual recebeu

o valor 4.55, o que a torna um ponto flutuante ou variável do tipo float. Por fim, foi declarada uma variável que recebeu um conteúdo de texto, logo, ela passou a ser do tipo string.

O Python possui cinco tipos de dados, são eles: números, string, lista, tupla e dicionário. Neste primeiro momento, você conhecerá os números e as strings. Os demais deixaremos para o próximo tópico.

1.1 NÚMEROS

Em Python, existem três tipos numéricos diferentes: inteiros (int, por exemplo 10, 100, -40, -0x260), pontos flutuantes (float, por exemplo 0.0, 88.5, -90., 32e100) e números complexos (complex, por exemplo 3.14j, .876j, 3e+26j).

1.2 STRINGS

Assim como em outras linguagens, em Python as strings são identificadas como um conjunto contíguo de caracteres representados entre aspas.

Também é possível trabalhar com substrings. Esse subconjunto pode ser obtido usando o operador de fatiamento `[]` e `[:]`, com índices que começam em 0 (primeiro caractere da string) e vão até o final da string. Observe um exemplo de fatiamento e obtenção de substring na figura 4.

```
>>> mensagem = "Olá Python!"
>>> print (mensagem[4:10])
Python
>>>
```

Figura 4 - Substring.
Fonte: elaborado pelo autor.

Na figura 4, temos uma variável *mensagem* que recebe uma frase **"Olá Python!"**. Essa mensagem é recortada, começando da posição 4 (ou seja, letra P) até a posição 10 (sinal de exclamação), porém nós não recuperamos a posição 10, apenas indicamos que o final é antes de 10. Por fim, é apresentada a substring Python.

Conforme você for avançando nos estudos, você verá outras técnicas de manipulação de strings em Python.

Assim como é possível realizar operações em tipos numéricos, também é possível realizar em strings. Veja algumas.

Aprofunde-se

Neste livro, das páginas 77 a 115, você poderá se aprofundar no conceito de variáveis em Python e nas operações que são possíveis realizar com elas.

Autor: David Ascher e Mark Lutz.

Título do livro: Aprendendo Python

Local: Porto Alegre.

Editora: Bookman.

Data de publicação: 2007.

ISBN: 9788577800131.



1.2.1 CONCATENANDO STRING

Podemos utilizar o sinal de adição (+) para concatenar (juntar) duas strings. Veja um exemplo na figura 5.

```
>>> mensagem1 = "Bom "
>>> mensagem2 = "Dia "
>>> mensagem3 = "Python"
>>> print (mensagem1 + mensagem2 + mensagem3)
Bom Dia Python
```

Figura 5 - Concatenando string.

Fonte: elaborado pelo autor.

Reflita

Em relação à manipulação de string, realizar a adição é um processo válido, entretanto a subtração não, bem como o uso dos demais operadores aritméticos. Se você tentar subtrair ou multiplicar duas cadeias de caracteres, será apresentada uma mensagem de erro para você. Experimente! Afinal, errar também faz parte da aprendizagem.

1.2.2 ATUALIZANDO A STRING

Vimos como tratar uma string em uma substring. Vamos supor que nós temos a string **Olá Python**, no entanto, por alguma necessidade, desejamos substituir a palavra Python por Brasil. Veja na figura 6 como faríamos.

```
>>> mensagem = "Olá Python!"  
>>> print (mensagem[:4]+'Brasil')  
Olá Brasil
```

Figura 6 - Atualizando a string.
Fonte: elaborado pelo autor.

1.2.3 OPERADOR DE FORMATAÇÃO DE STRING

O operador de formatação de string é semelhante ao operador da linguagem C da família de funções **printf()**. Utilizaremos **%s** quando quisermos referenciar strings, **%d** quando quisermos referenciar valores inteiros e **%f** quando quisermos referenciar pontos flutuantes. Veja o exemplo na figura 7.

```
>>> nome = "Juliana"  
>>> idade = 26  
>>> peso = 60.8  
>>> print ("Seu nome é %s, ela tem %d anos e pesa %f quilogramas" % (nome, idade, peso))  
Seu nome é Juliana, ela tem 26 anos e pesa 60.800000 quilogramas
```

Figura 7 - Operador de formatação.
Fonte: elaborado pelo autor.

Na mesma frase, colocamos os conteúdos das variáveis **nome**, **idade** e **peso**.

Existem outras operações sobre strings que você verá ao longo dos seus estudos, mas creio que essas, por enquanto, sejam as principais.

2 ESTRUTURA DE CONDIÇÃO E REPETIÇÃO

Você viu, no tópico anterior, os principais conceitos de Python no que tange a variáveis. Agora, vamos manipular essas variáveis com dois conceitos essenciais: estruturas condicionais e estruturas de repetição.

2.1 ESTRUTURA DE CONDIÇÃO

No código da figura 8 nós temos um simples jogo em que o usuário precisa adivinhar um número já predefinido pelo programador. Nesse caso, se o usuário digitar o valor 10, ele ganhará o jogo, se não, ele perderá. Veja o código.

```
1  print ("Bem-vindo ao jogo!")
2
3  entrada = input("Digite um número entre 0 e 100:")
4
5  valor = int(entrada)
6
7  if valor == 10:
8      print("Você venceu. Parabéns!")
9  else:
10     print("Você perdeu. Tente Novamente!")
11
12  print("G A M E   O V E R")
```

Figura 8 - Exemplo de código utilizando estrutura condicional.
Fonte: elaborado pelo autor.

A linha 3 apresenta uma novidade, a função **input()**, que permite o usuário realizar uma entrada de dados já mostrando uma mensagem dizendo o que ele tem que fazer. Em C equivale a um **printf()**, indicando para o usuário digitar um valor e um **scanf()** para armazenar esse valor.

A variável **entrada** recebe o valor que o usuário digitou. Entretanto, estamos convertendo esse valor para inteiro, afinal, nosso programa realizará o teste somente em valores inteiros. Desse modo, na linha 5 fazemos a conversão utilizando a função **int()**.

Na linha 7, temos o início da nossa condicional. Observe que estamos comparando o conteúdo da variável **valor** com o número 10. Se desejamos saber se o operando da esquerda é igual ao operando da direita, utilizamos dois sinais de igualdade (**==**), se queremos saber se o operando da esquerda é diferente do operando da direita, utilizamos o operador **!=**. Também podemos utilizar os operadores **>**, **<**, **>=** e **<=**.

Diferentemente de outras linguagens, no Python nós **não** precisamos abrir chaves para identificar o escopo da função, todavia muito cuidado com a indentação, ou seja, esse espaço que você pode observar na linha 8. Para colocar uma instrução dentro de uma estrutura como o **if**, é necessário pressionar uma vez a tecla tab.

Quando o usuário digitar o número 8, por exemplo, serão apresentadas duas mensagens: "Você perdeu. Tente novamente!" e "Game over". Se ele digitar o número 10, aparecerá: "Você venceu. Parabéns!" e "Game over". A mensagem da linha 12 será mostrada independentemente do valor digitado. Mas, se dermos um único tab na linha 12, veremos apenas a mensagem de "Game over" quando o usuário digitar um número diferente de 10.

Vamos aprimorar o nosso código dando uma chance ao usuário. Para ele ganhar, deverá digitar 10, 20 ou 30. Desse modo, ele utilizará o operador lógico **or**. Destaca-se ainda que temos os operadores lógicos **and** e **xor**. O operador **or** só retornará o resultado verdadeiro quando pelo menos uma das expressões for verdadeira. Já o operador **and** será verdadeiro quando todas as expressões forem verdadeiras, enquanto o operador **xor** retornará verdadeiro quando uma expressão for verdadeira e outra expressão for falsa.

Os operadores lógicos manipulam expressões inteiras e, diferente dos operadores aritméticos, eles não manipulam variáveis. Veja então como fica o código com essa mudança na figura 9.

```

1  print ("Bem-vindo ao jogo!")
2
3  entrada = input("Digite um número entre 0 e 100:")
4
5  valor = int(entrada)
6
7  if valor == 10 or valor == 20 or valor == 30:
8      |   print("Você venceu. Parabéns!")
9  else:
10     |   print("Você perdeu. Tente Novamente!")
11
12  print("G A M E   O V E R")

```

Figura 9 - Código com operador lógico.
Fonte: elaborado pelo autor.

Observe que agora, na linha 7, temos o operador lógico **or**, o que indica que todo **if** será verdadeiro caso qualquer uma das condições sejam verdadeiras, ou seja, o usuário vencerá se digitar **10, 20** ou **30**.

Um outro operador lógico que podemos utilizar é o **and**. Esse operador indica que, para que uma condição seja verdadeira, todos os valores precisam acontecer. Nesse caso, se tivéssemos **valor == 10 and valor == 20 and valor == 30**, o usuário **nunca** venceria. Isso ocorre pois, se ele digitar o valor 10, a primeira condição (**valor==10**) seria verdadeira, entretanto as outras duas seriam falsas. Podemos

concluir então que, para esse caso, seria impossível o usuário ganhar, pois como a variável **valor** só assume um único valor, não tem como as três condições serem verdadeiras.

Podemos também ter uma condição **if** dentro de outra condição **if**. É o que chamamos de **instruções aninhadas**.

No código, se o usuário digitar 10, 20 ou 30 ele ganha o jogo, porém, se digitar 10, ele ganhará 5 pontos, se digitar 20, ele ganhará 100 pontos e, se digitar 30, ele ganhará 1000 pontos. Veja como fica o código na figura 10. Observe atentamente o uso da indentação.

```
1  print ("Bem-vindo ao jogo!")
2
3  entrada = input("Digite um número entre 0 e 100:")
4
5  valor = int(entrada)
6
7  if valor == 10 or valor == 20 or valor == 30:
8      print("Você venceu. Parabéns!")
9      if valor == 10:
10         print ("Você ganhou 5 pontos")
11     elif valor == 20:
12         print("Você ganhou 100 pontos")
13     else:
14         print("Você ganhou 1000 pontos")
15 else:
16     print("Você perdeu. Tente Novamente!")
17
18 print("G A M E   O V E R")
```

Figura 10 - Código com aninhamento de estrutura condicional.
Fonte: elaborado pelo autor.

Observe na figura 10 que a condição para o usuário ganhar 100 pontos é digitar o valor 20. Veja que temos uma estrutura condicional **if** dentro de outra **if**. Temos também, na linha 11, a instrução **elif**, a qual permite verificar várias expressões para **true** e executar um bloco de códigos assim que uma dessas condições for verdadeira. Podemos não ter o **elif**, bem como podemos ter vários **elif**, dependerá do seu algoritmo.

2.2 ESTRUTURA DE REPETIÇÃO

O nosso jogo não está sendo justo. Ele permite que o usuário realize uma única tentativa. Vamos modificar nosso código utilizando o conceito de estruturas de repetição, ou seja, enquanto uma dada condição for verdadeira, o nosso código continuará em execução. Em Python, nós temos o laço **for** e o laço **while**. Vejamos a sintaxe de cada um.

2.2.1 LAÇO DE REPETIÇÃO WHILE

Enquanto uma dada condição for verdadeira (condição de parada), o conteúdo de seu bloco (aquele que está em recuo, indentado) será executado. A condição de parada pode ser uma relação simples ou uma expressão com diversos operadores relacionais e lógicos. Veja um exemplo de um laço **while** na figura 11.

```
1  contador = 1
2
3  ✓ while contador < 10:
4      print ("Valor do contador é ", contador)
5      contador = contador + 1
6  print ("Fim!")
```

Figura 11 - Uso de laço de repetição **while**.
Fonte: elaborado pelo autor.

Observe que este código inicializa uma variável antes de entrar no laço de repetição. A condição de parada é **contador < 10**, ou seja, enquanto o contador for menor do que 10, faça o que está dentro da estrutura **while**. Desse modo, será apresentado o valor do contador (linha 4) e, em seguida, é somado mais 1 no valor atual da variável contador. A essa ação de **somar mais 1** nós chamamos de incremento. Por fim, note que a linha 6 só será impressa na tela após o laço de repetição concluir a execução.

Aprofunde-se

Neste livro, das páginas 22 a 30, você poderá se aprofundar na sintaxe do Python, além de se aprofundar no conceito de estrutura de repetição e de controle de fluxo (estrutura condicional).

Autor: Luiz Eduardo Borges.

Título do livro: Python para desenvolvedores

Local: Rio de Janeiro.

Editora: Edição do autor.

Data de publicação: 2010.

ISBN: 9788590945116.



2.2.2 LAÇO DE REPETIÇÃO FOR

Esse tipo de laço segue o mesmo princípio do laço anterior, realizando um loop até que se atinja uma condição de parada. Entretanto, o **for** do Python possui uma sintaxe diferente do **for** de outras linguagens como Java ou C. Ele se assemelha mais com o **foreach**.

Sua estrutura é definida como a seguir, na figura 12:

```
for <variável> in <objeto iterável>:
    bloco de instrução
```

Figura 12 - Estrutura do **for**.
Fonte: elaborado pelo autor.

O código da figura 13 será responsável por percorrer a frase **meu programa Python** e mostrar cada uma das letras na tela.

```
1  ✓ for letras in 'meu programa python':  
2    |     print('ciclo: ', letras)
```

Figura 13 - Laço for.

Fonte: elaborado pelo autor.

Saiba mais

Toda estrutura de repetição, seja ela **for** ou **while**, necessita de uma condição de parada. Caso contrário, aquele trecho de código ficará em execução por tempo infinito. Muito cuidado, afinal não colocar uma condição de parada pode acabar com o seu programa.

Os laços de repetição são muito utilizados quando desejamos percorrer estruturas de dados.

Assista

Acesse na plataforma o vídeo: Construindo um jogo iterativo de adivinhação

3 ESTRUTURA DE DADOS

Além dos tipos primitivos de dados como strings e numéricos com variáveis que recebem apenas um único valor, nós podemos organizar variáveis de modo que elas recebam mais de um valor. Variáveis que recebem mais de um valor possuem uma estrutura diferente de armazenamento. Esse é o conceito de estrutura de dados. Veremos três delas: **listas**, **tuplas** e **dicionários**.

3.1 LISTAS

Uma lista é um tipo de dados que pode ser escrito com valores separados por vírgulas e entre colchetes. A principal característica é que os elementos em uma lista não precisam ser do mesmo tipo. Veja os exemplos na figura 14.

```
1 lista1 = ['matemática', 'estatística', 2000, 2100]
2 lista2 = [1, 2, 3, 4, 5]
3 lista3 = [1, "a", 2, "b", 3, "c", 4, "d"]
```

Figura 14 - Exemplos de listas.
Fonte: elaborado pelo autor.

Observe que temos três listas e cada uma delas possui um conjunto diferente de dados armazenados. Os índices das listas começam em 0, assim como os índices de uma string. Logo, se fizermos **print (lista1[0])** teremos como saída o valor **matemática**. Se fizermos **print (lista1[3])** teremos a saída **2100**. Porém, se fizermos **print (lista1[4])** teremos um erro, pois não existe o item na posição 4 em nossa lista1.

Veja este outro caso. Vamos executar o comando **print (lista3[1:5])**. Desse modo, teremos os valores na tela: **a, 2, b, 3**.

Reflita

As estruturas de dados estão muito próximas de nossa realidade. Pense, por exemplo, quando você faz sua lista de compras e vai ao mercado. Na lista de compras você tem o produto que deseja comprar e, porventura, a quantidade também. Conforme você vai colocando os itens no carrinho, você vai removendo ou rabiscando os itens da lista.

3.1.1 EXCLUINDO ELEMENTO DA LISTA

Para excluir um elemento utilizaremos o comando **del**. Por exemplo, na lista 1 desejamos excluir a palavra **estatística**. Para isso, temos que fazer o comando **del lista1[1]**, afinal a palavra que desejamos remover está na posição 1 da lista1.

3.2 TUPLAS

Tuplas são semelhantes às listas, a diferença entre elas é que as tuplas são imutáveis, ou seja, não podem ser alteradas, diferentemente das listas. Outra característica é que as tuplas usam parênteses, enquanto as listas usam colchetes. Na figura a seguir você vê exemplos de tuplas heterogêneas e tupla de inteiros.

```
1 tupla1 = ('matemática', 'estatística', 2000, 2100)
2 tupla2 = (1, 2, 3, 4, 5)
3 tupla3 = (1, "a", 2, "b", 3, "c", 4, "d")
```

Figura 15 - Exemplos de tuplas.
Fonte: elaborado pelo autor.

Para acessarmos valores na tupla podemos utilizar os colchetes para fatiá-la (como com as listas). Por exemplo, para mostrarmos o valor 2000 da tupla1 podemos fazer o comando **print (tupla1[2])**.

Aprofunde-se

Neste livro, das páginas 34 a 44, você poderá se aprofundar nos conceitos de lista e tuplas.

Autor: Luiz Eduardo Borges.

Título do livro: Python para desenvolvedores

Local: Rio de Janeiro.

Editora: Edição do autor.

Data de publicação: 2010.

ISBN: 9788590945116.



3.3 DICIONÁRIO

A terceira estrutura de dados é o dicionário. Nessa estrutura, cada chave é separada de seu valor por dois pontos (:), os itens são separados por vírgulas e todo conjunto de dados é colocado entre chaves. Um dicionário vazio, sem itens, é escrito com apenas duas chaves, assim: {}. Observe o exemplo de dicionário na figura 16.

```
1  dicionario1 = {'Nome': 'Paulo', 'Idade': 20, 'Sexo': 'Masculino'}
2  print ("Nome da pessoa: ", dicionario1['Nome'])
3  print ("Idade da pessoa: ", dicionario1['Idade'])
```

Figura 16 - Exemplo de dicionário.
Fonte: elaborado pelo autor.

Note que, para apresentarmos um valor do dicionário, não digitamos o número do índice como em listas ou em tuplas, mas sim digitamos o nome da chave a qual desejamos apresentar o valor.

3.4 PERCORRENDO UMA LISTA

Podemos utilizar as estruturas de repetição **for** ou **while** para percorrer uma lista. Observe o código da figura 17, o qual apresenta uma estrutura de repetição **for**, que apresenta todos os valores da lista 1.

```
1  lista1 = ['matemática', 'estatística', 2000, 2100]
2  for item in lista1:
3      print(item)
```

Figura 17 - Percorrendo uma lista.
Fonte: elaborado pelo autor.

O código da figura 17 mostrará na tela os 4 itens da lista 1.



Assista

Acesse na plataforma o vídeo: Avançando no jogo de adivinhação

4 MANIPULAÇÃO DE MATRIZES EM NUMPY

O NumPy é um pacote de processamento de matriz de uso geral projetado para manipular eficientemente grandes matrizes multidimensionais de registros arbitrários.

Usando o NumPy, você poderá executar operações matemáticas e lógicas em matrizes, transformadas de Fourier e rotinas para manipulação de formas, e também operações relacionadas à álgebra linear. O NumPy possui funções integradas para álgebra linear e geração de números aleatórios.

4.1 INSTALANDO O NUMPY

Como o NumPy é um pacote externo, você precisa instalá-lo em seu computador. Para isso, siga os procedimentos do infográfico a seguir:

Instalando o pacote



O NumPy é um pacote que lhe permitirá manipular matrizes mais facilmente.

Desse modo, você será capaz de realizar desde operações básicas, como aritmética entre matrizes, até operações mais avançadas. Para que isso seja possível, primeiramente é necessário seguir o procedimento passo a passo para a instalação do pacote NumPy.



1. No Windows, abra o prompt de comandos em modo administrador. No Linux, **abra o terminal**.



2. No Windows, navegue até a pasta que contém o arquivo **python.exe**



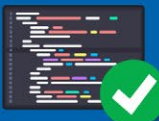
3. No Windows, digite o comando **python -m pip install --user numpy**. No Linux, digite o comando **sudo apt-get install python3-numpy**



4. Após concluir a instalação, tente importá-lo em um projeto python **import numpy as np**



5. Se **não** estiver instalado, aparecerá uma mensagem de erro contendo estes dizeres: **File "<pyshell#0>", line 1, in <module>**. Repita o processo ou procure por ajuda.



6. Se importação funcionar sem erros você deve ser capaz de executar o comando **np.log10(100)** no seu projeto, o que deve produzir como resultado o número **2.0**, afinal **$2^{10} = 100$** .

ED+ Content Hub © 2020

Infográfico 1 - Etapas para instalação do pacote NumPy.

Fonte: elaborado pelo autor.

4.2 NDARRAY

O objeto mais importante definido no NumPy é um tipo de matriz n-dimensional chamado **ndarray**. Ele descreve a coleção de itens de mesmo tipo. Os itens da coleção podem ser acessados usando um índice que se inicia em zero (como em listas ou tuplas).

Cada item em um **ndarray** tem o mesmo tamanho de bloco na memória. Cada elemento no **ndarray** é um objeto do tipo de dados (chamado **dtype**). Na figura 18 temos um exemplo de uma matriz de uma dimensão e de mais de uma dimensão.

```
1  # uma dimensão
2  import numpy as np
3  a = np.array([1,2,3])
4  print (a)
5
6  # mais de uma dimensão
7  import numpy as np
8  a = np.array([[1, 2], [3, 4]])
9  print (a)
```

Figura 18 - Exemplo de matrizes.
Fonte: elaborado pelo autor.

4.3 CRIANDO MATRIZES

Um novo objeto ndarray pode ser construído por qualquer uma das rotinas de criação de matriz a seguir ou usando um construtor ndarray de baixo nível.

4.3.1 NUMPY.EMPTY

Cria uma matriz não inicializada de forma e tipo especificados. Ele usa o seguinte construtor: **numpy.empty(forma, dtype = float, order = 'C')**, sendo: **forma** o formato de uma matriz vazia em **int** ou tupla de **int**; **dtype** o tipo de dados de saída – esse parâmetro é opcional; **order** refere-se a como será apresentada a matriz, sendo **'C'** para matriz principal de linha no estilo linguagem de programação C e **'F'** para matriz principal de coluna no estilo FORTRAN.

4.3.2 NUMPY.ZEROS

Retorna uma nova matriz de tamanho especificado, preenchida com zeros. Ele usa o seguinte construtor: **numpy.zeros(forma, dtype = float, order = 'C')**, sendo: **forma** o formato de uma matriz vazia em **int** ou sequência de **int**; **dtype** o tipo de dados de saída – esse parâmetro é opcional; **order** refere-se a como será apresentada a matriz, sendo **'C'** para matriz principal de linha no estilo linguagem de programação C e **'F'** para matriz principal de coluna no estilo FORTRAN.

A figura 19 apresenta três matrizes construídas com `numpy.zeros`.

```

1  import numpy as np
2
3  x = np.zeros(5)
4  print (x)
5
6  x = np.zeros((5,), dtype = np.int)
7  print (x)
8
9  x = np.zeros((2,2), dtype = [('x', 'i4'), ('y', 'i4')])
10 print (x)

```

Figura 19 - Matrizes criadas com `numpy.zeros`.
Fonte: elaborado pelo autor.

A figura 20 apresenta as três saídas de cada uma dessas matrizes (respectivamente).

```

[0. 0. 0. 0. 0.]
[0 0 0 0 0]
[(0, 0) (0, 0)]
[(0, 0) (0, 0)]

```

Figura 20 - Saída dos códigos da figura 19.
Fonte: elaborado pelo autor.

A primeira linha da figura 20 corresponde à matriz criada na linha 3 da figura 19; a segunda linha da figura 20 corresponde à matriz criada na linha 6 da figura 19; e a terceira e quarta linha da figura 20 correspondem à matriz criada na linha 9 da figura 19.

Aprofunde-se

Neste livro, no capítulo 4, você conhecerá os princípios do NumPy e os recursos desse pacote.

Autor: Wes McKinney.

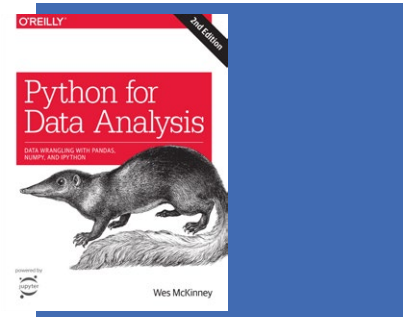
Título do livro: Python para análise de dados

Local: São Paulo.

Editora: Novatec.

Data de publicação: 2018.

ISBN: 9788575226476.



4.4 OPERAÇÕES COM MATRIZES

Em Python, podemos resolver as diferentes manipulações e operações da matriz. O NumPy Module fornece métodos diferentes para operações de matriz. Em todos os exemplos a seguir, vamos considerar que você já tenha realizado o comando **import numpy** no início do seu código.

Assista

Acesse na plataforma o vídeo: Operações com matrizes

Quando temos muitos dados a serem analisados, é mais interessante trazê-los localmente e processá-los do que realizar todo o processamento em um banco de dados. Desse modo, as matrizes podem ser grandes aliadas no arranjo e organização desses dados, o que nos facilitará as operações de análise.

CONSIDERAÇÕES FINAIS

Nesta unidade, você teve o seu primeiro contato com a poderosa linguagem Python. Aqui foquei principalmente em te mostrar os principais conceitos em relação ao Python como variáveis, estruturas condicionais e de repetição, bem como as estruturas de dados lista, tuplas e dicionários.

Você também conheceu o NumPy, um poderoso pacote do Python para manipularmos matrizes.

Realize a instalação do Python, implemente todos os códigos que estão aqui disponíveis, bem como os códigos dos vídeos e materiais de apoio.

REFERÊNCIAS

BARRY, Paul; GRIFFITHS, David. **Head first programming**. A learner's guide to programming using the Python language. Sebastopol: O'Reilly, 2009. ISBN 978-0-596-80237-0.

BORGES, Luiz Eduardo. **Python para desenvolvedores**. Rio de Janeiro: Edição do autor, 2010. ISBN 978-85-909451-1-6.

LUTZ, Mark; ASCHER, David. **Aprendendo Python**. Porto Alegre: Bookman, 2007. ISBN 978-85-7780-013-1.

Unidade 3

Análise exploratória de dados

Objetivos de aprendizagem:

Conhecer as maneiras tradicionais de encontrar informações úteis em um conjunto de dados e auxiliar na apresentação dessas informações.

Tópicos de estudo:

- Pandas e o carregamento de dados;
- Descritores estatísticos;
- Visualização gráfica com Seaborn.

Iniciando os estudos:

Os dados são o maior tesouro para uma organização, pois é por meio deles que as empresas podem realizar suas decisões estratégicas. Porém, para que você possa tomar uma decisão, primeiramente precisamos desenvolver modelos que nos ajudem a analisar esses dados.

Nesta unidade, você verá três assuntos muito interessantes. Começaremos estudando o pacote Pandas que fornece ferramentas para análise de dados e que, como você verá, é muito fácil de usar.

Para que possamos analisar os dados, é fundamental que saibamos quais métricas devemos utilizar. Desse modo, você verá sobre os quatro tipos principais de estatística descritiva.

Por fim, após conhecer a ferramenta de análise de dados e depois de conhecer a estatística descritiva, veremos o pacote de visualização de dados Seaborn. Com esse pacote, você irá gerar gráficos de linha, coluna, barra e de boxplot.

1 PANDAS E O CARREGAMENTO DE DADOS

O Pandas é um importante pacote de ferramentas à disposição dos cientistas e analistas de dados que trabalham atualmente com a linguagem Python. Com suas poderosas ferramentas de aprendizado de máquina e visualização de dados, o Pandas torna-se a espinha dorsal da maioria dos projetos de análise de dados.

Com o uso do Pandas, você pode limpar seus dados, transformá-los e analisá-los com praticidade e de modo intuitivo. Por exemplo, vamos supor que você deseja analisar um conjunto de dados que está armazenado em um arquivo Excel ou CSV. O Pandas realizará a extração desses dados para um Data-Frame (veremos logo mais sobre esse conceito). Assim que o DataFrame estiver montado, você poderá:

- Realizar cálculos estatísticos e responder perguntas como essas:
 - Qual a média de vendas de um produto X em janeiro?
 - A coluna A se correlaciona com a coluna B?
- Limpar os dados, removendo valores ausentes, ou remover alguma linha seguindo um critério específico;
- Visualizar os dados em gráficos com suporte da Matplotlib e da Seaborn;
- Criar um arquivo CSV e armazenar os dados, agora limpos, nesse arquivo.

Sendo assim, antes de partir para a tomada de decisão estratégica, você precisa conhecer seus dados e o comportamento que eles possuem.

1.1 INSTALAÇÃO DO PANDAS

O site oficial do Pandas indica que a maneira mais fácil e prática de instalá-lo é por meio da distribuição Anaconda, que é uma distribuição para análise de dados e computação científica.

O Anaconda é uma distribuição gratuita e de código aberto das linguagens de programação Python e R. A distribuição vem com o intérprete Python e vários pacotes relacionados ao aprendizado de máquina e Ciência de dados.

Instalação da distribuição



O objetivo por trás do Anaconda é facilitar, para as pessoas interessadas no campo de Ciência de dados, a instalação de todos (ou quase todos) os pacotes necessários em uma única instalação.

Veja abaixo o passo a passo para a instalação da distribuição Anaconda.



Passo 01

Acesse o site oficial da distribuição **Anaconda** em <https://www.anaconda.com/>



Passo 02

No menu, escolha por **Produtos** (Products) e, em seguida, selecione **Edição Individual** (Individual Edition).



Passo 03

Clique no botão **download** na página ou role a página até encontrar a sessão com os instaladores.



Passo 04

Escolha o instalador que melhor lhe atende (**Windows - macOS - Linux**).



Passo 05

Execute o instalador com as **configurações padrão**. Você não precisará modificar nenhum critério.

ED+ Content Hub © 2020

Infográfico 1 - Instalação da distribuição Anaconda.

Fonte: elaborado pelo autor.

Se você seguiu os passos, certamente a distribuição Anaconda já está instalada e você já pode aproveitar os pacotes nela contidos.

Desse momento em diante, nós desenvolveremos em Python utilizando o Jupyter Notebook, pois ele oferece um bom ambiente para o uso de Pandas na exploração e modelagem de dados, mas o Pandas também pode ser usado em editores de texto com a mesma facilidade.

O Jupyter Notebook permite que você execute o código em uma célula específica em vez de executar o arquivo inteiro. Isso economiza muito tempo ao trabalhar com grandes conjuntos de dados e transformações complexas. Destaca-se também a característica de fornecer uma maneira fácil de visualizar os DataFrames e as plotagens dos gráficos.

1.2 PREPARANDO O AMBIENTE PARA O PANDAS

Se você chegou até aqui, é sinal que tudo já está instalado. Agora, execute o Anaconda Navigator (no menu Iniciar, digite Anaconda Navigator). Ele apresentará uma interface como a da figura 1.

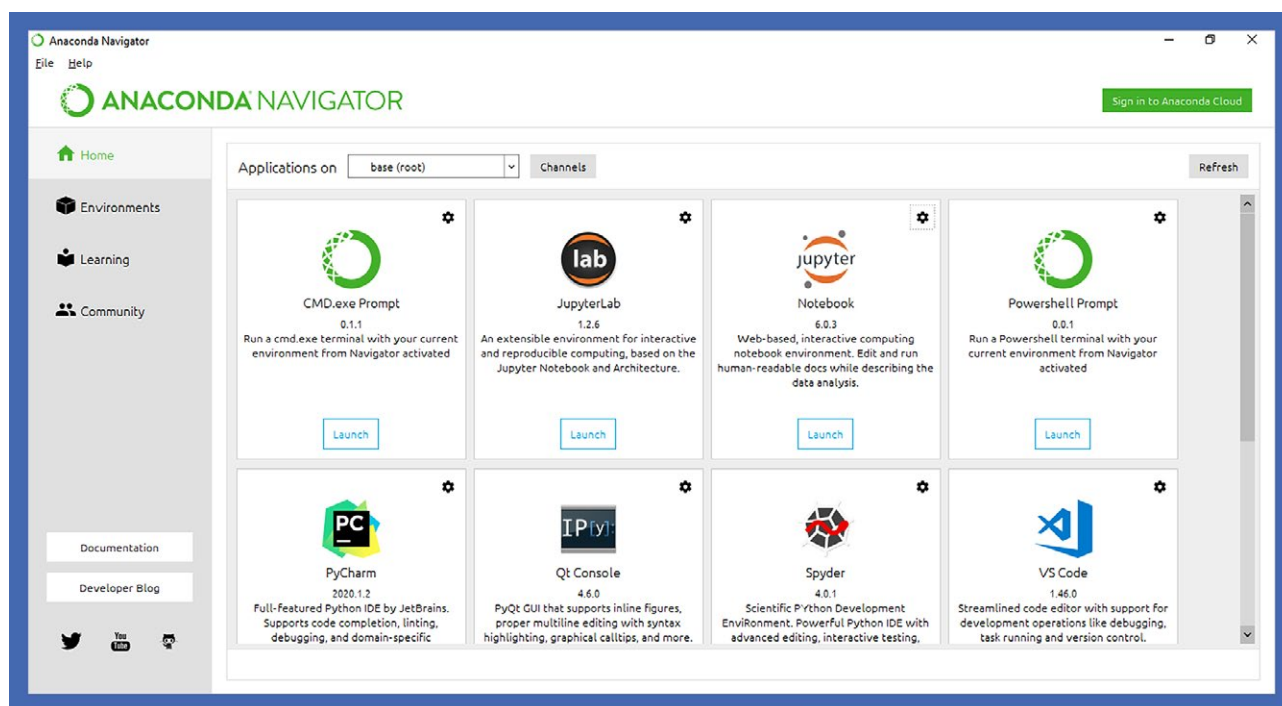


Figura 1 - Interface do Anaconda Navigator.
Fonte: elaborado pelo autor.

Agora, clique em **Launch** no terceiro item dessa interface, que é o Jupyter Notebook. Abrirá, no navegador, uma lista de diretórios. Trata-se do diretório **c:\users** ou **c:\usuarios**. Vamos criar um diretório chamado scripts. Para isso, clique em **new** e, em seguida, em folder, conforme indica a figura 2. A esse novo diretório você colocará o nome de scripts (note, poderia ter qualquer nome, porém vamos chamá-lo nesse momento de scripts).

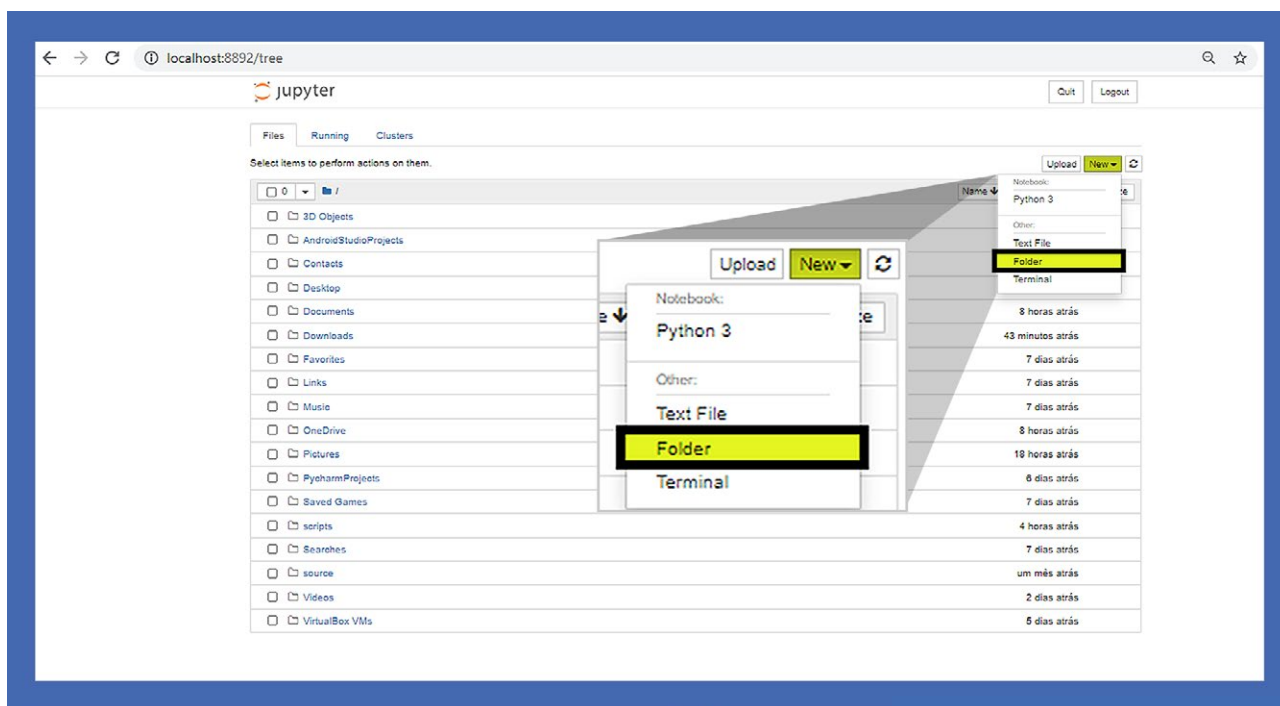


Figura 2 - Criar um novo diretório.
Fonte: elaborado pelo autor.

Agora, com o diretório criado, acesse-o e crie um novo arquivo. Siga o mesmo caminho que você utilizou para criar a pasta. Vá em **new**, no entanto, desta vez escolha a opção **Python3**. Isso criará um arquivo para nós editarmos em Python.

Saiba mais

Em termos simples, você pode pensar no Anaconda como uma maneira fácil de instalar um interpretador Python e mais outros vários pacotes de Ciência de dados. Desse modo, da próxima vez que precisar instalar o Python e seus pacotes de Ciência de dados (Pandas, NumPY, entre outros), opte diretamente por instalar o Anaconda e, depois, é só importar os pacotes em seu projeto.

Sempre que formos trabalhar com o Pandas precisamos importá-lo. Além do Pandas, também utilizaremos o pacote NumPy. Por questão de padronização, sempre que realizarmos a importação do Pandas o chamaremos de **pd** e, ao importar o NumPy, o chamaremos de **np**. A importação ficará como na figura 3 a seguir:

```
In [ ]: import pandas as pd  
import numpy as np
```

Figura 3 - Importações.
Fonte: elaborado pelo autor.

1.3 ESTRUTURA DE DADOS NO PANDAS

O Pandas trabalha com duas estruturas de dados principais: as Séries e os DataFrames.

Aprofunde-se

Saiba mais sobre Séries e DataFrames lendo os capítulos 2 e 3, que falam sobre essas duas estruturas de dados muito utilizadas em Ciência de dados.

Autor: Eduardo Corrêa.

Título do livro: Pandas Python: Data Wrangling para Ciência de dados

Local: São Paulo.

Editora: Casa do Código.

Data de publicação: 24/01/2020.

ISBN: 9788572540490.



1.3.1 SÉRIES

Pense em Séries como sendo as listas em Python e realmente é isso que as Séries são. A característica delas é ser um array unidimensional com valores indexados e esses índices são como rótulos para os elementos da lista. Veja na figura 4 dois exemplos de Séries que apresentam a quantidade de maçãs e a quantidade de laranjas.

Séries			
	Maçãs		Laranjas
0	6	0	2
1	4	1	3
2	3	2	1
3	7	3	4

Figura 4 - Exemplo de Séries.

Fonte: elaborado pelo autor.

Observe que as Séries de maçãs possuem quatro valores e cada um deles possui um índice que vai de 0 até 3. O mesmo acontece com as Séries de laranjas.

Vamos então colocar o código que representa essas Séries. Desse modo, coloque o seguinte código da figura 5 no Jupyter (após a linha de importação).

```
In [1]: import pandas as pd
import numpy as np

macas = pd.Series([6,4,3,7])
macas

Out[1]: 0    6
        1    4
        2    3
        3    7
        dtype: int64
```

Figura 5 - Código-fonte.

Fonte: elaborado pelo autor.

Observe na figura 5 que foi colocado um código na entrada **(In)** e assim obtivemos uma saída **(Out)**. As Séries de maçãs foram criadas e agora serão criadas as Séries de laranjas. Veja no código da figura 6 que não foi necessário realizar as duas importações novamente.

```
In [2]: laranjas = pd.Series([2,3,1,4])
        laranjas

Out[2]: 0    2
        1    3
        2    1
        3    4
        dtype: int64
```

Figura 6 - Código-fonte.
Fonte: elaborado pelo autor.

Ao criar as Séries, o Pandas atribuiu automaticamente o índice para cada um dos valores. Entretanto, pode ser conveniente que você atribua um valor de índice. Por exemplo, a Amanda comeu 2 maçãs, a Beatriz comeu 3 maçãs, o Carlos comeu 1 maçã e o Daniel comeu 4 maçãs. Veja como fica o código na figura 7 quando desejamos atribuir o índice.

```
In [3]: macas = pd.Series([6,4,3,7], index=["Amanda", "Beatriz", "Carlos", "Daniel"])
        macas

Out[3]: Amanda    6
        Beatriz   4
        Carlos    3
        Daniel    7
        dtype: int64
```

Figura 7 - Código-fonte.
Fonte: elaborado pelo autor.

O conteúdo continua o mesmo nas Séries, porém o índice, que antes era números, se tornou nomes. Com isso, é possível responder uma pergunta pontual como, por exemplo, quantas maçãs a Beatriz comeu? Veja o código na figura 8. Observe que foram utilizadas duas chamadas para verificar quantas maçãs a Beatriz comeu. Ambas estão corretas.

```
In [4]: macas.Beatriz  
Out[4]: 4  
  
In [5]: macas["Beatriz"]  
Out[5]: 4
```

Figura 8 - Código-fonte.
Fonte: elaborado pelo autor.

E quantas maçãs foram comidas no total? Simples, basta você dar o comando **macas.sum()**. Se quiser saber o maior número de maçãs comidas, basta dar o comando **macas.max()**. E para saber o nome da pessoa que comeu mais maçãs? Veja a resposta na figura 9.

```
In [11]: macas.idxmax()  
Out[11]: 'Daniel'
```

Figura 9 - Código-fonte.
Fonte: elaborado pelo autor.

Existem muitos outros métodos. Se você ficar em dúvida sobre algum, digite o nome da sua Série, coloque um ponto e, em seguida, pressione a tecla Tab. Automaticamente serão listados todos os métodos possíveis a serem utilizados.

1.3.2 DATAFRAMES

Se você compreendeu o conceito de Séries, certamente não terá dificuldade com DataFrames, pois eles são muito semelhantes. Pense em Séries como sendo uma coluna (como vimos na figura 4). Por outro lado, pense em DataFrames como uma tabela com várias Séries ou várias colunas. Veja o exemplo na figura 10.

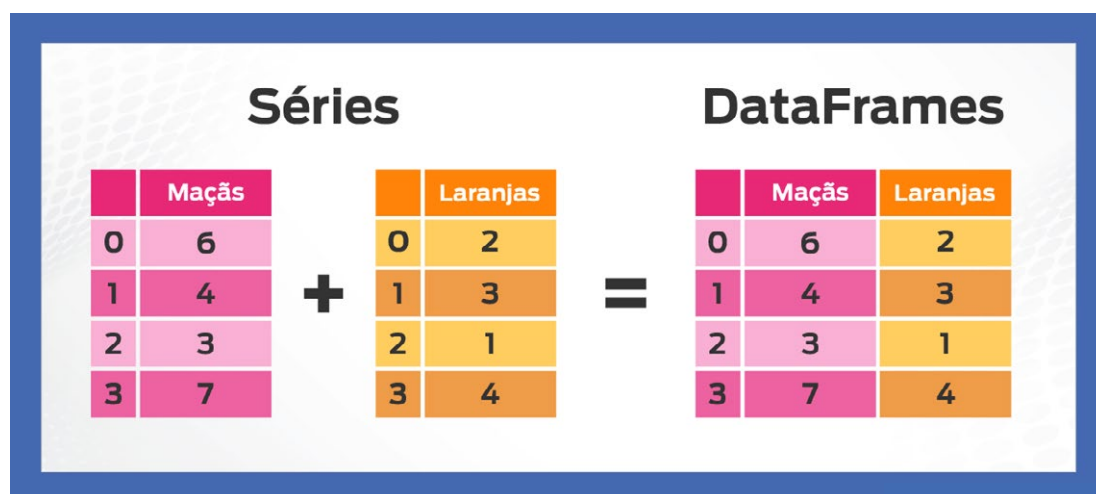


Figura 10 - DataFrames.
Fonte: elaborado pelo autor.

Muitas operações que podem ser feitas com Séries também podem ser feitas com DataFrames. Veja na figura 11 como criar um DataFrame com os dados da figura 10.

```
In [12]: df = pd.DataFrame({'Pessoa' : ["Amanda", "Beatriz", "Carlos", "Daniel"],
                             'Maçãs' : [6,4,3,7],
                             'Laranjas': [2,3,1,4]})
df
```

```
Out[12]:
```

	Pessoa	Maçãs	Laranjas
0	Amanda	6	2
1	Beatriz	4	3
2	Carlos	3	1
3	Daniel	7	4

Figura 11 - Código-fonte.
Fonte: elaborado pelo autor.

Vamos ordenar nosso DataFrame pelo número de maçãs, do menor para o maior. Veja na figura 12. Atenção: ordenar não modifica o valor dos campos, apenas reposiciona as linhas de acordo com a coluna que foi ordenada.

```
In [18]: df.sort_values(by="Maçãs")
```

```
Out[18]:
```

	Pessoa	Maçãs	Laranjas
2	Carlos	3	1
1	Beatriz	4	3
0	Amanda	6	2
3	Daniel	7	4

Figura 12 - Código-fonte.
Fonte: elaborado pelo autor.

Vamos agora supor que desejamos saber quais as pessoas que comeram mais de três maçãs e mais de duas laranjas. Para isso, podemos realizar a seguinte consulta (figura 13).

```
In [19]: df[(df["Maçãs"] > 3) & (df["Laranjas"] > 2)]
```

```
Out[19]:
```

	Pessoa	Maçãs	Laranjas
1	Beatriz	4	3
3	Daniel	7	4

Figura 13 - Código-fonte.
Fonte: elaborado pelo autor.

Observe que nessa consulta utilizamos o operador bit a bit **&** para representar o comando **and** do Python e o operador **|** para representar o comando **or** do Python. A negação é feita com o sinal **~** ao invés de **not**.

Assista

Acesse na plataforma o vídeo: Trabalhando com DataFrame

Para praticar o conteúdo desse vídeo, é necessário fazer o download dos arquivos abaixo, disponíveis em sua plataforma:

`dados.xlsx`

`un3_top1.ipynb`

2 DESCRITORES ESTATÍSTICOS

Estatísticas descritivas são coeficientes descritivos que resumem um determinado conjunto de dados, que podem ser uma representação do todo ou uma amostra de uma população. As estatísticas descritivas são divididas em medidas de frequência, medidas de tendência central, medidas de variabilidade e medidas de posição.

2.1 MEDIDAS DE FREQUÊNCIA

A função básica da estatística é organizar e resumir dados. Os dados coletados em qualquer projeto de pesquisa são apresentados de forma bruta e desorganizada. Assim, possuem pouco (ou nenhum) significado, a menos que sejam organizados ou agrupados para fornecer mais informações.

Podemos organizar os dados em população ou em uma amostra. Por exemplo, se você quiser saber a média de idade de todos os brasileiros, você calculará a média da população. Porém, a população brasileira está em torno de 209,5 milhões de pessoas. Ou seja, seria praticamente impossível pegar a idade de cada um e calcular. Além disso, até o final de toda a coleta, é possível que muitos morram e muitos nasçam, o que mudará o resultado final. A medida que caracteriza uma população inteira é chamada de parâmetro.

Para isso nós podemos pegar uma amostra da população. Você já deve ter visto as pesquisas eleitorais. Os institutos de pesquisa não ouvem os 209,5 milhões de brasileiros, porém ouvem uma amostra dessa população, que se trata de uma parcela significativa dela. A medida que caracteriza uma amostra é chamada de estatística.

Temos aqui uma tabela chamada **dados.xlsx** que representa as 600 vendas da empresa YLSports durante toda a sua existência (ou seja, trata-se da população).

Veja a tabela 1 a seguir. Ela contempla os 16 primeiros dados da tabela **dados.xlsx**. Ela representa uma amostra dos dados dessa tabela.

Cliente	Região	Mês	Valor Compra
Cliente 4	Nordeste	Junho	1164
Cliente 1	Centro-Oeste	Junho	1517
Cliente 5	Nordeste	Setembro	2993
Cliente 10	Sudeste	Março	1490
Cliente 13	Sul	Novembro	2087
Cliente 7	Norte	Janeiro	2585
Cliente 14	Sul	Março	3510
Cliente 11	Sudeste	Outubro	1325
Cliente 15	Sul	Setembro	2077
Cliente 8	Norte	Janeiro	2759
Cliente 9	Norte	Dezembro	3536
Cliente 2	Centro-Oeste	Novembro	1482
Cliente 7	Norte	Março	2506
Cliente 13	Sul	Janeiro	3474
Cliente 6	Nordeste	Abril	2578
Cliente 12	Sudeste	Agosto	3291

Tabela 1 - Vendas da empresa YLSports.
Fonte: elaborado pelo autor.

Vamos calcular com o Pandas nessa tabela a frequência de vezes que aparecem os clientes. Veja o código na figura 14.

```
In [31]: df["cliente"].value_counts().to_dict()
Out[31]: {'cliente 13': 2,
          'cliente 7': 2,
          'cliente 14': 1,
          'cliente 12': 1,
          'cliente 11': 1,
          'cliente 8': 1,
          'cliente 2': 1,
          'cliente 9': 1,
          'cliente 1': 1,
          'cliente 15': 1,
          'cliente 4': 1,
          'cliente 10': 1,
          'cliente 5': 1,
          'cliente 6': 1}
```

Figura 14 - Código-fonte.
Fonte: elaborado pelo autor.

Logo, temos que os clientes 13 e 7 realizaram 2 compras. Neste intervalo de dados da tabela 1, quantas compras acima de R\$2300,00 foram realizadas? Vamos lá calcular! Veja o código da figura 15.

```
In [55]: df[df["Valor Compra"] > 2300].Cliente.value_counts().sum()  
Out[55]: 9
```

Figura 15 - Código-fonte.
Fonte: elaborado pelo autor.

Este comando talvez seja o mais complexo visto até o momento, então vou explicar bem detalhadamente. A nossa primeira ação é **df["Valor Compra"] > 2300**. Nesse caso, estamos trazendo da tabela todos os valores que sejam maiores que 2300 e armazenando em um DataFrame chamado **df**. O próximo passo é listar os clientes, então fazemos **df[...].Cliente**. Desse modo, serão recuperados todos os clientes desse novo DataFrame que possuem as compras acima de 2300. Nesse novo DataFrame nós teremos alguns clientes, mas quantos? O método **value_counts()** vai contar quantos clientes estão presentes nesse novo DataFrame. Por fim, o método **sum()** é responsável por realizar a soma do total de vendas para cada cliente, que nada mais é do que o cálculo da frequência.

Para que você compreenda toda a execução, execute por partes as consultas realizadas e acompanhe cada saída.

2.2 MEDIDAS DE TENDÊNCIA CENTRAL

Pense na medida de tendência central como a tendência dos dados em se agruparem em torno de um valor médio. Na estatística, as três medidas mais comuns de tendência central são: a média, mediana e a moda. Cada uma dessas medidas calcula a localização do ponto central usando um método diferente, que será visto a seguir.

2.2.1 MÉDIA

A média é a medida da tendência central que você está mais familiarizado. Ela, assim como as demais que você verá, descreve uma amostra inteira com um único número que representa o centro dos dados. Você calcula a média somando todas as observações e depois dividindo o total pelo número de observações:

$$\bar{x} = \frac{x_1 + x_2 + \dots + x_n}{n} = \frac{1}{n} \sum_{i=1}^n x_i$$

Na figura 16, é calculada a média de vendas em reais do período apresentado na tabela 1.

```
In [58]: df.mean()
Out[58]: Valor Compra    2398.375
         dtype: float64
```

Figura 16 - Código-fonte.
Fonte: elaborado pelo autor.

Reflita

O cálculo da média incorpora todos os valores dos dados da distribuição. Suponha que você altere qualquer valor, você verá que a média será alterada. No entanto, a média nem sempre localiza o centro dos dados com precisão.

2.2.2 MODA

A moda é o valor que ocorre com mais frequência no seu conjunto de dados. Em um gráfico de barras a moda é a barra mais alta. Se os dados tiverem vários valores vinculados ocorrendo com mais frequência, você terá uma distribuição multimodal. Se nenhum valor se repetir, os dados não terão uma moda e serão denominados distribuição amodal. Na figura 17, é calculada a moda de vendas em reais do período apresentado na tabela 1.

```
In [66]: df["Valor Compra"].mode()

Out[66]: 0    1164
         1    1325
         2    1482
         3    1490
         4    1517
         5    2077
         6    2087
         7    2506
         8    2578
         9    2585
        10    2759
        11    2993
        12    3291
        13    3474
        14    3510
        15    3536
        dtype: int64
```

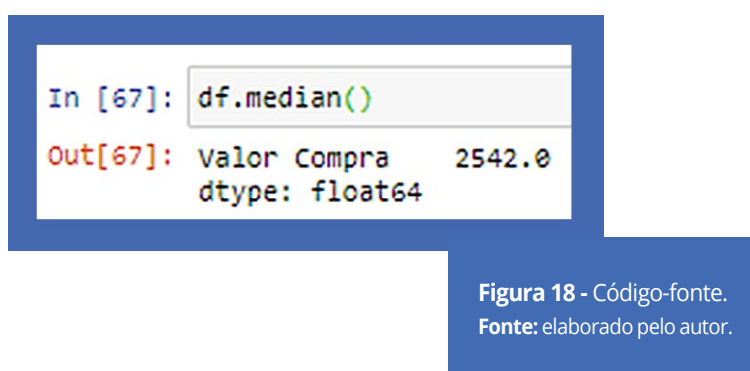
Figura 17 - Código-fonte.

Fonte: elaborado pelo autor.

Observe que foram retornados 15 dos 16 dados. Logo, você pode considerar essa amostra da tabela 1 como multimodal.

2.2.3 MEDIANA

A mediana é o valor do meio, logo ela divide o conjunto de dados pela metade. Para encontrar a mediana, ordene seus dados do menor para o maior e, em seguida, encontre o ponto de dados que tenha uma quantidade igual de valores acima e abaixo. Se a amostra tiver uma quantidade par de itens (como é o caso da tabela 1), você pegará os dois valores do meio e encontrará a média desses dois valores centrais. Na figura 18, é calculada a mediana de vendas em reais do período apresentado na tabela 1.



2.3 MEDIDAS DE VARIABILIDADE

As medidas de variabilidade, também chamadas de medidas de propagação, ajudam a analisar variabilidade da distribuição para um conjunto de dados. Por exemplo, embora as medidas de tendência central possam dar a uma pessoa a média de um conjunto de dados, elas não descrevem como os dados são distribuídos dentro do conjunto. Portanto, embora a média das vendas possa ser R\$2398,37, ainda há valores muito menores (por exemplo R\$1164,00 e R\$1325,00) e maiores (por exemplo R\$3536,00 e R\$3510,00).

Como medidas de variabilidade, você conhecerá sobre a variância, desvio padrão e máximos e mínimos.

2.3.1 VARIÂNCIA

A variância é a diferença quadrática média dos valores da média. Em outras palavras, ela expressa quantos dados da amostra (ou da população) estão afastados da média. Para o cálculo da variância, você considerará um conjunto de diferenças entre os pontos de dados e a média ao quadrado, soma-os e depois divide pelo número de observações. Portanto, é a **diferença quadrática média**. A figura 19 calcula a variância da amostra da tabela 1.

```
In [70]: df.var()
Out[70]: valor Compra    690914.516667
dtype: float64
```

Figura 19 - Código-fonte.

Fonte: elaborado pelo autor.

Observe que o valor da variância foi aproximadamente 690914. Quanto maior for o valor da variância, mais distantes os valores estão da média e, nesse caso, os valores estão muito distantes da média. Os dados da variância são difíceis de analisar. Para tanto, temos o desvio padrão para facilitar a análise.

Saiba mais

Como os cálculos usam as diferenças ao quadrado, a variância está nas unidades ao quadrado e não nas unidades originais dos dados. Enquanto valores mais altos da variância indicam maior variabilidade, não há interpretação intuitiva para valores específicos. Apesar dessa limitação, é importante você saber que vários testes estatísticos usam a variação em seus cálculos.

2.3.2 DESVIO PADRÃO

O desvio padrão é a diferença padrão ou típica entre cada ponto de dados e a média. Quando os valores em um conjunto de dados são agrupados mais próximos, você tem um desvio padrão menor. Por outro lado, quando os valores estão mais dispersos, o desvio padrão é maior porque a distância padrão também é maior.

O desvio padrão usa as unidades originais dos dados, o que facilita a interpretação. Consequentemente, o desvio padrão é a medida de dispersão mais amplamente usada. Para calcular o desvio padrão, devemos encontrar a raiz quadrada da variância. A figura 19 calcula o desvio padrão da amostra da tabela 1.

```
In [72]: df.std()
Out[72]: valor Compra    831.212678
dtype: float64
```

Figura 20 - Código-fonte.

Fonte: elaborado pelo autor.

O desvio padrão também foi alto, o que significa que nossos dados estão muito dispersos na amostra (quando forem apresentados os gráficos, o conceito de dispersos ficará mais claro).

Como o desvio padrão mostra a dispersão dos dados, ele será sempre maior ou igual a 1, não havendo possibilidade de ser igual ou menor do que zero.

2.3.3 AMPLITUDE, MÁXIMOS E MÍNIMOS

A amplitude é a medida mais fácil de se entender e de ser calculada porque é a mais direta entre os cálculos da variabilidade. A amplitude de um conjunto de dados é a diferença entre os valores máximos e mínimos desse conjunto. Por exemplo, considere o conjunto de dados $A=\{5,16,15,15,12\}$ e o conjunto de dados $B=\{84,92,51,66,79\}$. A amplitude do conjunto A é $16-5=11$, enquanto a amplitude do conjunto B é $92-51=41$. Para calcular a amplitude da amostra da tabela 1, vamos primeiramente encontrar os máximos e mínimos. Veja na figura 21 os cálculos citados.

```
In [74]: df["Valor Compra"].max()
Out[74]: 3536

In [75]: df["Valor Compra"].min()
Out[75]: 1164

In [76]: df["Valor Compra"].max() - df["Valor Compra"].min()
Out[76]: 2372
```

Figura 21 - Código-fonte.
Fonte: elaborado pelo autor.

Embora a amplitude seja fácil de entender, ela se baseia apenas nos dois valores mais extremos do conjunto de dados, o que a torna muito suscetível a discrepâncias. Se um desses números for extraordinariamente alto ou baixo, isso afeta o intervalo inteiro, mesmo que seja atípico.

Aprofunde-se

Saiba mais sobre estatística descritiva lendo este livro da página 41 até a página 84. Nelas, o autor apresenta uma análise mais profunda sobre distribuição de frequência, medidas de tendência central e medidas de dispersão.

Autor: Nelson Pereira Castanheira.

Título do livro: Estatística aplicada a todos os níveis

Local: Curitiba.

Editora: Ibpx.

Data de publicação: 21/07/2017.

ISBN: 9788578380199.



2.3.4 QUARTIL

Quartis são valores que dividem seus dados em quatro segmentos chamados de quartos, de acordo com o local em que os números caem na linha numérica. Os quatro quartos que dividem um conjunto de dados em quartis são:

- Os 25% menores números;
- Os próximos 25% menores números (até a mediana);
- Os 25% maiores números acima da mediana;
- Os 25% maiores números.

Veja um exemplo para entender melhor como dividir um conjunto de dados em quartis.

Considere o seguinte conjunto $A = \{2, 5, 6, 7, 10, 22, 13, 14, 16, 65, 45, 12\}$ e divida esse conjunto em quartis.

Etapa 1: coloque os valores do conjunto em ordem crescente:

$A = \{2, 5, 6, 7, 10, 12, 13, 14, 16, 22, 45, 65\}$

Etapa 2: agora conte quantos números existem no seu conjunto e divida por 4 para cortar a lista de números em quartos. Existem 12 números nesse conjunto, então teremos 3 números em cada quartil conforme apresenta o quadro 1:

2, 5, 6	7, 10, 12	13, 14, 16	22, 45, 65
---------	-----------	------------	------------

Quadro 1 - Quartis do conjunto de dados.
Fonte: elaborado pelo autor.

Então, resumidamente podemos dizer:

- Quartil 1 (0.25) = $(6 + 7) / 2 = 6,5$
- Quartil 2 (mediana ou 0.50) = $(12 + 13) / 2 = 12,5$
- Quartil 3 (0.75) = $(16 + 22) / 2 = 19$

Agora, vamos calcular o primeiro, segundo e terceiro quartil dos dados da tabela 1. Veja na figura 22 como é feito utilizando a função **quantile()**. Observe que o valor do segundo quartil (2542) é o mesmo da mediana (2542).

```
In [95]: df["Valor Compra"].quantile(q=0.25)
Out[95]: 1510.25

In [96]: df["Valor Compra"].quantile(q=0.50)
Out[96]: 2542.0

In [97]: df["Valor Compra"].quantile(q=0.75)
Out[97]: 3067.5

In [98]: df["Valor Compra"].median()
Out[98]: 2542.0
```

Figura 22 - Código-fonte.
Fonte: elaborado pelo autor.

Assista

Acesse na plataforma o vídeo: Estatística com Python e Pandas

Para praticar o conteúdo desse vídeo, é necessário fazer o download dos arquivos abaixo, disponíveis em sua plataforma:

dados.xlsx

un3_top2.ipynb

3 VISUALIZAÇÃO GRÁFICA COM SEABORN

Seaborn é uma biblioteca de visualização de dados Python baseada na Matplotlib. Ela fornece uma interface de alto nível para desenhar gráficos estatísticos atraentes e informativos.

Antes de mais nada, é importante realizar a importação dos pacotes do NumPy, Pandas, Seaborn e da Matplotlib. Utilizaremos as denominações padrões, inclusive é recomendado que sempre as mantenha. Veja a importação na figura 23.

```
In [100]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

Figura 23 - Código-fonte.
Fonte: elaborado pelo autor.

Aprofunde-se

Aprofunde-se mais no Seaborn e nos diversos comandos para plotagem de gráfico. Neste livro, das páginas 268 a 284 você aprofundará os estudos sobre os tipos de gráficos do Pandas com Seaborn.

Autor: Wes McKinney.

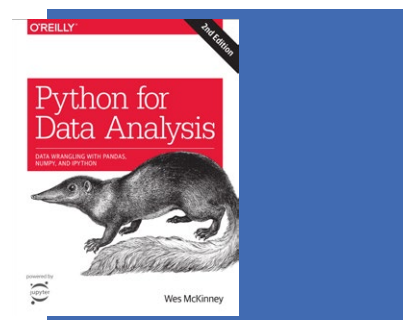
Título do livro: Python para análise de dados: tratamento de dados com Pandas, NumPy e IPython

Local: São Paulo.

Editora: Novatec.

Data de publicação: 25/09/2017.

ISBN: 9788575227510.



3.1 GRÁFICO DE DISPERSÃO

O gráfico de dispersão é um dos pilares da visualização estatística. Ele descreve a distribuição conjunta de duas variáveis usando uma nuvem de pontos em que cada ponto representa uma observação no conjunto de dados. Essa representação permite que o olho deduza uma quantidade substancial de informações sobre a existência de algum relacionamento significativo entre eles. Veja na figura 24 o código e o gráfico gerado.

```
In [111]: sns.scatterplot(x='Região',y='Valor Compra',data=df)  
Out[111]: <matplotlib.axes._subplots.AxesSubplot at 0x1bca81c1408>
```

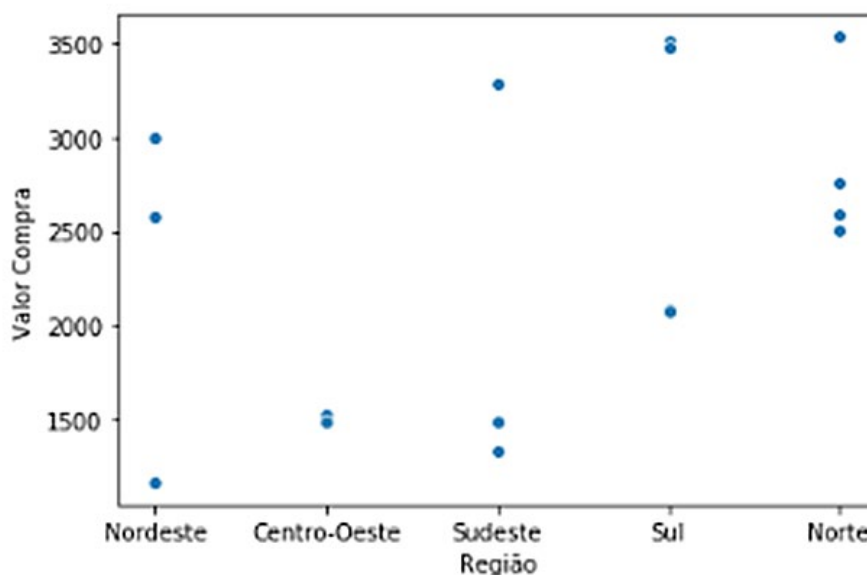


Figura 24 - Código-fonte e gráfico de dispersão.
Fonte: elaborado pelo autor.

Pelo gráfico podemos ver o motivo do desvio padrão (que foi calculado anteriormente) apresentar valor tão alto. Os valores estão fortemente dispersos. Observe também que podemos tirar conclusões melhores olhando o gráfico de dispersão se comparado a analisar uma tabela. Por exemplo, a região Norte concentra as compras acima de R\$2400,00 e a região Centro-Oeste possui as compras centradas aproximadamente em R\$1500,00.

Vamos adicionar alguns parâmetros a esse gráfico. Veja na figura 25 a nova apresentação. Nela, colocamos a matiz que apresenta, em uma escala de cor, os valores comprados. Observe que quanto mais escuro, maior foi o valor da compra.

```
In [122]: sns.scatterplot(x='Região',y='Cliente',hue='Valor Compra',data=df)
```

```
Out[122]: <matplotlib.axes._subplots.AxesSubplot at 0x1bca879f3c8>
```

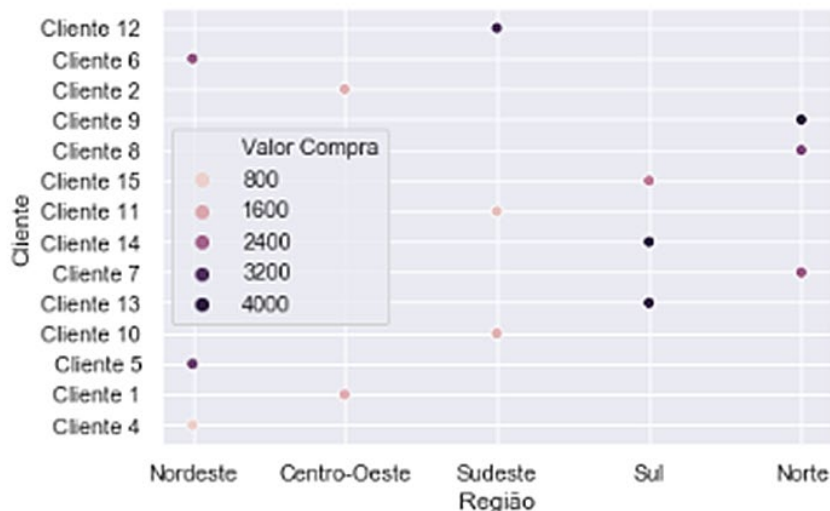


Figura 25 - Código-fonte e gráfico de dispersão.
Fonte: elaborado pelo autor.

3.2 GRÁFICO DE LINHA

Os gráficos de dispersão são altamente eficazes, mas não há um tipo de visualização universalmente ideal. Em vez disso, a representação visual deve ser adaptada às especificidades do conjunto de dados e à pergunta que você está tentando responder com o gráfico.

Neste subtópico, o gráfico será o de linha e utilizaremos um DataFrame formado por 100 números aleatórios. No código da figura 26, a primeira linha gera o DataFrame com os números e a segunda linha gera o gráfico. Observe que a única mudança em relação ao gráfico de dispersão foi trocar o tipo de gráfico no parâmetro `kind`.

```
In [139]: df = pd.DataFrame(dict(time=np.arange(100), value=np.random.randn(100).cumsum()))  
g = sns.relplot(x="time", y="value", kind="line", data=df)
```

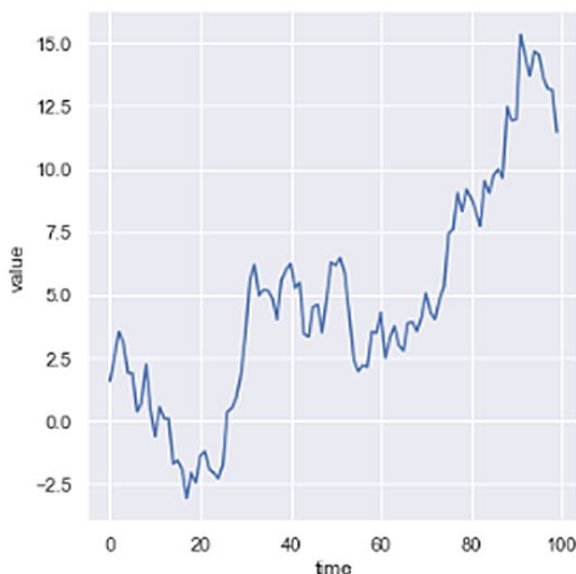


Figura 26 - Código-fonte e gráfico de linhas.

Fonte: elaborado pelo autor.

3.3 BOXPLOT

Boxplots são também chamados de gráfico de caixa ou whisker plot e fornecem uma boa imagem gráfica da concentração dos dados. Eles também mostram a qual distância os valores extremos estão da maioria dos dados. Um gráfico de caixa é construído a partir de cinco valores: o valor mínimo (menor valor), o primeiro quartil, a mediana (ou segundo quartil), o terceiro quartil e o valor máximo (maior valor). Usamos esses valores para comparar quão próximos os outros valores de dados estão deles.

Na figura 27, são apresentados o código bem como o gráfico do boxplot referentes aos dados da tabela 1.

```
In [144]: sns.catplot(x="Região",y="Valor Compra",kind='box',data=df)
```

```
Out[144]: <seaborn.axisgrid.FacetGrid at 0x1bcaa6c2f88>
```

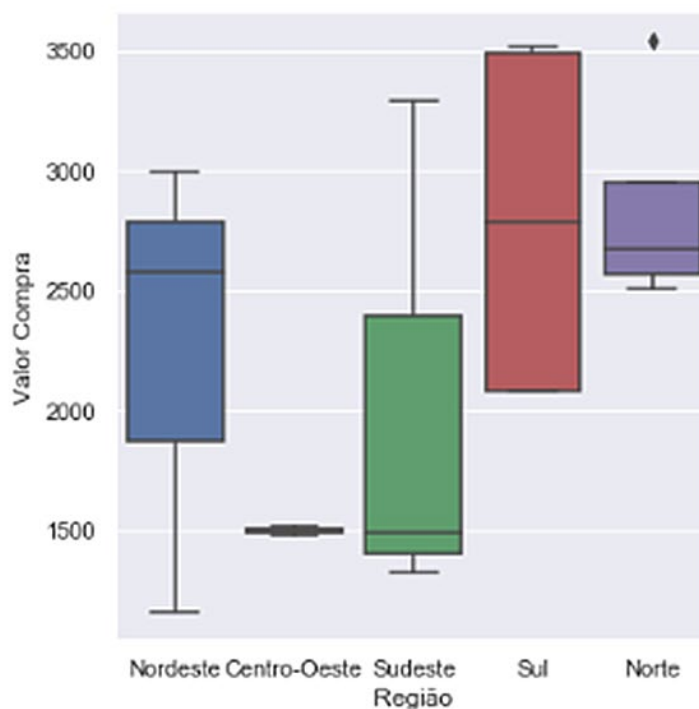


Figura 27 - Código-fonte e gráfico boxplot.
Fonte: elaborado pelo autor.

Observe, por exemplo, que na região Nordeste os dados se concentram entre R\$1900,00 a R\$2750,00, tendo a máxima em R\$3000,00 e a mínima próximo a R\$0,00. Vemos também que a mediana (a linha que corta no meio o gráfico de caixa) está próxima a R\$2600,00.

Assista

Acesse na plataforma o vídeo: Visualização de dados com Pandas e Seaborn

Para praticar o conteúdo desse vídeo, é necessário fazer o download dos arquivos abaixo, disponíveis em sua plataforma:

[dados.xlsx](#)

[un3_top3.ipynb](#)

Existem outros gráficos que podemos utilizar como gráficos de barras, histogramas, gráficos de violino, entre outros, porém, conforme você for avançando nos estudos, será apresentado a eles.

CONSIDERAÇÕES FINAIS

Nesta unidade, você conheceu três tópicos importantes quanto ao assunto análise de dados. Começamos abordando o pacote Pandas e vendo os conceitos de Séries e DataFrames. Você pôde realizar algumas implementações por meio de comandos e conhecer um pouco do poder desse pacote.

Em seguida, você conheceu os descritores estatísticos como frequência, medidas de tendência central e medidas de variabilidades. Cada um desses descritores pode ser aplicado em momentos específicos de acordo com a necessidade da análise.

Por fim, você conheceu o Seaborn, um pacote para Python que permite visualizar os dados a partir de um array. No nosso caso, você viu a análise em um DataFrame, entretanto não muda quando o assunto é analisar uma Série.

GLOSSÁRIO

R: é uma linguagem de programação que possui um extenso catálogo de métodos estatísticos e gráficos. O **R** não é utilizado apenas por acadêmicos, mas muitas grandes empresas também usam essa linguagem de programação, incluindo Uber, Google, Airbnb, Facebook, entre outras.

REFERÊNCIAS

BARRY, Paul; GRIFFITHS, David. **Head first programming**. A learner's guide to programming using the Python language. Sebastopol: O'Reilly, 2009. ISBN 978-0-596-80237-0.

BORGES, Luiz Eduardo. **Python para desenvolvedores**. Rio de Janeiro: Edição do autor, 2010. ISBN 978-85-909451-1-6.

BOSCHETTI, Alberto.; MASSARON, Luca. **Python data science essentials**. Birmingham: Packt Publishing, 2016.

LUTZ, Mark; ASCHER, David. **Aprendendo Python**. Porto Alegre: Bookman, 2007. ISBN 978-85-7780-013-1.

MCKINNEY, Wes. **Python para análise de dados: tratamento de dados com Pandas, NumPy e IPython**. São Paulo: Novatec, 2017.

Unidade 4

Aprendizagem automática

Objetivos de aprendizagem:

Conhecer os conceitos e os paradigmas dos diferentes tipos de aprendizagem automática, bem como explorar o método baseado em distância chamado de K-Vizinhos Próximos (ou KNN).

Tópicos de estudo:

- Paradigmas de aprendizado;
- Algoritmo K-Vizinhos Próximos (KNN);
- Métricas e Matriz de Confusão.

Iniciando os estudos:

Gostaria que você pensasse que a Ciência de dados é uma enorme caixa de ferramentas (toolbox) que, quando aberta, tem dentro dela compartimentos de ferramentas estatísticas, de bancos de dados, de machine learning e de programação, no nosso caso, a linguagem de programação Python. A sabedoria está em escolher as ferramentas corretas para executar uma tarefa que pode ser um relatório, uma análise aprofundada de dados, uma tentativa de descobrir insights, tudo depende da necessidade.

Agora, você vai entender quais são os principais paradigmas de aprendizado em Inteligência Artificial e quando empregá-los para extrair conhecimento dos dados. Depois, irá aprender como agrupar dados com o algoritmo dos Vizinhos Próximos e, por fim, compreenderá o que é uma Matriz de Confusão – o nome não está errado e não tem nada de confuso, como você verá.

Bons estudos!

1 PARADIGMAS DE APRENDIZADO

O paradigma é um conjunto de padrões relacionados a um conceito e que serve como modelo. Você já deve ter ao menos ouvido falar dos paradigmas de linguagem de programação: estruturada, orientada a objetos, funcional, lógica, entre outras.

Já o aprendizado de máquina (ou machine learning) tem como premissa principal a aprendizagem a partir dos dados (MITCHELL, 1997). Dessa forma, procura-se inferir um conceito mais geral a partir da observação de dados e exemplos passados. Quando se fala em Ciência de dados, há basicamente três paradigmas do aprendizado de máquina que são importantes conhecer: o aprendizado supervisionado, o aprendizado não supervisionado e o aprendizado por reforço.

1.1 APRENDIZADO SUPERVISIONADO

No aprendizado supervisionado, tendo como base um conjunto de dados, o algoritmo procura **inferir o valor da variável dependente a partir de um conjunto de variáveis independentes**. Confira na tabela 1 a seguir:

Variáveis independentes	Variável dependente
Idade do motorista, cidade, carro, ano de fabricação	Risco de acidente
Local, tamanho da casa, área construída	Valor do imóvel
Temperatura, tosse, dor no corpo, dor de cabeça	Diagnóstico de resfriado
Texto de review de filmes de cinema, vendas de bilheteria, número de pessoas que assistiram ao filme	Pontuação de popularidade

Tabela 1 - Exemplos de relação de dependência entre variáveis.
Fonte: elaborado pelo autor.

Após esses exemplos, acredito que você consiga pensar em uma série de outros em que exista essa dependência de determinação de uma variável em função de outras.

Como você pode notar, a característica básica do aprendizado supervisionado é possuir variáveis independentes e uma única variável dependente cujo valor tenha relação direta com as variáveis independentes. Normalmente, a variável dependente também é chamada de **rótulo**.

Um conjunto de dados rotulados pode ser visto na tabela 2 abaixo, na qual a decisão de **Pedalar** (andar de bicicleta) depende das demais variáveis (**Tempo**, **Temperatura** e **Vento**). A variável **Dia** é apenas um identificador de cada linha.

Dia	Tempo	Temperatura	Vento	Pedalar
D1	Sol	Calor	Fraco	Sim
D2	Sol	Calor	Forte	Não
D3	Nublado	Calor	Fraco	Sim
D4	Chuva	Amena	Fraco	Sim
D5	Chuva	Frio	Fraco	Não
D6	Chuva	Frio	Forte	Não
D7	Nublado	Frio	Forte	Sim
D8	Sol	Amena	Fraco	Não
D9	Sol	Frio	Forte	Não
D10	Chuva	Amena	Fraco	Sim
D11	Sol	Amena	Forte	Sim
D12	Nublado	Amena	Forte	Sim
D13	Nublado	Calor	Fraco	Sim
D14	Chuva	Amena	Forte	Não

Tabela 2 - Exemplo de dados rotulados - Pedalar.
Fonte: elaborado pelo autor.

Existem muitos algoritmos de aprendizado de máquina supervisionado para esse tipo de problema: a regressão linear, a regressão logística, as redes neurais artificiais, a máquina de suporte vetorial (ou máquinas kernel), as Árvores de Decisão, os Vizinhos Próximos e o Naive Bayes.

1.2 APRENDIZADO NÃO SUPERVISIONADO

Se no aprendizado supervisionado há os dados rotulados, então, é razoável pensar que no aprendizado não supervisionado os dados não estejam anotados? Sim! Esta é a resposta correta. No aprendizado não supervisionado, existem somente as variáveis independentes que descrevem os dados, como a tabela 3, na qual há informações de navegação dos usuários em um e-commerce.

Data	Hora	Plataforma	Sistema	Origem	Produto	Valor
12/05/2020	11h45	Smartphone	Android 10	E-mail marketing	Camiseta	R\$ 80,00
12/05/2020	02h09	Smartphone	Android 7	-	Camiseta	R\$ 59,00
14/06/2020	18h32	Computador	Ubuntu 19	Google	Calça	R\$ 120,00
01/07/2020	14h02	Computador	Windows 10	E-mail marketing	Boné	R\$ 25,00

Tabela 3 - Informações de navegação num e-commerce.
Fonte: elaborado pelo autor.

Note que não há informações sobre os rótulos nas linhas de forma a categorizar cada cliente. No aprendizado não supervisionado, o algoritmo de aprendizado de máquina deve procurar por grupos (também chamados de clusters) que tenham as mesmas características com base nos valores das variáveis contidas no conjunto de dados.

No exemplo da tabela 3, supondo que houvesse milhares de linhas nesse conjunto de dados, seria possível determinar clusters de clientes que tiveram o mesmo perfil de compra em termos de produtos, gastos, horários etc.

1.3 APRENDIZADO SUPERVISIONADO VS. APRENDIZADO NÃO SUPERVISIONADO

Como você viu no aprendizado de máquina, os exemplos na forma de um conjunto de dados possuem um rótulo que é a variável dependente. O sistema aprende a diferenciar os resultados das variáveis dependentes a partir dos valores das variáveis independentes. Disso decorre que, dado um novo exemplo desconhecido e com base no que foi aprendido, o algoritmo consegue prever qual será o rótulo daquele exemplo.

Aprofunde-se

Este livro é um clássico de aprendizado de máquina. Embora seja um título em inglês, é de fácil leitura e lhe encoraja a ler e estudar essa indicação. Embora pareça uma literatura antiga, serve para mostrar que estamos há anos ainda somente “arranhando” a superfície da Inteligência Artificial.

Título do livro: Machine learning

Autor: Tom Mitchell.

Local: Porto Alegre.

Editora: Bookman.

Data de publicação: 1997.

ISBN: 0070428077.



Já no aprendizado não supervisionado, os exemplos não contêm os rótulos e o algoritmo é capaz de agrupá-los por semelhança com base nas variáveis, que são todas independentes. Dado um novo exemplo desconhecido, o algoritmo consegue determinar a qual grupo esse novo exemplo pertence, embora os agrupamentos não tenham um rótulo explícito que os defina.

Na figura 1 abaixo, você pode observar que um conjunto de bolas pode ser dado a um algoritmo supervisionado se for possível saber os rótulos de cada uma das bolas (de tênis, de baseball ou de futebol americano). Com isso, o algoritmo aprende as características que levam à classificação (rotulação) delas e, dada uma bola que não se sabe previamente o tipo, o algoritmo consegue determinar (predizer) qual o seu tipo.

Já no aprendizado não supervisionado, como as bolas não estão rotuladas, o algoritmo irá agrupá-las pela semelhança de suas características e, dada uma nova bola, o algoritmo irá agrupá-la com um conjunto que lhe pareça ter as mesmas características, mas sem dizer que cada bola é de um determinado tipo, porque não lhe foi ensinado isso.

Aprendizado supervisionado



Bola de futebol americano

Sabendo-se os rótulos, o aprendizado de máquina aprende as **características com os rótulos**.



Aprendizado não supervisionado



Não se sabendo os tipos, o aprendizado de máquina **agrupa por semelhança**.

Figura 1 - Comparativo de aprendizado supervisionado vs. aprendizado não supervisionado.

Fonte: elaborado pelo autor.

1.4 APRENDIZADO POR REFORÇO

O aprendizado por reforço, em aprendizado de máquina, deriva diretamente de uma das teorias de aprendizagem criada pelo psicólogo behaviorista Burrhus Frederic Skinner. Nessa teoria de aprendizagem, o elemento-chave é o estímulo-resposta, no qual um agente produz respostas comportamentais a depender de estímulos positivos ou negativos, os quais, por sua vez, vão gerar novas respostas em um ciclo (MOREIRA, 2011).

Aprofunde-se

Conheça um exemplo de aprendizagem por reforço empregado no famoso jogo chinês chamado de Go.

Título: AlphaGo - Full documentary

Acesso em: 17/07/2020.

Disponível em: <https://youtu.be/WXuK6gekU1Y>

No computador, o aprendizado por reforço pode ser implementado a partir de um cenário em que há ações possíveis a serem tomadas, mas não se sabe ao certo a combinação das ações que levam ao sucesso de uma tarefa.

Reflita

Tendo como base o exemplo do veículo autônomo, e considerando que na Aprendizagem por Reforço deva existir um agente, as ações e os estímulos (recompensa ou punição), quais outros exemplos você consegue imaginar?

Por exemplo, em uma rodovia, um carro autônomo com uma Inteligência Artificial pode tomar várias decisões sobre a condução do veículo. Cada uma delas é avaliada em relação a manter o veículo na rodovia sem causar colisão com outros veículos e, a depender dessa avaliação, um reforço positivo (recompensa) ou negativo (penalização) é dado à decisão do sistema.

Assista

Acesse na plataforma o vídeo: Paradigmas de aprendizado

2 ALGORITMO K-VIZINHOS PRÓXIMOS (KNN)

Espero que até aqui você tenha compreendido a diferença entre os tipos de algoritmos de aprendizado de máquina. Vamos iniciar o estudo dos algoritmos por um de **AM** supervisionado: o **KNN** (K-Nearest Neighbors), conhecido como K-Vizinhos (mais) Próximos. Mas antes é necessário entender outro conceito fundamental em aprendizado de máquina supervisionado, que será visto a seguir.

2.1 TREINAMENTO E TESTE

Conforme visto, os exemplos de treinamento são fornecidos de maneira tabular em um conjunto de dados, que muitas vezes está na forma de planilhas, arquivos CSV, tabelas de bancos de dados etc.

Pensando no modo como um algoritmo de **ML** supervisionado funciona, como é possível avaliar o quão bom foi o aprendizado? No exemplo da tabela 2 do tópico 1.1, como dizer qual será a taxa de acertos (e erros consequentemente) da predição de **Pedalar** com base nas condições climáticas de um novo dia? Ainda, no exemplo da figura 1 do tópico 1.3, como dizer qual será o percentual de acertos na predição do tipo de uma nova bola?

Essas perguntas remetem a pensar se é possível determinar se um algoritmo de AM aprendeu, não aprendeu ou, se aprendeu, apenas decorou. Para isso existe uma etapa importante, que é a divisão dos dados em dois subconjuntos chamados de conjunto de treinamento e conjunto de teste.

O conjunto de treinamento são os dados que você fornece ao algoritmo de AM supervisionado para que ele possa aprender. E o conjunto de teste são os dados previamente separados dos dados de treinamento que, após o processo de aprendizado, você utiliza para avaliar se o algoritmo realmente aprendeu ou se ele simplesmente decorou. Importante ressaltar que os conjuntos de treinamento e de teste devem ser disjuntos, ou seja, todos os dados contidos no conjunto de teste não podem ser os mesmos contidos no conjunto de treinamento.

A divisão em treinamento e teste depende muito da quantidade de dados do conjunto inicial. Na prática, acaba-se dividindo em partes iguais se você tiver um conjunto com poucos dados. Para conjuntos maiores, um bom ponto de partida é dividir em 80% para treinamento e 20% para teste.

Saiba mais

Dividir os dados em treinamento e teste tem um paralelo em como a aprendizagem das pessoas pode ser avaliada. O professor ensina com exercícios e, para verificar se o aluno aprendeu, ele é submetido a uma prova (teste).

2.2 CONCEITO DO ALGORITMO KNN

O algoritmo dos K-Vizinhos (mais) Próximos (KNN) funciona de forma que, dado um exemplo desconhecido, ele será classificado de acordo com a classe mais frequente dentre os **K** exemplos mais próximos.

Aprofunde-se

Este é um livro de referência com notas, tabelas e exemplos detalhados para ajudar você a navegar pelo básico do machine learning com dados estruturados. Ele também apresenta uma visão geral da classificação com dados estruturados para a classificação e o agrupamento, entre outros assuntos.

Título do livro: Machine learning – Guia de referência rápida: trabalhando com dados estruturados em Python

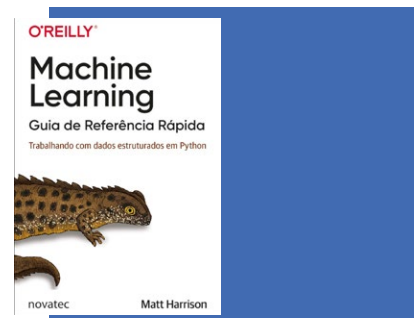
Autor: Matt Harrison.

Local: São Paulo.

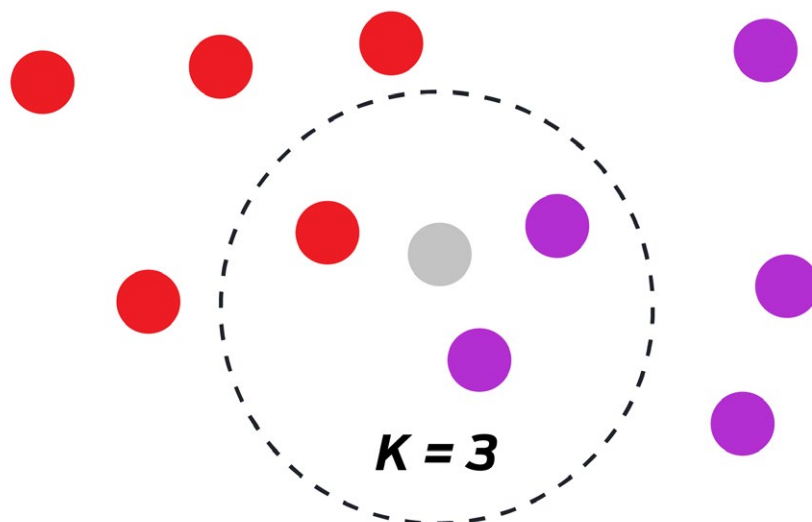
Editores: Novatec.

Data de publicação: 2019.

ISBN: 857522817X.



Observe na figura 2 abaixo uma simulação com duas classes (bolinhas roxas e vermelhas), sendo que a bolinha cinza é um exemplo que prediz a classe.

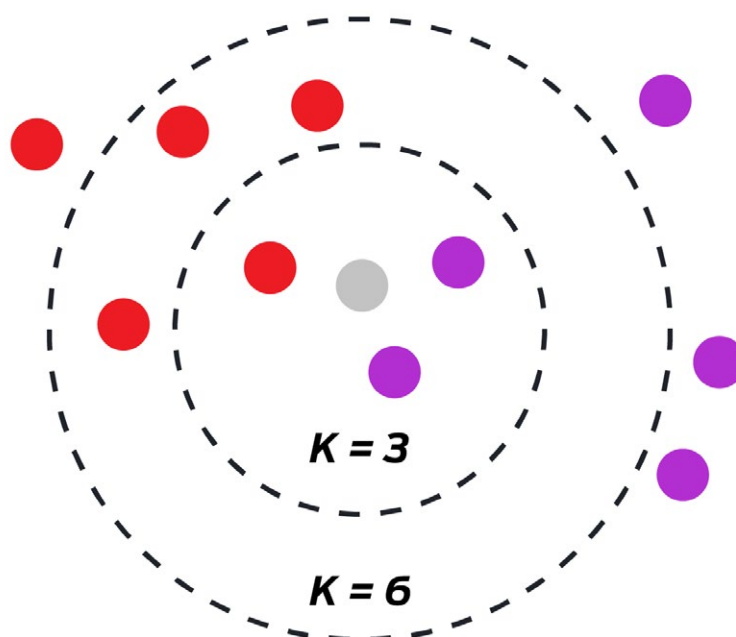


ED+ Content Hub © 2020

Figura 2 - K-Vizinhos (mais) Próximos com $K=3$.
Fonte: elaborado pelo autor.

Tomando a bolinha cinza como a classe desconhecida e os seus 3 vizinhos mais próximos, observa-se que duas bolinhas possuem a classe roxa e uma, a classe vermelha. Logo, a bolinha cinza seria classificada como sendo roxa.

A figura 3 abaixo é outra simulação, mas agora com $K=6$. Nesse caso, dentre os seis vizinhos mais próximos, há duas bolinhas roxas e quatro bolinhas vermelhas, sendo, portanto, a classe da bolinha cinza classificada como vermelha.



ED+ Content Hub © 2020

Figura 3 - K-Vizinhos (mais) Próximos com $K=3$ e $K=6$.
Fonte: elaborado pelo autor.

Reflita

O que você consegue notar em relação à classificação desconhecida da bolinha cinza utilizando o KNN a depender dos valores do hiperparâmetro **K**?

Entendido o funcionamento do algoritmo, a questão que agora surge é sobre o cálculo da distância. A distância entre um exemplo e outro pode ser calculada utilizando o cálculo de distância entre os pontos, tais como:

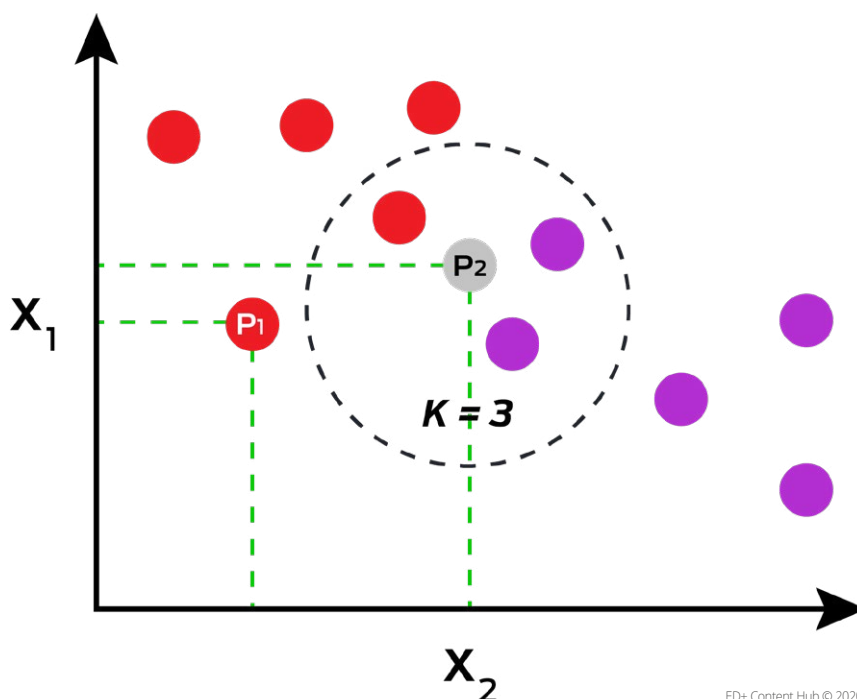
- Distância euclidiana;
- Distância de Hamming;
- Distância Manhattan;
- Distância de Markowski.

A métrica mais comum é a distância euclidiana, na qual, dados dois pontos, a distância entre eles pode ser calculada considerando os valores das variáveis que os definem conforme a equação:

$$D_{p_i, p_j} = \sqrt{\sum_{k=1}^n (x_{ik} - x_{jk})^2}$$

Na figura 4, há o par de pontos (**P₁**, **P₂**). Como são somente duas variáveis (dimensões) para cada ponto, a equação poderia ser simplificada para:

$$D_{p_1, p_2} = \sqrt{(x_{11} - x_{21})^2 + (x_{12} - x_{22})^2}$$



ED+ Content Hub © 2020

Figura 4 - Distância euclidiana.
 Fonte: elaborado pelo autor.

2.3 IMPLEMENTAÇÃO EM PYTHON DO KNN

Entendido o funcionamento do algoritmo de forma conceitual, a seguir você será apresentado a uma possível implementação do KNN em Python, utilizando os conhecimentos de Python e da biblioteca NumPy. A implementação está toda feita dentro do ambiente Jupyter Notebook.

Assista

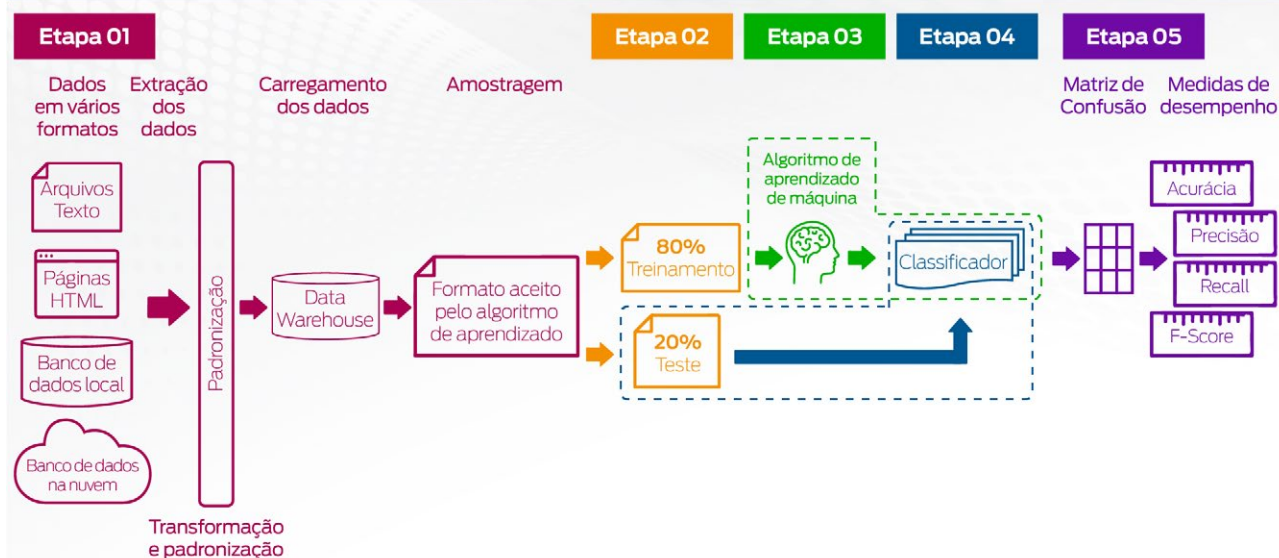
Acesse na plataforma o vídeo: Aprendizado supervisionado

Para melhor compreensão desse vídeo, acesse também o arquivo abaixo.

[Fundamentos_IA_U4_T3_Tópico_2.3.html](#)

Processo de avaliação do classificador em aprendizado de máquina para a validação do aprendizado

O aprendizado de máquina e os seus conceitos para a ciência de dados é parte de uma caixa de ferramentas. **Então, como montar um processo a partir de um conjunto de dados inicial até ter o conhecimento extraído e validado? Veja a seguir:**

**1. ETL (Extração, Transformação e Carregamento)**

Nesta etapa, os dados são extraídos de uma ou mais fontes de informação e transformados em um formato de estrutura de arquivo que o algoritmo de aprendizado de máquina consiga ler. Em geral, esses dados são também carregados e gravados em outro tipo de banco de dados ou num arquivo separado, porque serão apenas uma amostra do conjunto de todos os dados iniciais.

2. Divisão do conjunto de dados em treinamento e teste

Os dados são divididos de forma a ter dois novos conjuntos de dados, um com 80% para treinamento e outro com 20% para teste. **Em ambos os casos, os dados são selecionados de forma aleatória e disjunta.**

3. Algoritmo de aprendizado de máquina

Aplicar o algoritmo de aprendizado no conjunto de dados de treinamento, observando a configuração dos hiperparâmetros e garantindo que não haja confusão entre os conjuntos de dados originais, de treinamento e de teste. **O resultado será o classificador.**

4. Avaliar o classificador

Utilizar o conjunto de dados de teste para avaliar o classificador obtido e produzir a Matriz de Confusão.

5. Matriz de Confusão

A partir da Matriz de Confusão, é possível calcular as medidas de desempenho do classificador, tais como a acurácia, a precisão, o recall e o F-Score.

ED+ Content Hub © 2020

Infográfico 1 - Avaliação do classificador em aprendizado de máquina.
Fonte: elaborado pelo autor.

3 MÉTRICAS E MATRIZ DE CONFUSÃO

Até aqui, você estudou sobre os tipos de algoritmos de aprendizado, iniciando o estudo pelo KNN, que é um algoritmo de aprendizado de máquina supervisionado. No tópico anterior, também foi abordado o assunto dos conjuntos de treinamento e de teste e a sua importância para a avaliação de um modelo de predição. Agora é o momento de convergir para as formas de avaliar um modelo de predição.

Aprofunde-se

Este é um livro nacional, escrito pelos melhores pesquisadores da área no Brasil, com os principais conceitos e algoritmos de aprendizado de máquina, os quais apontam caminhos para transformar esse conhecimento em aplicações práticas.

Título do livro: Inteligência Artificial - Uma abordagem de aprendizado de máquina

Autores: Katti Faceli, Ana Carolina Lorena, João Gama e André C.P.L.F. de Carvalho.

Local: São Paulo.

Editora: LTC.

Data de publicação: 2015.

ISBN: 8521618808.



Existem muitas medidas de avaliação dos modelos preditivos, e a boa notícia é que essas medidas podem ser derivadas dos resultados da Matriz de Confusão. Cada célula da matriz corresponde à frequência de classificação para cada classe do modelo, considerando as classes reais e as classes preditas. O cálculo da Matriz de Confusão é feito a partir do modelo induzido, utilizando os dados de treinamento e é aplicado aos dados de teste para verificar a verdadeira efetividade do aprendizado.

Na tabela 4, há um exemplo de uma Matriz de Confusão hipoteticamente calculada do modelo de classificação inferido pelo KNN para o problema de predição das bolinhas vermelhas e roxas.

		Classe esperada	
		Vermelha	Roxa
Classe predita	Vermelha	Verdadeiro Vermelha	Falso Vermelha
	Roxa	Falso Roxa	Verdadeiro Roxa

Tabela 4 - Matriz de Confusão – Bolinhas roxas/vermelhas.
Fonte: elaborado pelo autor.

Na primeira célula (verdadeiro/vermelha), espera-se encontrar o percentual de bolinhas em relação ao total do exemplo de treinamento que tinham como classe real vermelha e foram preditas pelo classificador como sendo vermelhas.

Já na segunda célula (falso/vermelha), espera-se encontrar o percentual de bolinhas em relação ao total do exemplo de treinamento que tinham como classe real roxa e foram preditas pelo classificador como sendo vermelhas.

Na terceira célula (falso/roxa), espera-se encontrar o percentual de bolinhas em relação ao total do exemplo de treinamento que tinham como classe real vermelha e foram preditas pelo classificador como sendo roxas.

E, finalmente, na quarta célula (verdadeiro/roxa), espera-se encontrar o percentual de bolinhas em relação ao total do exemplo de treinamento que tinham como classe real roxa e foram preditas pelo classificador como sendo roxas.

De forma mais geral, a tabela 5 abaixo apresenta a Matriz de Confusão como um problema de classificação binária de forma que se tem a classe predita e a negação disso, ou seja, a predição de outras classes. Pode-se também representar a Matriz de Confusão como sendo uma matriz quadrada contendo todas as classes do problema de predição.

		Classe esperada	
		Positivo	Negativo
Classe predita	Positivo	VP Verdadeiro positivo	FP Falso positivo
	Negativo	FN Falso negativo	VN Verdadeiro negativo

Tabela 5 - Matriz de Confusão – geral.
Fonte: elaborado pelo autor.

A partir da Matriz de Confusão, muitas medidas de desempenho do classificador podem ser obtidas, conforme apresentadas a seguir:

- **Acurácia:** diz o quanto o modelo acertou em relação às previsões possíveis. O cálculo é feito pela soma das previsões corretas dividida pela soma de todas as previsões.

$$Acurácia = \frac{VP + VN}{VP + VN + FP + FN}$$

- **Recall:** diz respeito a quanto dos exemplos positivos foram identificados corretamente. O cálculo é feito pela previsão dos verdadeiros positivos dividida pela soma dos verdadeiros positivos mais os falsos negativos.

$$Recall = \frac{VP}{VP + FN}$$

- **Precisão:** diz respeito ao quão bom o modelo é em prever a classe positiva, se o modelo acertou em relação às previsões possíveis. O cálculo é feito pela previsão dos verdadeiros positivos dividida pela soma dos exemplos identificados como positivos, sejam eles verdadeiros ou falsos.

$$Precisão = \frac{VP}{VP + FP}$$

- **F-Score:** essa medida procura promover um balanceamento entre as medidas de precisão e recall.

$$F-Score = 2 * \frac{precisão * recall}{precisão + recall}$$

Como citado anteriormente, mas agora de forma mais explícita, as medidas acima foram feitas visando entender o impacto da predição sobre os dados positivos. Quando o problema envolve exemplos com três ou mais classes, o raciocínio é feito de forma análoga, considerando todas as classes existentes no problema.

Reflita

Um conjunto de dados pode ser incrivelmente desbalanceado em termos de número de exemplos em cada classe. Um exemplo típico são os dados de fraude em seguros. O conjunto de dados pode ter milhões de linhas de segurados e chamadas de sinistros, mas apenas um pequeno percentual (menor que 1%) será de fraudes. Como isso impacta na predição? Quais medidas se utilizar?

As medidas apresentadas são úteis para avaliar um classificador e é recomendável analisá-las em conjunto, pois um classificador pode ter uma acurácia muito grande, mas uma baixa precisão.

Assista

Acesse na plataforma o vídeo: Matriz de Confusão e medidas

CONSIDERAÇÕES FINAIS

Nesta unidade, você conheceu conceitos importantes sobre os paradigmas de aprendizado em aprendizado de máquina, como o aprendizado supervisionado, o aprendizado não supervisionado e o aprendizado por reforço.

Também viu a necessidade e a importância de dividir os exemplos em treinamento e teste antes do algoritmo propriamente aprender o conceito relativo ao rótulo dos exemplos. Além de compreender como funciona na prática o aprendizado supervisionado, utilizando o algoritmo dos Vizinhos (mais) Próximos (KNN) implementado em Python.

Finalmente, estudou o poder das métricas e da Matriz de Confusão de um classificador. As métricas têm um papel fundamental na avaliação dos resultados obtidos por sua análise de ML antes de apresentar qualquer resultado ou insight para um gestor de negócios.

GLOSSÁRIO

AM: Aprendizado de Máquina é uma técnica de Inteligência Artificial utilizada para que o computador possa aprender com informações passadas presentes no conjunto de dados.

KNN: (K-Nearest Neighbors): K-Vizinhos (mais) Próximos é um algoritmo de aprendizado de máquina que procura classificar exemplo de dados não vistos com base nos demais exemplos já conhecidos. Leva esse nome porque a classificação é feita com base na proximidade, a qual remete à similaridade.

ML: Machine Learning é o termo original em inglês cuja tradução livre deu origem ao termo Aprendizado de Máquina.

REFERÊNCIAS

FACELI, Katti; LORENA, Ana Carolina; GAMA, João; CARVALHO, André C. P. L. F. **Inteligência Artificial. Uma Abordagem de Aprendizado de Máquina**. Rio de Janeiro: LTC, 2015.

HARRISON, Matt. **Machine Learning** – Guia de Referência Rápida: Trabalhando com Dados Estruturados em Python. São Paulo: Novatec, 2019.

MITCHELL, Tom. **Machine Learning**. Porto Alegre: Bookman, 1997.

MOREIRA, Marco Antônio. **Teorias de Aprendizagem**. São Paulo: E.P.U, 2011.

Unidade 5

Pré-processamento

Objetivo de aprendizagem:

Identificar problemas normalmente presentes em conjuntos de dados e técnicas para minimizar o efeito nos modelos de aprendizagem automática.

Tópicos de estudo:

- Tratamento de dados faltantes;
- Normalização e padronização;
- Transformação de variáveis categóricas;
- Redução de dimensionalidade.

Iniciando os estudos:

A etapa de pré-processamento em muitos casos é negligenciada pela maioria das pessoas que trabalham com o machine learning. Segundo Thomas Edison, o inventor da lâmpada, “o sucesso é constituído por 10% de inspiração e 90% de transpiração”. Pois bem, para se construir um sistema computacional especialista com uma engenharia de machine learning eu diria que 10% é responsabilidade do algoritmo de aprendizado de máquina em construir um modelo (extrair inspiração). Já os 90% de transpiração ficam por conta da fase de ETL e pré-processamento.

Afinal, qualquer sistema computacional aprende aquilo que você ensina, ou pior, aplica-se ao conceito de “Garbage In, Garbage Out”. É no pré-processamento que você vai formatar, limpar, padronizar, transformar e facilitar o aprendizado de máquina. Portanto não menospreze essa etapa. Ela parece não ter muito glamour, mas como argumentado, ela é fundamental.

Você verá nesta unidade, então, como trabalhar com um conjunto de dados em que possa existir uma ou mais características que afetem positivamente ou negativamente o algoritmo de aprendizado, como: o tratamento de dados faltantes; a normalização e a padronização dos dados; a transformação de variáveis categóricas; e a redução de dimensionalidade. Bons estudos!

Entendendo as etapas do pré-processamento

Os algoritmos de aprendizado de máquina são muito sensíveis às variações e ausências de informações, além das ferramentas de pré-processamento estarem à disposição para ajudar a remediar essas questões.

O **tratamento de dados ausentes** é muito importante para que toda a informação prevista nos dados tenha consistência e faça sentido no contexto geral.



A **normalização e a padronização** proporcionam um ajuste na ordem de grandeza para que todas as informações possam ter chances iguais de colaborar com o modelo de aprendizado de máquina.



A **transformação de variáveis categóricas** traz uma possibilidade adicional de essas informações cooperarem positivamente com as variáveis numéricas para induzirem o modelo de aprendizado de máquina.



A **redução de dimensionalidade** é essencial para que seja entregue ao algoritmo de aprendizado informações mais sucintas, de forma que ele tenha mais tempo para olhar o que interessa.



ED - Content Hub © 2020

Infográfico 1 - Etapas do pré-processamento.
Fonte: elaborado pelo autor.

1 TRATAMENTO DE DADOS FALTANTES

Este tópico é dividido em duas seções para melhor entendimento. No primeiro, você vai entender a razão pela qual os dados ausentes se originam e alguns exemplos. Depois, na outra seção, você vai ver algumas formas mais usuais para remediar o problema dos dados ausentes.

1.1 DADOS FALTANTES: O QUE SÃO E COMO SE ORIGINAM

Um conjunto de dados proveniente de um processo de **ETL**, embora já seja padronizado do ponto de vista de formato, após a integração, ainda contará com os vícios e as mazelas que os sistemas que capturam essas informações proporcionam.

Os dados faltantes (ausentes) são decorrentes de sistemas falhos, campos não obrigatórios, campos irregulares, campos abertos, entre outros. Veja a seguir alguns exemplos que podem proporcionar o aparecimento de dados faltantes em conjuntos de dados:

- **Campos não obrigatórios:** idade, endereço, renda etc.
- **Campos obrigatórios** (mas sem a correta consistência que obrigue a correta captura do preenchimento): nome completo, CPF, RG etc.
- **Campos com opção correta faltante:** lista dos estados onde não consta um estado em particular. Nesse caso, a pessoa, ao preencher e notar que o seu estado não está na lista, provavelmente irá deixá-la em branco.
- **Campos que dependem de uma ação temporal:** dados de navegação em um e-commerce são capturados na medida em que o cliente navega e executa as ações. Nesse caso, o nome do produto adquirido e o valor somente existirão no conjunto de dados se o cliente fizer a compra, senão eles estarão em branco e haverá no conjunto de dados somente as demais informações de navegação. A documentação e as informações sobre os dados extraídos do Google Analytics são um exemplo.

Note que algumas dessas inconsistências podem, inclusive, gerar dados discrepantes (aberrantes ou outliers) com relação à distribuição geral dos dados daquela coluna do seu conjunto de dados, como as categorias inexistentes (por exemplo, Estado = "KX"), os valores absurdos (salário = R\$0,20) e os valores incompletos (CPF com 6 dígitos, dos 11 corretos). Mas será tratada neste tópico somente a questão dos dados ausentes.

Apenas para fim de exemplificar, veja a tabela 1 a seguir com uma amostra dos dados contidos em **dados.xlsx** (acesse o arquivo na plataforma), que está sendo utilizada nos **Jupyter Notebooks**. Já estão os valores ausentes utilizando a notação utilizada pelo Python, que é sinalizar o valor ausente como **NaN**.

Cliente	Região	Mês	Valor Compra
Cliente 4	Nordeste	Junho	R\$ 1.164,00
Cliente 1	Centro-Oeste	Junho	R\$ 1.517,00
Cliente 5	Nordeste	Setembro	R\$ 2.993,00
Cliente 10	Sudeste	Março	NaN
Cliente 13	Sul	Novembro	R\$ 2.087,00
Cliente 7	NaN	Janeiro	R\$ 2.585,00
Cliente 14	Sul	Março	R\$ 3.510,00
Cliente 11	Sudeste	Outubro	R\$ 1.325,00
Cliente 15	Sul	Setembro	R\$ 2.077,00
Cliente 8	Norte	NaN	R\$ 2.759,00
Cliente 9	Norte	Dezembro	NaN
Cliente 2	Centro-Oeste	Novembro	R\$ 1.482,00
Cliente 7	Norte	Março	R\$ 2.506,00

Tabela 1 - Exemplos de conjunto de dados com valores faltantes.
Fonte: elaborado pelo autor.

1.2 DADOS FALTANTES: COMO TRATÁ-LOS EM PYTHON

Como você viu, ter conhecimento da possibilidade de que em um conjunto de dados existam valores ausentes é muito importante. Mas como corrigir isso? A primeira coisa que vem à mente é a solução mais simples: basta remover as linhas (registros), nas quais existam campos com dados faltantes. Sim, isso é uma possibilidade. A questão é que, ao fazer isso, você também está removendo conhecimento implícito existente nos dados e perdendo informação de outros campos que têm informações. Remover todas as linhas onde exista ao menos um campo com dados ausentes irá deixá-lo sem nada!

Você deve estar pensando: “Está bom, já estou convencido! E então?!”. Existem formas inteligentes para lidar com isso, para que ocorra a imputação automática e a substituição dos dados ausentes com base em medidas estatísticas básicas, como a média, a mediana ou a moda.

Agora, vamos ao Python! O primeiro ponto é saber identificar a presença de dados ausentes, por exemplo, o trecho de código Python a seguir para carregar um conjunto de dados já preparados de antemão que devem conter os dados ausentes, conforme a figura 1.

```

[1] ▶ M4
import pandas as pd
import numpy as np

[2] ▶ M4
# Vamos definir o arquivo Excel a ser aberto
xlsx = pd.ExcelFile('dados_aus.xlsx')

[3] ▶ M4
# cria um DataFrame a partir do arquivo Excel, lendo apenas os dados contidos na 'Planilha1'
df = pd.read_excel(xlsx, 'Planilha1')
df

```

Figura 1 - Leitura de **dados_aus.xlsx** e carregamento em um DataFrame.

Fonte: elaborado pelo autor.

Visualizando o resultado do DataFrame **df** na tela, você é capaz de notar a existência de colunas para as quais o valor está como NaN, ou seja, valor ausente, como na figura 2.

	Cliente	Região	Mês	Valor Compra
0	Cliente 4	Nordeste	Junho	1164.000000
1	Cliente 1	Centro-Oeste	Junho	1517.000000
2	Cliente 5	NaN	Setembro	2993.000000
3	Cliente 10	Sudeste	Março	1490.000000
4	Cliente 13	Sul	NaN	2087.000000
...	...	...	...	...
595	Cliente 13	Sul	Setembro	2261.151079
596	Cliente 8	Norte	Abril	3311.151079
597	Cliente 14	Sul	Julho	1854.496403
598	Cliente 9	Norte	Junho	1676.978417
599	Cliente 2	Centro-Oeste	Fevereiro	2199.460432

600 rows x 4 columns

Figura 2 - Presença de dados ausentes.

Fonte: elaborado pelo autor.

Outra forma de detectar a existência de valores ausentes é comparar o shape do DataFrame com os resultados do método **Describe()** para uma determinada coluna. Como no exemplo da figura 3.

```
▶ MI
df.shape

(600, 4)

▶ MI
df['Região'].describe()

count          586
unique           5
top      Sudeste
freq          151
Name: Região, dtype: object
```

Figura 3 - Presença de dados ausentes na coluna 'Região'.
Fonte: elaborado pelo autor.

Pela diferença entre o total de linhas do DataFrame e o total (count) de valores para o campo '**Região**', você consegue deduzir que existem 14 registros para os quais os valores de '**Região**' estão ausentes.

Você também pode utilizar os métodos **info()** e **isnull()** (o qual testa logicamente cada linha e campo do DataFrame para verificar se a informação é ausente). Este último, por sua vez, pode ser utilizado de forma conjugada com o método **sum()** para retornar a soma dos ausentes em cada coluna. Veja na figura 4.

```

▶ MI
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 600 entries, 0 to 599
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Cliente         600 non-null   object
1   Região          586 non-null   object
2   Mês             584 non-null   object
3   Valor Compra    572 non-null   float64
dtypes: float64(1), object(3)
memory usage: 18.9+ KB

▶ MI
df.isnull().sum()

Cliente         0
Região         14
Mês            16
Valor Compra    28
dtype: int64

```

Figura 4 - Presença de dados ausentes utilizando outras formas.

Fonte: elaborado pelo autor.

Antes de continuar, quero que você saiba que não existe forma correta para lidar com os dados ausentes. Assim como outras escolhas que você terá que fazer no seu toolbox para modelar o seu sistema de Inteligência Artificial, as escolhas devem ser feitas com a maior consciência possível e conhecimento de possíveis consequências. Sempre que escolher um caminho, você colocará algum viés (bias em inglês) no seu modelo. Mas não se assuste: pense, escolha e faça!

1.2.1 DADOS FALTANTES: EXCLUSÃO

Já que existe essa opção, vou lhe ensinar! A remoção de linhas e colunas em um DataFrame também pode ser necessária quando você cria linhas e/ou colunas artificialmente com base em outras informações e precisa remover as anteriores. Então aprender como isso é feito tem as suas utilidades.

Para remover os dados ausentes, utiliza-se o método **df.dropna()**, o qual remove os valores NaN encontrados no DataFrame. Por padrão, são eliminadas as linhas que contêm ao menos um campo. Se você não informar o eixo, serão eliminadas todas as linhas relativas à célula contendo o valor ausente. Isso se equivale ao método **df.dropna(axis=0)**.

Se a ideia for eliminar a coluna inteira que contém dados ausentes, basta alterar o **axis** de forma que **df.dropna(axis=1)** fará isso. Nos dois casos, a opção na verdade é um filtro e não ocorre diretamente no DataFrame. Para eliminar diretamente, terá que colocar a opção **inplace=True**, por exemplo, **df.dropna(axis=1,inplace=True)**.

Veja na figura 5 abaixo que, ao executar o método **dropna()**, foi retornado um DataFrame com 543 linhas, mas que ao executar, logo em seguida, a chamada de **shape**, ainda é exibida a tupla (600,4), o que significa que nenhuma linha foi na realidade removida. Isso é para evitar a perda de informação e o Python trabalhar com os DataFrames com a ideia de filtros.

```

> MI
df.dropna()

  Cliente  Região  Mês  Valor Compra
0  Cliente 4  Nordeste  Junho  1164.000000
1  Cliente 1  Centro-Oeste  Junho  1517.000000
3  Cliente 10  Sudeste  Março  1490.000000
5  Cliente 7  Norte  Janeiro  2585.000000
6  Cliente 14  Sul  Março  3510.000000
...  ...  ...  ...  ...
595  Cliente 13  Sul  Setembro  2261.151079
596  Cliente 8  Norte  Abril  3311.151079
597  Cliente 14  Sul  Julho  1854.496403
598  Cliente 9  Norte  Junho  1676.978417
599  Cliente 2  Centro-Oeste  Fevereiro  2199.460432

543 rows x 4 columns

> MI
df.shape

(600, 4)

```

Figura 5 - Remoção de dados ausentes por **dropna()**.
Fonte: elaborado pelo autor.

Ou seja, ao chamar **df.dropna()**, o ideal é que você atribua isso a um novo DataFrame que conterá somente os dados sem os ausentes, tipo **ndf = df.dropna()**. Dessa forma, no DataFrame chamado de **df**, você terá os dados originais, enquanto que, no DataFrame chamado de **ndf**, você terá os dados sem os ausentes.

Se você quiser remover diretamente, como dito, use o argumento **inplace=True**, cujo resultado pode ser visto na figura 6 abaixo. Observe o valor do shape logo após a execução do método com esse parâmetro.

```
▶ MI  
df.dropna(inplace=True)  
  
▶ MI  
df.shape  
  
(543, 4)
```

Figura 6 - Remoção de dados ausentes por **dropna(inplace=True)**.

Fonte: elaborado pelo autor.

1.2.2 DADOS FALTANTES: PREENCHIMENTO COM MÉDIA, MEDIANA OU MODA

Para preencher uma coluna do DataFrame com a média, a mediana ou a moda, primeiramente você precisa entender a que tipo de coluna está se referindo. Se a coluna for categórica, por exemplo, **Estado**, ela deve aceitar apenas alguns valores nominais pré-determinados. Se for uma coluna tipo string, deve aceitar somente dados que não sejam numéricos. Nesses casos, faz sentido somente usar a **moda**. Para colunas numéricas é indicado utilizar a **média** ou a **mediana**.

Você pode fazer o preenchimento dos dados ausentes utilizando formas mais rebuscadas com base na distribuição dos valores em combinação com outras colunas. Mas, para fins de simplificação do aprendizado, vamos focar na média, mediana ou moda de uma coluna de forma isolada.

Para todas essas três possibilidades, pode ser utilizado o método **fillna(value, inplace=True)**, com **value** sendo o valor a ser utilizado na substituição dos ausentes e **inplace** para que a substituição possa ser feita no próprio DataFrame. Note que **value** pode assumir valores numéricos ou string a depender do tipo da coluna que você está chamando.

Reflita

O Python é uma linguagem de Programação Orientada a Objetos. No caso da chamada do método **fillna(value, inplace=True)**, tente relembrar qual o conceito de Programação Orientada a Objetos que permite que **value** “possa assumir” valores string ou numéricos.

Na figura 7 abaixo, está sendo calculada a frequência de cada valor presente na coluna **'Região'**. Isso irá retornar um array cujo índice é a região e o valor é a frequência. Como se quer capturar o nome da região mais frequente, é preciso armazenar na variável **moda** apenas o índice zero.

```

> MI
df['Região'].value_counts()

Sudeste      151
Centro-Oeste 136
Norte        116
Sul          93
Nordeste     90
Name: Região, dtype: int64

> MI
moda = df['Região'].value_counts().index[0]
moda

'Sudeste'

```

Figura 7 - Calculando a moda da coluna **'Região'**.
Fonte: elaborado pelo autor.

Na figura 8 está sendo calculada a mediana referente à coluna **'Valor Compra'**.

```

> MI
mediana = df['Valor Compra'].median()
mediana

2949.1690647482014

```

Figura 8 - Calculando a mediana da coluna **'Valor Compra'**.
Fonte: elaborado pelo autor.

Na figura 9 está sendo calculada a média referente à coluna **'Valor Compra'**.

```

> M4
media = df['Valor Compra'].mean()
media
3131.5985812748404

```

Figura 9 - Calculando a média da coluna **'Valor Compra'**.
Fonte: elaborado pelo autor.

Na figura 10 abaixo, de posse dos valores da mediana, média e moda, substitui-se os valores ausentes das colunas correspondentes de acordo com o tipo da coluna, como eu disse anteriormente.

Note que existe um passo adicional para fazer a substituição dos valores ausentes também na coluna **'Mês'**, de forma que, ao chamar o método **info()**, todas as colunas contêm o mesmo número de valores **non-null** que o total de linhas do DataFrame, ou seja, não existem mais valores ausentes no DataFrame.

```

> M4
# Preenche os valores ausentes da coluna 'Região' com a moda
df['Região'].fillna(modas, inplace=True)
# Extrai a moda da coluna 'Mês'
modas = df['Mês'].value_counts().index[0]
# Preenche os valores ausentes da coluna 'Mês' com a moda
df['Mês'].fillna(modas, inplace=True)
# Preenche os valores ausentes da coluna 'Valor Compra' com a média
df['Valor Compra'].fillna(media, inplace=True)
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 600 entries, 0 to 599
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Cliente     600 non-null    object
1   Região      600 non-null    object
2   Mês         600 non-null    object
3   Valor Compra 600 non-null    float64
dtypes: float64(1), object(3)
memory usage: 18.9+ KB

```

Figura 10 - Preenchimento e validação a respeito dos valores ausentes.
Fonte: elaborado pelo autor.

Assista

Acesse na plataforma o vídeo: Tratamento de dados ausentes

Importante: para compreender o tutorial deste vídeo, você deverá acessar os arquivos `jupyter_notebook_un5_top1.ipynb` e `TA_PRECO_MEDICAMENTO.xlsx` disponíveis na plataforma.

2 NORMALIZAÇÃO E PADRONIZAÇÃO

Nos conjuntos de dados, também são observados, além da distribuição dos dados, a ordem de grandeza dela, principalmente quando comparada uma com a outra.

A maior parte dos algoritmos de aprendizado de máquina supervisionado e não supervisionado faz uso de modelos matemáticos. Para esses casos, ter variáveis com valores cuja ordem de grandeza destoa das demais é como fazer uma média ponderada com pesos muito diversos. Por exemplo, em:

$$z = 10^9 x + y, x, y \in \{0, 100\}$$

Pode-se notar que, independentemente dos valores que **x** e **y** possam assumir no intervalo, para o valor resultante de **z** a componente de **x** exerce um peso que praticamente elimina a participação de **y** no cálculo final. Analogamente, algo da mesma forma ocorre nos algoritmos de aprendizado de máquina.

Acredito que já tenha ficado claro, mas quando se fala em normalizar ou padronizar as variáveis, isso está relacionado a variáveis numéricas. Tanto uma como outra técnica irão deixar todas as variáveis na mesma ordem de grandeza.

Padronizar os dados significa que os novos valores da variável terão média 0 (zero) e desvio padrão 1 (um), enquanto normalizar significa que os novos valores da variável ficarão na faixa {0,1}. Se a variável contiver valores negativos, ficarão na faixa {-1,1}.

$$x_{padronizado} = \frac{x - \mu}{\sigma} \quad x_{normalizado} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Utilizando o Python, essas duas transformações são bem simples. Veja primeiro a padronização. Na figura 11 abaixo, leia os dados do **arquivo Excel** mais uma vez. Desta vez, adicionei uma coluna chamada

'Valor Imposto', na qual todos os valores são 12% do 'Valor Compra', sempre dando aquela olhada para ver se carregou conforme esperado usando o método `head()`.

```

> Mi
import pandas as pd
import numpy as np

> Mi
# Vamos definir o arquivo Excel a ser aberto
xlsx = pd.ExcelFile('dados.xlsx')

> Mi
# cria um DataFrame a partir do arquivo Excel, lendo apenas os dados contidos na 'Planilha1'
df = pd.read_excel(xlsx, 'Planilha1')
df.head(5)

```

	Cliente	Região	Mês	Valor Compra	Valor Imposto
0	Cliente 4	Nordeste	Junho	1164.0	139.68
1	Cliente 1	Centro-Oeste	Junho	1517.0	182.04
2	Cliente 5	Nordeste	Setembro	2993.0	359.16
3	Cliente 10	Sudeste	Março	1490.0	178.80
4	Cliente 13	Sul	Novembro	2087.0	250.44

Figura 11 - Carregando os dados antes da padronização.
Fonte: elaborado pelo autor.

A figura 12 mais abaixo mostra o processo de importação da classe **StandardScaler** da biblioteca Scikit-learn. Feito isso, foi definido um objeto que chamei de **scaler** para instanciar essa classe. Com o objeto estamos dizendo que os dados a serem padronizados estão no DataFrame de nome **df** e nas colunas 3 e 4 somente porque são as colunas que têm valores numéricos e podem ser padronizadas, a saber, 'Valor Compra' e 'Valor Imposto'. Os valores padronizados ficam armazenados em **df_rescaled**.

```

M4
from sklearn.preprocessing import StandardScaler

M4
cols = df.columns
cols

Index(['Cliente', 'Região', 'Mês', 'Valor Compra', 'Valor Imposto'], dtype='object')

M4
scaler = StandardScaler()
df_rescaled = scaler.fit_transform(df[cols[3:5]])

M4
df_rescaled

array([[ -1.51292967,  -1.51292967],
       [ -1.23623839,  -1.23623839],
       [ -0.07930829,  -0.07930829],
       ...,
       [ -0.97169927,  -0.97169927],
       [ -1.11084283,  -1.11084283],
       [ -0.70130681,  -0.70130681]])

```

Figura 12 - Importando biblioteca de padronização do Scikit-learn.

Fonte: elaborado pelo autor.

Aprofunde-se

Este livro é para quem quer aprender colocando a mão na massa. É um guia prático com muitos exemplos e cases para se aprofundar no uso das bibliotecas Scikit-learn e TensorFlow.

Título do livro: Mãos à Obra: Aprendizado de Máquina com Scikit-Learn & TensorFlow

Autor: Aurélien Géron.

Local: São Paulo.

Editora: Alta Books.

Data de publicação: 2019.

ISBN: 8550803812.



Na figura 13 abaixo, são atribuídos os valores padronizados para as devidas colunas do DataFrame. Como o **'Valor Imposto'** é uma razão linear direta do **'Valor Compra'**, os valores padronizados linha a linha ficaram idênticos. Você também pode verificar se a padronização deu certo utilizando o método **describe()**. Note como a média e o desvio padrão estão de acordo com o pressuposto anterior (média = 0 e desvio padrão = 1).

▶ M4

```
df[['Valor Compra', 'Valor Imposto']] = df_rescaled
```

▶ M4

```
df.head(5)
```

	Cliente	Região	Mês	Valor Compra	Valor Imposto
0	Cliente 4	Nordeste	Junho	-1.512930	-1.512930
1	Cliente 1	Centro-Oeste	Junho	-1.236238	-1.236238
2	Cliente 5	Nordeste	Setembro	-0.079308	-0.079308
3	Cliente 10	Sudeste	Março	-1.257402	-1.257402
4	Cliente 13	Sul	Novembro	-0.789456	-0.789456

▶ M4

```
df.describe()
```

	Valor Compra	Valor Imposto
count	6.000000e+02	600.000000
mean	-1.302662e-16	0.000000
std	1.000834e+00	1.000834
min	-1.522336e+00	-1.522336
25%	-7.720653e-01	-0.772065
50%	-1.470023e-01	-0.147002
75%	5.636266e-01	0.563627
max	2.615688e+00	2.615688

Figura 13 - Incorporação dos dados padronizados e verificação.

Fonte: elaborado pelo autor.

Outra forma de equiparar a ordem de grandeza entre as variáveis é a normalização. Será utilizada a normalização por Min-Max, conforme a fórmula mostrada no início do tópico 2 desta unidade.

A figura 14 abaixo mostra o processo de importação da classe **MinMaxScaler** da biblioteca Scikit-learn. Feito isso, foi definido um objeto que chamei de **norma** para instanciar essa classe. Depois, seguindo o mesmo raciocínio da padronização, são informadas as colunas a serem normalizadas e obtém-se o array normalizado.

```
▶ M4
from sklearn.preprocessing import MinMaxScaler

▶ M4
norma = MinMaxScaler()

▶ M4
df_norma = norma.fit_transform(df[cols[3:5]])

▶ M4
df_norma

array([[0.00227305, 0.00227305],
       [0.06913861, 0.06913861],
       [0.34872378, 0.34872378],
       ...,
       [0.13306746, 0.13306746],
       [0.09944186, 0.09944186],
       [0.19841084, 0.19841084]])
```

Figura 14 - Importando a biblioteca de normalização do Scikit-learn.

Fonte: elaborado pelo autor.

Na figura 15 abaixo, há a atribuição dos valores normalizados às colunas do DataFrame e, em seguida, a verificação utilizando o método **Describe()**, em que você pode notar que os valores mínimo e máximo são 0 e 1, respectivamente, de acordo com o pressuposto explicado anteriormente.

```

▶ MI
df[['Valor Compra', 'Valor Imposto']] = df_norma

▶ MI
df.head(5)

```

	Cliente	Região	Mês	Valor Compra	Valor Imposto
0	Cliente 4	Nordeste	Junho	0.002273	0.002273
1	Cliente 1	Centro-Oeste	Junho	0.069139	0.069139
2	Cliente 5	Nordeste	Setembro	0.348724	0.348724
3	Cliente 10	Sudeste	Março	0.064024	0.064024
4	Cliente 13	Sul	Novembro	0.177108	0.177108

```

▶ MI
df.describe()

```

	Valor Compra	Valor Imposto
count	600.000000	600.000000
mean	0.367890	0.367890
std	0.241863	0.241863
min	0.000000	0.000000
25%	0.181311	0.181311
50%	0.332365	0.332365
75%	0.504096	0.504096
max	1.000000	1.000000

Figura 15 - Incorporação dos dados normalizados e verificação.
Fonte: elaborado pelo autor.

Assista

Acesse na plataforma o vídeo: Padronização e normalização

Importante: para compreender o tutorial deste vídeo, você deverá acessar os arquivos `jupyter_notebook_un5_top2.ipynb` e `TA_PRECO_MEDICAMENTO.xlsx` disponíveis na plataforma.

3 TRANSFORMAÇÃO DE VARIÁVEIS CATEGÓRICAS

Como dito no tópico 2, a maior parte dos algoritmos de aprendizado de máquina supervisionado e não supervisionado faz uso de modelos matemáticos. Para fins de entendimento, é possível pensar em um modelo matemático simples para o aprendizado supervisionado, conforme a seguir:

$$y = a_1x_1 + a_2x_2 + \dots + a_nx_n$$

De forma que: y é a variável dependente que se quer obter, cada x_i é uma variável independente e cada a_i é um peso associado a essas variáveis independentes, tal que o algoritmo de aprendizado tem que determinar os corretos valores de cada a_i de modo a aprender essa associação entre y e os x_n .

A limitação imediata pode ser observada independente do valor de cada a_i . Se cada x_i não for numérico, como pode ser calculado cada componente que na verdade é uma multiplicação $a_i \times x_i$? Então, para os algoritmos de aprendizado de máquina, como por exemplo, a regressão linear, as regressões não lineares e alguns tipos de redes neurais, transformar os dados categóricos em dados numéricos é essencial. É possível utilizar três formas para isso, embora outras surjam a cada momento:

- **One Hot Encoding:** transforma cada valor da categoria em uma nova coluna e associa os valores 0 ou 1 a cada coluna a depender do valor de categoria original;
- **Contagem de frequências:** substitui o valor da categoria pelo valor da frequência dela dentro do conjunto de dados como um todo;
- **Mapeamento de target:** substitui o valor da categoria por um mapeamento direto dela, por exemplo, o ordinal associado à primeira letra do valor da categoria.

Nenhum método é perfeito, já que, quando se fala em números, existem nuances envolvidas. Por exemplo, se João tem 15 anos e Maria tem 30, decorre que Maria tem o dobro da idade de João. Também, que há um lapso temporal de 15 anos entre ambos. Então, se você tem em um conjunto de dados uma coluna com valores categóricos, digamos 'A', 'C' e 'T' distribuídos em alguma ordem e com alguma frequência, substituí-los por, respectivamente, 1, 2 e 3 seria a melhor escolha? Ou 1, 3 e 20, respeitando a ordem das letras do alfabeto? Ou substituir 'A'=100, 'C'=010 e 'T'=001?

Aprofunde-se

Nesta obra, são abordadas as ferramentas principais para o pré-processamento de dados utilizando o Pandas. Obtenha instruções completas para manipular, processar, limpar e extrair informações de conjuntos de dados em Python.

Título do livro: Python para Análise de Dados: Tratamento de Dados com Pandas, NumPy e IPython

Autor: Wes McKinney.

Local: São Paulo.

Editora: Novatec.

Data de publicação: 2018.

ISBN: 8575226479.



Não tem certo e errado nesses casos. Tudo vai depender do conjunto de dados e do tipo de algoritmo que melhor se enquadra para o problema e os resultados que se almeja alcançar. Você vai ter que fazer uma escolha, testar e aferir os resultados. Isso é uma regra, praticamente uma lei, puro empirismo!

Veja na figura 16 abaixo um exemplo de One Hot Encoding (**OHE**) para a coluna categórica '**Mês**'. Foi utilizado o método **concat()** do Pandas para unir dados de diferentes DataFrames sendo produzidos com os recortes do DataFrame original, bem como do DataFrame proveniente do OHE utilizando o método **get_dummies()**, também do Pandas.

```

> Mi
df_OHE = pd.concat([df[['Cliente', 'Região']], pd.get_dummies(df['Mês']), df[['Valor Compra', 'Valor Imposto']]], axis=1)
> Mi
df_OHE.head(5)

```

Cliente	Região	Abril	Agosto	Dezembro	Fevereiro	Janeiro	Julho	Junho	Maio	Março	Novembro	Outubro	Setembro	Valor Compra	Valor Imposto
0 Cliente 4	Nordeste	0	0	0	0	0	0	1	0	0	0	0	0	0.002273	0.002273
1 Cliente 1	Centro-Oeste	0	0	0	0	0	0	1	0	0	0	0	0	0.069139	0.069139
2 Cliente 5	Nordeste	0	0	0	0	0	0	0	0	0	0	0	1	0.348724	0.348724
3 Cliente 10	Sudeste	0	0	0	0	0	0	0	0	1	0	0	0	0.064024	0.064024
4 Cliente 13	Sul	0	0	0	0	0	0	0	0	0	1	0	0	0.177108	0.177108

```

> Mi
df.shape, df_OHE.shape
((600, 5), (600, 16))

```

Figura 16 - One Hot Encoding.
Fonte: elaborado pelo autor.

Observe os valores de **df.shape**, **df_OHE.shape** na figura 16 e faça uma análise se existe uma diferença na dimensão dos DataFrames. Procure identificar a causa disso e entender como as colunas com muitas categorias podem afetar o OHE.

Assista

Acesse na plataforma o vídeo: Transformação de variáveis categóricas

Importante: para compreender o tutorial deste vídeo, você deverá acessar os arquivos **jupyter_notebook_un5_top3.ipynb** e **TA_PRECO_MEDICAMENTO.xlsx** disponíveis na plataforma.

4 REDUÇÃO DE DIMENSIONALIDADE

A redução de dimensionalidade tem como objetivo diminuir a sobrecarga computacional empregada pelos métodos de aprendizado de máquina, de forma a se obter resultados mais rápidos. Pense agora em problemas envolvendo o Big Data e potenciais conjuntos de dados de dezenas de Gigabytes. As principais formas de redução de dimensionalidade podem ser feitas utilizando:

- **Frequência de dados ausentes:** pouco provável que colunas com muitos dados ausentes possam conter informação relevante. Portanto, as colunas com número de dados ausentes acima de um limiar pré-definido podem ser removidas.
- **Filtro de baixa variância:** as colunas que contêm pouca variância têm baixa propensão a terem informação relevante para o aprendizado. Portanto, as colunas com variância inferior a um limiar pré-definido podem ser removidas. Para ter mais certeza ao adotar esse método, é necessária a normalização das colunas previamente.
- **Filtro de alta correlação:** colunas com alta correlação entre si significam que potencialmente estão agregando informações muito similares. Neste caso, somente uma delas precisa ser mantida no conjunto de dados. Portanto, colunas com correlação muito próximas e acima de um limiar pré-definido podem ser removidas. Esse método é muito sensível à escala dos valores e, por isso, é necessária a normalização das colunas previamente.
- **Filtro de colunas de forma incremental:** desta forma, você pode iniciar a construção do modelo e aferir a performance utilizando uma coluna do conjunto de dados e ir incrementando-as, aferindo a performance do modelo até que se pare o avanço. Outra maneira é fazer de forma contrária, iniciando com todas as colunas e, uma a uma, ir eliminando a participação das colunas no modelo.
- **Filtro com ensembles:** pode parecer estranho, mas esta também é uma forma, em que se utiliza um grupo de classificadores e verificam-se as colunas mais utilizadas em todos ou na maioria dos casos, o que indicará um maior apetite por determinado subconjunto de colunas.

- **Análise de Componentes Principais (PCA):** é um método estatístico que transforma ortogonalmente as coordenadas originais de um conjunto de dados em um novo conjunto de coordenadas chamadas de componentes principais, os quais são ordenados de forma a propiciar a maior variação possível. O que se faz é remover os componentes de menor ordem por agregar menor variância e, portanto, menos informação.



Assista

Acesse na plataforma o vídeo: Redução de dimensionalidade

Importante: para compreender o tutorial deste vídeo, você deverá acessar os arquivos `jupyter_notebook_un5_top4.ipynb` e `TA_PRECO_MEDICAMENTO.xlsx` disponíveis na plataforma.

CONSIDERAÇÕES FINAIS

Nesta unidade, você aprendeu conceitos importantes sobre: o tratamento de dados faltantes; a normalização e a padronização; a transformação de variáveis categóricas; e a redução de dimensionalidade. Lembre-se sempre do conceito da caixa de ferramentas, ela está crescendo em funcionalidades na medida em que você avança com os seus estudos.

A sabedoria e o sucesso acontecem quando se escolhe as ferramentas corretas. Chamo especial atenção para a redução de dimensionalidade. Eu diria que ela é praticamente um mundo à parte, que merece um estudo aprofundado para extrair todo o seu potencial.

GLOSSÁRIO

ETL: Extração, Transformação e Carregamento.

OHE: One Hot Encoding.

PCA: Principal Component Analysis.

REFERÊNCIAS

GÉRON, Aurélien. **Mãos à Obra: Aprendizado de Máquina com Scikit-Learn & TensorFlow**. 1. ed. Rio de Janeiro: Alta Books, 2019. 576 p.

MCKINNEY, Wes. **Python para Análise de Dados: Tratamento de Dados com Pandas, NumPy e IPython**. 1. ed. São Paulo: Novatec, 2018. 616 p.

Unidade 6

Seleção de modelo

Objetivo de aprendizagem:

Conhecer o método de aprendizagem baseado em decisão, compreender as técnicas de validação de modelo e como utilizá-las, bem como avaliar o erro associado ao modelo (overfitting e underfitting).

Tópicos de estudo:

- Árvores de decisão;
- Validação de modelo;
- Hiperparametrização;
- Overfitting e underfitting.

Iniciando os estudos:

Continuando nossa jornada no mundo da Ciência de dados, quero lhe apresentar o que eu acredito ser o momento principal do curso. Claro que existe muito mais conhecimento adiante e em outras fontes, mas, para um cientista de dados, alguém que conhece a Inteligência Artificial e os algoritmos de aprendizado – principalmente aprendizado de máquina supervisionado –, conhecer a respeito de árvores de decisão –como implementar, interpretar e o seu poder –, é fundamental. Já vimos também como validar o modelo utilizando a divisão em treinamento e teste. Na oportunidade, dividimos o conjunto de dados original em 80% de treinamento e 20% de teste. Agora, vamos aprofundar a validação dos modelos com outras maneiras estatisticamente mais válidas. Você também já deve ter percebido que na nossa caixa de ferramentas cada ferramenta tem suas configurações. Por exemplo, no KNN tem o hiperparâmetro K. Então como escolher? Em outras ferramentas, digo, algoritmos, cada qual com seus hiperparâmetros. E, também, como o uso dos hiperparâmetros associados às escolhas corretas de conjuntos de treinamento e teste, sob a ótica da validação, podem levar a resultados conhecidos como overfitting e underfitting. Bons estudos!

1 ÁRVORES DE DECISÃO

Árvores de decisão são um tipo de algoritmo de aprendizado de máquina supervisionado, ou seja, procura fazer uma relação entre a **variável dependente** e as **variáveis independentes**. São usadas, portanto, para problemas de classificação em que você tem uma variável classe ou rótulo e outras variáveis que servem para determinar a quais classes os exemplos pertencem com base nessas árvores (FACELI *et al.*, 2011).

Uma árvore de decisão tem o mesmo aspecto de outras árvores que você conhece na computação, como as árvores binárias e árvores B. Na figura 1, você pode ver um esquema representativo de uma árvore de decisão. Os nós são compostos pelas variáveis independentes. Para cada nó, existem condições sendo testadas que levam a novos nós, com novas condições. Os nós folhas são as variáveis dependentes, os rótulos.

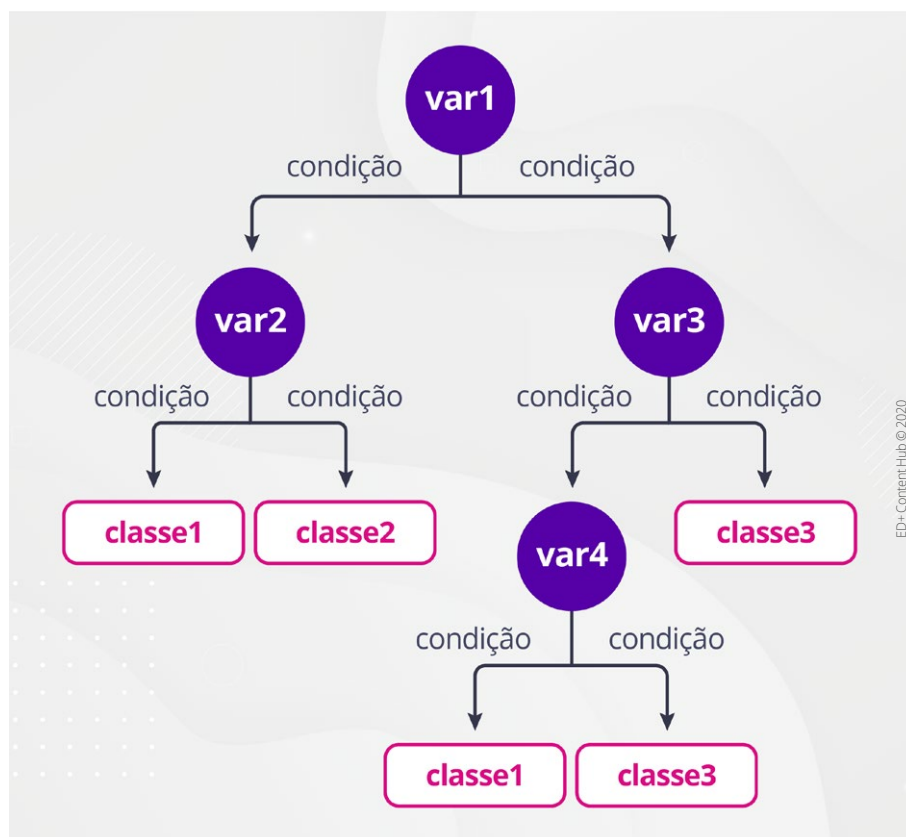


Figura 1 - Árvore de decisão.
Fonte: elaborado pelo autor.

Árvores de decisão recebem esse nome pela própria estrutura dos caminhos que as definem, fazendo com que a decisão do pertencimento do exemplo a uma determinada classe dependa das decisões dos caminhos escolhidos em cada nó.

Aprofunde-se

As árvores de decisão são uma maneira simples de converter uma tabela de dados que você possui na sua mesa em um meio de prever e classificar novos dados à medida que eles chegam. Confira o vídeo para descobrir como. Lembrando que o vídeo está em inglês mas possui geração automática de legenda em português.

Título: Árvores de decisão

Acesso em: 10/08/2020.

Disponível em: <https://youtu.be/7VeUPuFGJHk>

Tornando mais claro, veja o exemplo na figura 2 para o conjunto de dados Iris. Lembre-se que esse conjunto tem 150 exemplos, com 3 classes de flores da família Iris (Iris-virginica, Iris-setosa e Iris-versicolor), bem como 4 variáveis independentes que as definem (petallength, petalwidth, sepallength e sepalwidth). Na árvore de decisão gerada, considerando o primeiro ramo mais à esquerda na figura 2, você pode ler da seguinte forma, como uma regra condicional:

se **petalwidth** ≤ 0.6 , então **classe** = **Iris-setosa**

Outro ramo, à direita agora:

se **petalwidth** > 0.6 e **petalwidth** > 1.7 , então **classe** = **Iris-virginica**

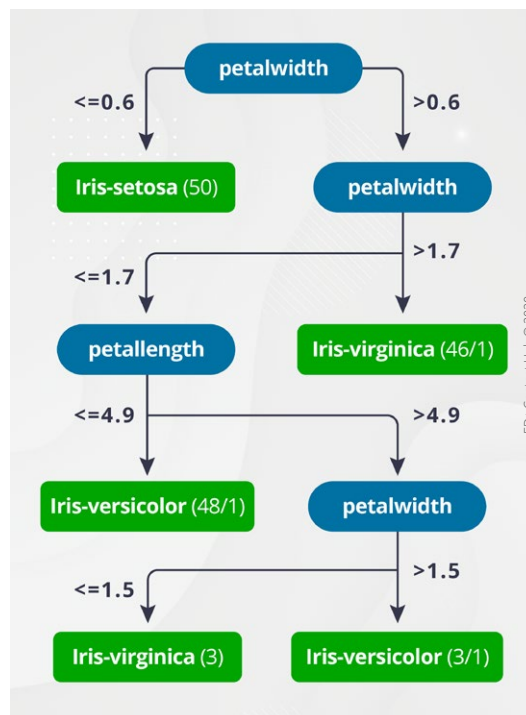


Figura 2 - Árvore de decisão para conjunto de dados Iris.
Fonte: adaptado de: <https://bit.ly/3e3zgKG>

Reflita

O conjunto de dados Iris possui as variáveis independentes petallength, petalwidth, sepaldenght e sepalwidth. Veja na figura 2 quais foram utilizadas para montar a árvore de decisão. Retome os conceitos de redução de dimensionalidade e reflita a partir desse exemplo de árvore de decisão como pode ser feita a redução de dimensionalidade baseada em modelos.

As árvores de decisão têm a grande vantagem de produzirem um classificador com modelo simbólico. O **modelo simbólico** diz respeito à inteligibilidade do que o modelo expressa (MITCHELL, 1997). Em outras palavras, ao ler o modelo, é possível entender e interpretar o conhecimento extraído dos dados. Isso é muito diferente quando trabalhamos com redes neurais artificiais ou regressão. O fato de ser facilmente interpretável também pode ser uma desvantagem se o modelo ficar muito complexo, de forma que existam muitas variáveis e ramos.

Você deve estar se perguntando: “Ok, mas como saber a variável que vai em cada nó?”. Existem diversas medidas para isso, sendo a entropia e o índice de Gini os mais utilizados. As métricas procuram os melhores atributos, com os intervalos ideais, de forma a produzir hiperplanos que recortam o espaço multidimensional que define os exemplos.

Na figura 3, considerando um espaço tridimensional para o conjunto de dados Iris e, por isso, utilizando somente três variáveis para visualização, veja como os hiperplanos são utilizados para dividir os exemplos, cada qual com sua classe. Em um hiperespaço multidimensional, existem multi-hiperplanos separando os exemplos em suas classes.

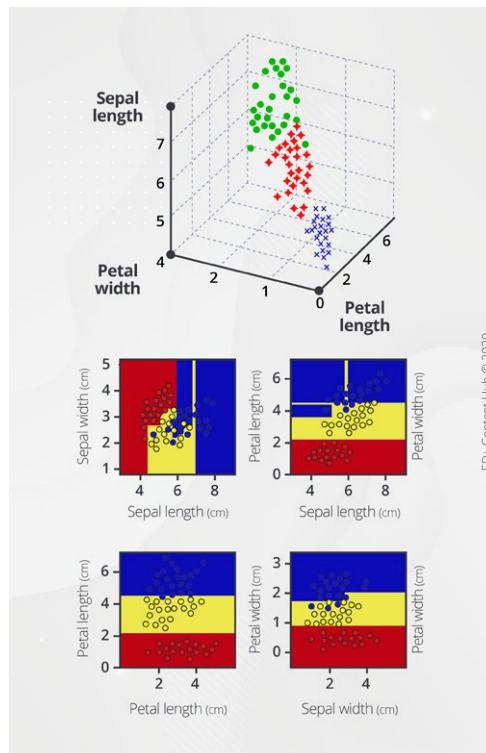


Figura 3 - Hiperplanos da árvore de decisão.

Fonte: Elaborado pelo autor.

Como exemplo de forma para aplicar os conceitos em Python, vamos utilizar o conjunto de dados **iris.csv** e executar os comandos dentro do **Jupyter Notebook**. A figura 4 mostra os comandos já conhecidos para o carregamento dos dados a partir de um arquivo **CSV**.

```

> ML
import pandas as pd
import numpy as np

> ML
df = pd.read_csv('iris.csv')
df.head(10)

```

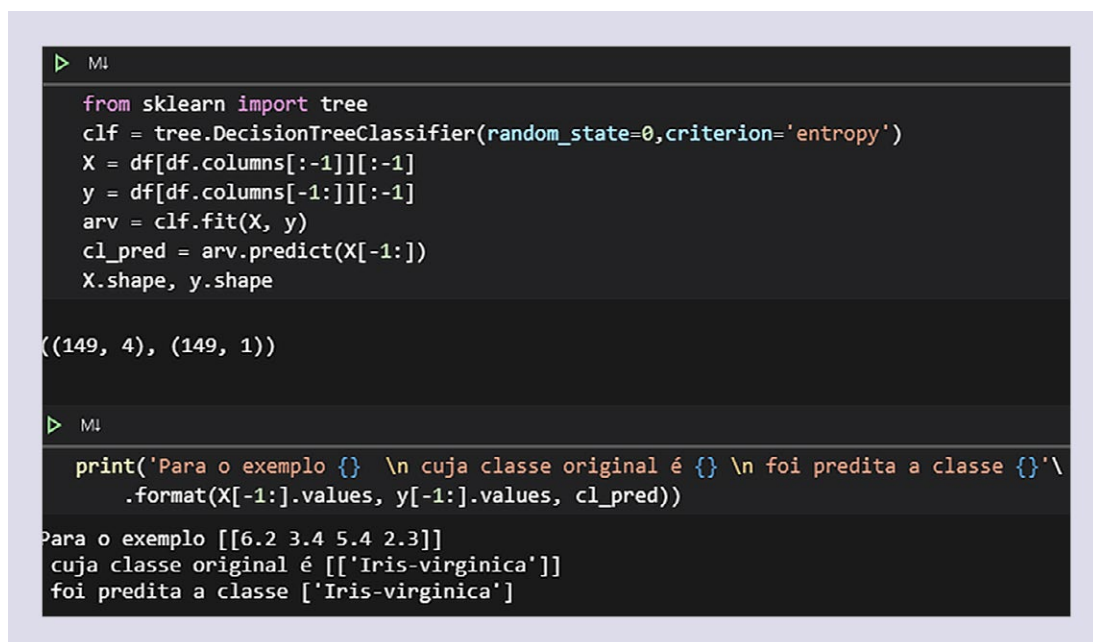
	sepal_length	sepal_width	petal_length	petal_width	class
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
5	5.4	3.9	1.7	0.4	Iris-setosa
6	4.6	3.4	1.4	0.3	Iris-setosa
7	5.0	3.4	1.5	0.2	Iris-setosa
8	4.4	2.9	1.4	0.2	Iris-setosa
9	4.9	3.1	1.5	0.1	Iris-setosa

Figura 4 - Carregamento do conjunto de dados 'iris.csv'.

Fonte: elaborado pelo autor.

Na figura 5 é que aparece a novidade sobre a árvore de decisão. Primeiramente, há a importação de toda uma família de classes da biblioteca **sk-learn**. Em seguida, é instanciado em **clf** um objeto do tipo **DecisionTreeClassifier** (McKINNEY, 2018). Os parâmetros são a semente de inicialização para valores aleatórios e a métrica utilizada, que pode ser **'entropy'** ou **'gini'**. Em **X**, é filtrado somente as variáveis independentes com seus valores e em **y** somente a variável dependente (classe) e seus valores. Confira isso pelo resultado das tuplas de **X.shape** e **y.shape**.

Ao executar o método **fit** em **clf**, a árvore de decisão será treinada com os valores de **X** e **y**. Vale notar que ao separar **X** e **y** eu deixei de propósito o último exemplo (dos 150 totais) de fora e, por isso, tem 149 linhas no shape e não 150. Essa última linha, cuja classe é desconhecida para o classificador, eu usei na chamada do método **predict** para mostrar para você se o classificador seria capaz de acertar essa classe com o uso do classificador. Finalmente, veja na saída do comando **print** qual mostra o exemplo desconhecido, a classe verdadeira e a classe predita. Não é que o classificador acertou!



```

> M1
from sklearn import tree
clf = tree.DecisionTreeClassifier(random_state=0,criterion='entropy')
X = df[df.columns[:-1]][:-1]
y = df[df.columns[:-1]][:-1]
arv = clf.fit(X, y)
cl_pred = arv.predict(X[-1:])
X.shape, y.shape

((149, 4), (149, 1))

> M1
print('Para o exemplo {} \n cuja classe original é {} \n foi predita a classe {}'\
      .format(X[-1:].values, y[-1:].values, cl_pred))

Para o exemplo [[6.2 3.4 5.4 2.3]]
cuja classe original é [['Iris-virginica']]
foi predita a classe ['Iris-virginica']

```

Figura 5 - Treinamento da árvore de decisão e teste.
Fonte: elaborado pelo autor.

Se você quiser visualizar a árvore de decisão, isso é possível com o método **plot_tree**. Na figura 6 temos um exemplo disso. São vários parâmetros para você personalizar a saída, sendo que é obrigatório informar qual é o classificador que você deseja visualizar. Foram utilizados os parâmetros **rounded** para mostrar quadrados com cantos arredondados, **fontsize** para ajustar o tamanho da fonte, **class_names** para mostrar o nome das classes e **feature_names** para mostrar o nome das variáveis. Também veja que foi utilizado o método **subplot** para poder configurar uma imagem maior a ser visualizada. Se você não utilizar esses argumentos, pode ser que a visualização fique prejudicada por conta do tamanho da imagem e da fonte que, por padrão, é pequena.

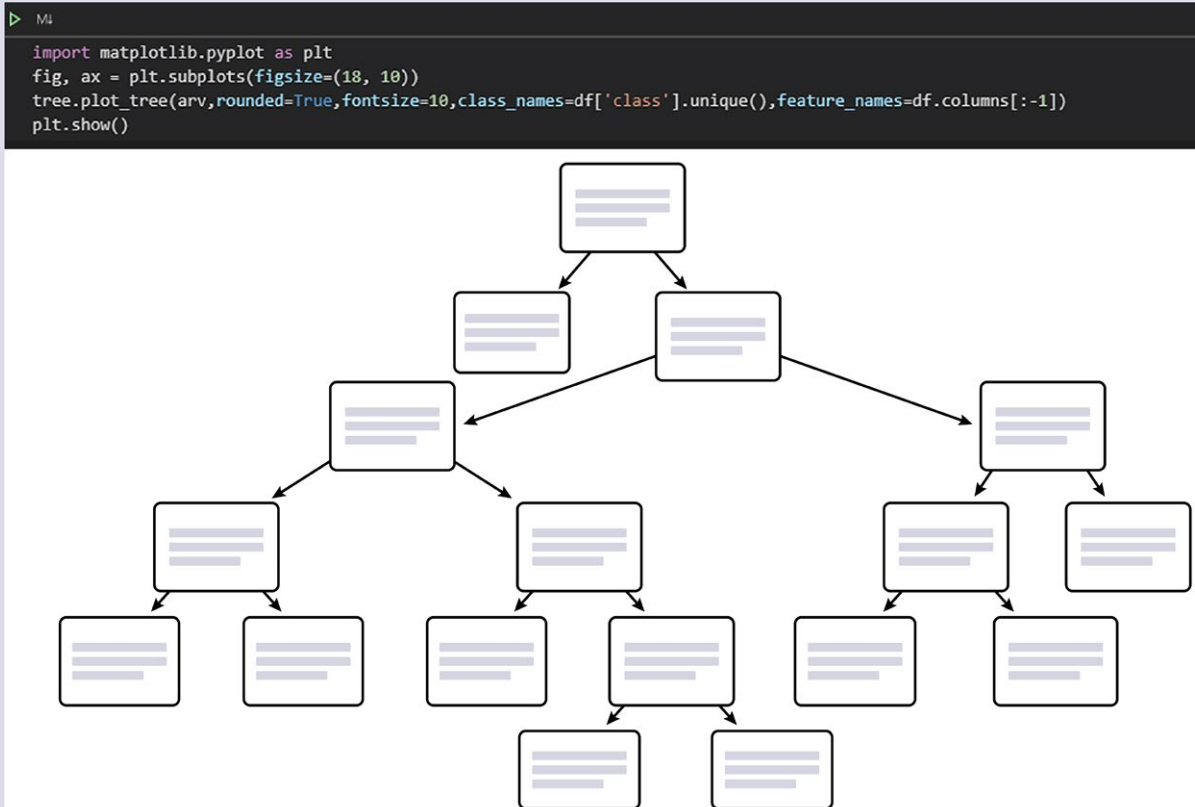


Figura 6 - Visualização da árvore de decisão.
Fonte: elaborado pelo autor.

Assista

Acesse na plataforma o vídeo: Árvore de decisão

Importante: para compreender o tutorial deste vídeo, você deverá acessar os arquivos `Jupyter_notebook_un6.ipynb` e `adult.csv` disponíveis na plataforma.

2 VALIDAÇÃO DE MODELO

No exemplo anterior, sobre como construir uma árvore de decisão, não me preocupei em dividir os dados em treinamento e teste porque eu apenas queria lhe mostrar como fazer uma árvore de decisão, testar um exemplo e produzir uma visualização. Agora, precisamos entender melhor a questão da validação dos modelos e as possibilidades.

2.1 SPLIT SAMPLE

Já vimos isso em outro momento – quando dividimos o conjunto de dados inicial em treinamento e teste. Usualmente, adota-se, nesse caso, uma divisão de 80% dos dados para treinamento e 20% para teste. Pode ser que você encontre exemplos em que a divisão ocorra em outras proporções também, como {50%,50%}, {70%,30%} e {75%,25%}. Como disse a você, não há correto e nem incorreto, tudo depende do conjunto de dados e do modelo a ser empregado. É praticamente empírico (FACELI *et al.*, 2011).

2.2 K-FOLD CROSS-VALIDATION

O método de validação cruzada é o mais utilizado na academia e entre os profissionais por permitir uma estimativa bem realista do desempenho futuro do classificador (MITCHELL, 1997). Nesse método, o conjunto de dados inicial é dividido em **K** partições. O mais comum é que sejam divididos em 10 partições de forma que **K = 10**. O algoritmo de aprendizado então é executado **K** vezes, sendo que a cada vez o algoritmo utiliza **K-1** partições para treinamento e 1 partição para teste. Isso faz com que cada partição participe **K-1** vezes do treinamento e 1 vez do teste.

Interessante é que a cada rodada temos uma estimativa do erro do classificador sobre os dados de teste, e ao final teremos **K** estimativas de erro que são utilizadas para chegar à média e ao desvio padrão da estimativa de erro do classificador (ou complementarmente à acurácia).

Cada partição terá **(100/K)%** dos exemplos do conjunto de dados original, sem repetição. Para também evitar qualquer viés na definição das partições, é importante que a separação seja feita de forma aleatória e sem repetição ou com prévio embaralhamento dos dados. Imagine o conjunto de dados Iris que vem com 50 exemplos de cada classe de forma ordenada. Separar em **K-fold** (partições), seguindo a ordem dos exemplos, vai resultar em partições que têm somente exemplos de uma determinada classe. A figura 7 ilustra o processo de iteração da indução do classificador com **(k-1)** partições e o teste com 1 partição, diferente a cada iteração.

	Teste	Treinamento									Erro (%)
Iteração 1	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10	E1
Iteração 2	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10	E2
Iteração 3	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10	E3
Iteração 4	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10	E4
Iteração 5	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10	E5
Iteração 6	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10	E6
Iteração 7	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10	E7
Iteração 8	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10	E8
Iteração 9	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10	E9
Iteração 10	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10	E10

Figura 7 - 10 fold cross-validation.

Fonte: elaborado pelo autor.

Ao final da avaliação por 10 fold cross-validation, conforme exemplo na figura 7, obtêm-se as avaliações para os erros futuros calculados a partir dos dados de teste no classificador, induzidos sobre os dados de treinamento. A partir da média desses erros, é possível obter uma taxa de erro **E** e desvio padrão **d**, tal que se espera que, para o classificador induzido, a performance classifique corretamente exemplos futuros (não vistos) que possam ocorrer com taxa de erro **E** e desvio padrão **d** (FACELI *et al.*, 2011).

Para implementar em Python a validação cruzada, vamos retomar o exemplo anterior com o conjunto de dados Iris e a árvore de decisão. Na figura 8, temos o uso da biblioteca sk-learn e a classe **cross_val_score** (GÉRON, 2019). Ao utilizar o construtor da classe, deve-se passar como parâmetro o classificador para calcular as estimativas dos erros, os dados com as variáveis independentes (**X**), os dados somente com a variável dependente (**classe, y**) e em **cv** o número de partições (no caso 10). Essa simples linha já faz todo o processo e retorna um array com as estimativas dos erros, restando para nós somente calcular a média e o desvio padrão. Simples demais.

```

> ML

from sklearn import tree
clf = tree.DecisionTreeClassifier(random_state=0,criterion='gini')
X = df[df.columns[:-1]][:-1]
y = df[df.columns[-1:]][:-1]
arv = clf.fit(X, y)
cl_pred = arv.predict(X[:-1])

> ML

from sklearn.model_selection import cross_val_score
scores = cross_val_score(clf, X, y, cv=10)
print("Accuracy: %0.2f (+/- %0.2f)" % (scores.mean(), scores.std() ))

Accuracy: 0.96 (+/- 0.04)

> ML

from sklearn.model_selection import StratifiedKFold
strat_k_fold = StratifiedKFold(n_splits=10, shuffle=True, random_state=0)
scores = cross_val_score(clf, X, y, cv=strat_k_fold)
print(" Accuracy: %0.2f (+/- %0.2f)" % (scores.mean(), scores.std() ))

Accuracy: 0.93 (+/- 0.06)

```

Figura 8 - sk-learn 10 fold cross-validation e stratified cross-validation.

Fonte: elaborado pelo autor.

Ainda na figura 8, se você quiser utilizar um particionamento dos dados de forma embaralhada (shuffle) e ainda que preserve a representatividade da distribuição das classes (stratified), você pode utilizar a classe **StratifiedKFold** para criar um método diferenciado de cross-validation. Ao executar depois o construtor para **cross_val_score**, ao invés de determinar que serão 10 partições (cv=10), basta informar que o **cv** será igual ao objeto criado da classe **StratifiedKFold** (no exemplo, **cv=strat_k_fold**).

Veja que, para cada tipo de estimativa de erro, está sendo mostrada a acurácia e o desvio padrão, com leve vantagem para **StratifiedKFold**, como esperado, por conta do melhor balanceamento da distribuição das classes em cada partição, além da questão de os dados estarem embaralhados.

Aprofunde-se

Um dos conceitos fundamentais do aprendizado de máquina é a validação cruzada. É assim que decidimos qual método de aprendizado de máquina seria melhor para nosso conjunto de dados. Confira o vídeo para descobrir como. Lembrando que o vídeo está em inglês, mas com geração automática de legenda em português.

Título: Fundamentos de aprendizado de máquina: validação cruzada

Acesso em: 08/08/2020.

Disponível em: <https://youtu.be/fSytzGwwBVw>

2.3 LEAVE-ONE OUT

Pode-se dizer que essa metodologia de validação é um caso específico do k-fold cross-validation pelo fato de que as partições são criadas de acordo com o número total de exemplo, exceto um. Ou seja, para um conjunto de dados com n exemplos, teríamos n partições de treinamento de tamanho $(n-1)$ e n partições de teste com um único exemplo. Também teríamos n iterações, com n taxas de erro do classificador para calcular a média e o desvio padrão finais (MITCHELL, 1997).

Essa metodologia é a que mais leva a uma aproximação do erro real em exemplos futuros, embora tenha o terrível inconveniente de ser computacionalmente custoso para conjuntos de dados grandes. Imagine um conjunto de dados com 1 milhão de exemplos. Seriam 999.999 partições e iterações. Haja CPU!

2.4 HOLD OUT

Utilizar a validação por hold out envolve dividir o conjunto de dados em uma partição dita imaculada para executar uma outra metodologia de avaliação, digamos k-fold cross-validation. A ideia consiste em separar uma partição inicial, digamos 10% dos dados. Os demais 90% tornam-se a base de dados amostral para executar o processo de 10 fold cross-validation. O erro para exemplos futuros passa então a ser tão somente o erro do classificador sobre a partição de hold out.

A vantagem dessa metodologia é que o classificador será validado sobre dados nunca vistos. Já a desvantagem é que a estimativa do erro será somente sobre uma partição, descartando as estimativas obtidas utilizando k-fold cross-validation.

Assista

Acesse na plataforma o vídeo: Avaliação do modelo.

Importante: para compreender o tutorial deste vídeo, você deverá acessar os arquivos `Jupyter_notebook_un6.ipynb` e `adult.csv` disponíveis na plataforma.

3 HIPERPARAMETRIZAÇÃO

Há quem diga que na vida tudo é relativo. Na computação, com certeza tudo é relativo, pois depende dos parâmetros. Ao longo do que temos visto até o momento, você se recorda que para o **KNN** precisamos definir o **K**. Para uma árvore de decisão, podemos definir o critério (gini ou entropia). Para o k-fold cross-validation, tem o parâmetro **K** também que, nesse caso, é o número de partições.

Ainda bem que a biblioteca sk-learn fornece esse tipo de facilidade. Na figura 9, estamos importando a classe **GridSearchCV** para fazer a combinação de parâmetros. Os parâmetros são combinados por meio de uma lista em que cada item da lista é um dicionário com as chaves e os possíveis valores. As chaves são, na verdade, os parâmetros a serem testados com diferentes valores.

```
▶ MI
from sklearn.model_selection import GridSearchCV
# Set the parameters by cross-validation
tuned_parameters = [{'criterion': ['gini'], 'max_depth': [1, 2, 3, 4]},
                    {'criterion': ['entropy'], 'max_depth': [1, 2, 3, 4]}]
clf = GridSearchCV(tree.DecisionTreeClassifier(), tuned_parameters)
arv = clf.fit(X, y)
clf.best_params_

{'criterion': 'gini', 'max_depth': 3}
```

Figura 9 - Busca de hiperparâmetros.
Fonte: elaborado pelo autor.

Tendo a lista de parâmetros, basta você chamar o construtor da classe **GridSearchCV** com o classificador. Nesse exemplo, usei um outro parâmetro para construir a árvore de decisão, que é o **max_depth**, que indica a profundidade máxima que ela pode assumir. Note que, nesse exemplo da figura 9, estamos testando a melhor árvore de decisão com 4 combinações diferentes de profundidade e 2 combinações diferentes de critério para métrica. Portanto, são 8 árvores de decisão sendo geradas.

Por fim, **best_params_** é um atributo do classificador **clf** que armazena um dicionário contendo os parâmetros que produzem a melhor árvore (nesse caso, por padrão, acurácia).

Aprofunde-se

Neste vídeo, você aprenderá a pesquisar com eficiência os parâmetros de ajustes ideais (ou hiperparâmetros) para seu modelo de aprendizado de máquina a fim de maximizar seu desempenho. Será demonstrado um processo exaustivo de "pesquisa em grade" usando a classe **GridSearchCV** do sk-learn e depois a **RandomizedSearchCV** que, geralmente, pode alcançar resultados semelhantes em muito menos tempo.

Lembrando que o vídeo está em inglês, mas com geração automática de legenda em português.

Título: How to find the best model parameters in scikit-learn

(Como encontrar os melhores parâmetros de modelo no scikit-learn)

Acesso em: 10/08/2020.

Disponível em: https://youtu.be/Gol_qOgRqfA

Assista

Acesse na plataforma o vídeo: Hiperparâmetros.

Importante: para compreender o tutorial deste vídeo, você deverá acessar os arquivos **Jupyter_notebook_un6.ipynb** e **adult.csv** disponíveis na plataforma.

4 OVERFITTING E UNDERFITTING

Os critérios de avaliação do classificador buscam entender como ele se ajusta (fit) aos dados de forma a explicar o conhecimento intrínseco nos exemplos e ter a capacidade de prever os exemplos futuros cuja classe é desconhecida.

Entretanto, como vimos, escolher os parâmetros, partições e critérios de avaliação, bem como o modelo de aprendizado, tudo isso que leve ao melhor classificador para um determinado conjunto de dados, não parece nada trivial.

Se o conjunto de dados é amplamente utilizado, como o Iris, você vai encontrar toneladas de artigos e metodologias explorando esses exemplos. Então fica mais fácil você determinar se o seu modelo de classificador produz um bom ajuste (fit). Se o seu classificador obtiver desempenho pior sobre os dados de treinamento comparado à maioria dos resultados dos outros métodos, definitivamente, o método de aprendizado não está aprendendo adequadamente e você tem em mãos um terrível caso de **underfitting**. Quando isso ocorre, o resultado é pobre já no treinamento, e no teste tende a ser tão ruim quanto ou pior.

Já no caso do **overfitting**, pode ser que você fique feliz por um tempo. Isso ocorre porque, quando o método de aprendizado produz um classificador sobreajustado (overfitting), o resultado sobre os dados de treinamento é excepcional, mas quando o classificador é aplicado sobre os dados de teste, o desempenho é muito pior do que no treinamento. Isso ocorre porque o classificador produziu hiperplanos muito ajustados para separar os dados. No caso das árvores, e em exemplos desconhecidos, isso não produz o que chamamos de generalização. Ou seja, o classificador entende bem dos dados de treinamento, mas não consegue lidar com os dados de teste (MITCHELL, 1997).

Veja na figura 10 uma ilustração do que seria o underfitting (subajuste), o fit (ajuste) e o overfitting (sobreajuste ou superajuste).

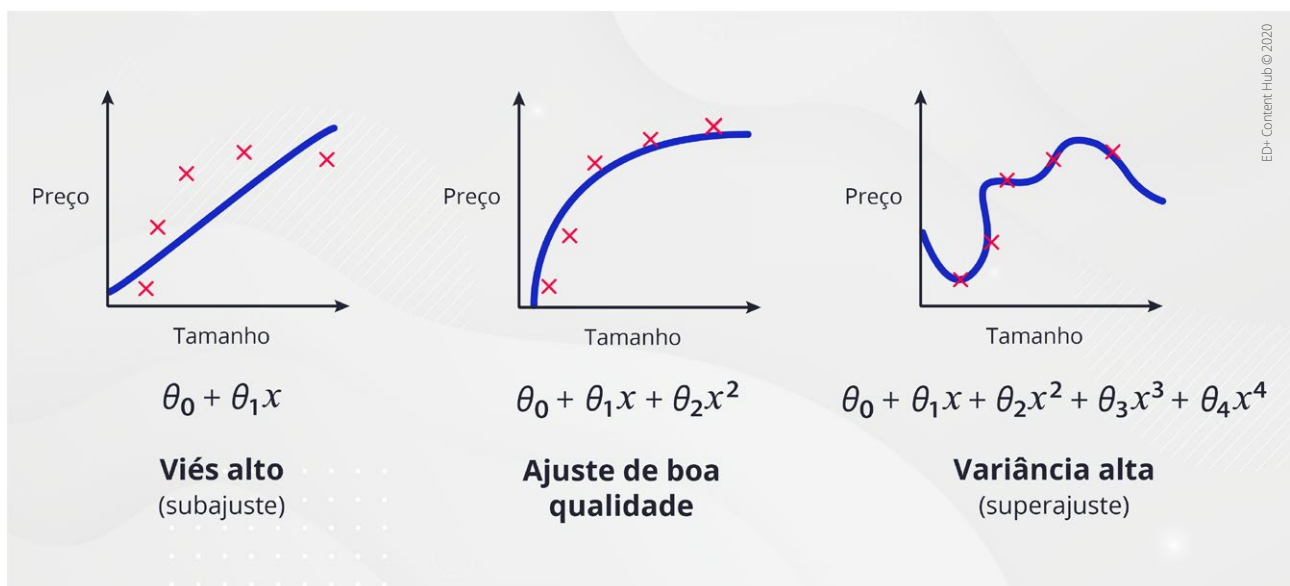
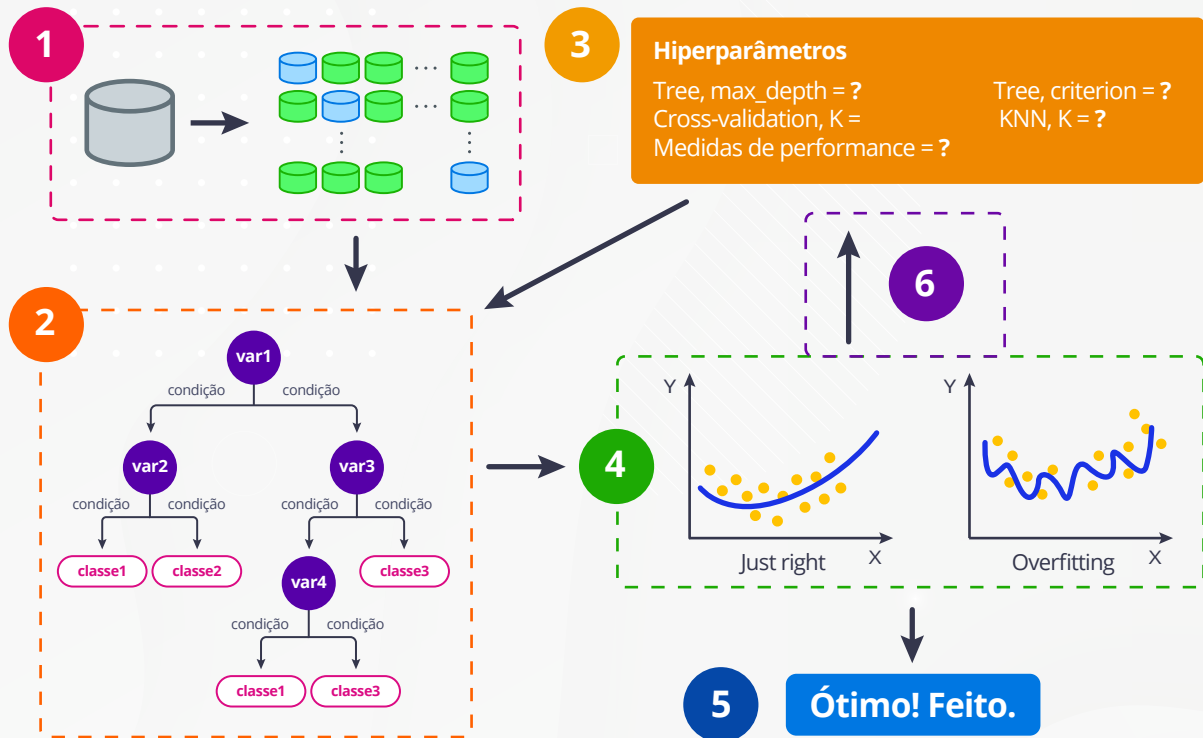


Figura 10 - Exemplo de underfitting, fit e overfitting.
 Fonte: adaptado de <https://i.stack.imgur.com/ml3cs.png>

Escolha do modelo, avaliação e tuning dos hiperparâmetros

As ferramentas de Ciência de dados, no que tange aos algoritmos, critérios de avaliação e setup dos hiperparâmetros, resultam em inúmeras possibilidades de escolha. Entender como esse fluxo funciona, na busca dos melhores resultados, faz de você um cientista de dados.



1 O conjunto de dados inicial é dividido já pensando na avaliação. Ou seja, aqui já é necessário pensar em k-fold cross-validation, leave-one out, hold out e split sample.

2 De acordo com o modelo de avaliação, os dados serão utilizados para induzir o classificador e avaliá-lo, o que pode ser computacionalmente bem custoso se forem muitos processos com um conjunto de dados grande.

3 Os hiperparâmetros juntam-se ao passo anterior para que as configurações a serem adotadas na geração do modelo façam sentido e possam produzir o resultado esperado.

4 O classificador é gerado a partir dos dados e com o algoritmo de aprendizado de máquina configurado com os hiperparâmetros.

5 Se o resultado da avaliação final do modelo mostra que o classificador se ajustou ao modelo e, portanto, consegue prever com a mesma exatidão as classes de dados futuros, ótimo. Temos um modelo.

6 Se o resultado produz overfitting, ou seja, o classificador tem um bom resultado no treinamento mas é sofrível em prever a classe de dados futuros, então é hora de voltar para a prancheta e reavaliar, no mínimo, os hiperparâmetros.

Infográfico 1 - Escolha do modelo, avaliação e tuning dos hiperparâmetros.
Fonte: elaborado pelo autor.

Por isso que a combinação de técnicas de validação como o k-fold, cross-validation e a seleção de hiperparâmetros de forma automatizada podem auxiliar no desenvolvimento de grupos de teste a fim de se obter classificadores bem ajustados aos exemplos de treinamento e com capacidade de predição de exemplos desconhecidos.

Aprofunde-se

Viés e variância são dois conceitos fundamentais em aprendizagem de máquina, e a intuitividade deles pode ser um pouco diferente do que você provavelmente aprendeu nas aulas de estatística. Aqui eu uso dois exemplos que tornam esses conceitos super simples de se entender.

Lembrando que o vídeo está em inglês mas com geração automática de legenda em português.

Título: Fundamentos de aprendizagem de máquina: viés e variância

Acesso em: 08/08/2020.

Disponível em: <https://youtu.be/EuBBz3bI-aA>

Assista

Acesse na plataforma o vídeo: Underfitting e overfitting

Importante: para compreender o tutorial deste vídeo, você deverá acessar os arquivos **Jupyter_notebook_un6.ipynb** e **adult.csv** disponíveis na plataforma.

CONSIDERAÇÕES FINAIS

Nesta unidade, você aprendeu conceitos importantes sobre árvores de decisão, validação de modelo e hiperparametrização, bem como as questões associadas ao overfitting e underfitting dos modelos. Existem muitas possibilidades de ajustes dos modelos e resultados quando colocamos tudo junto na mesma caixa. Agora a questão é pensar em como produzir um modelo, fazendo a escolha correta dos hiperparâmetros de forma a produzir bons resultados de acurácia, erro, recall, f-score e forma para que o modelo seja ajustado aos dados e com boa previsão futura, ou seja, sem overfitting. Por isso é chamada de Ciência de dados essa maravilhosa área. Procure fazer novos testes, com novos conjuntos de dados. Bons estudos!

GLOSSÁRIO

CSV (Comma-Separated Values): é um tipo de arquivo muito disseminado porque pode ser carregado e lido em diversos sistemas operacionais e aplicações. Leva esse nome porque os valores são separados por vírgulas.

REFERÊNCIAS

FACELI *et al.* **Inteligência Artificial. Uma Abordagem de Aprendizado de Máquina.** 1. ed. Rio de Janeiro: LTC, 2011.

GÉRON, Aurélien. **Mãos à Obra: Aprendizado de Máquina com Scikit-Learn & TensorFlow.** 1. ed. Rio de Janeiro: Alta Books, 2019. 576 p.

MCKINNEY, Wes. **Python para Análise de Dados:** Tratamento de Dados com Pandas, NumPy e IPython. 1. ed. São Paulo: Novatec, 2018. 616 p.

MITCHELL, Tom. **Machine Learning.** Porto Alegre: Bookman, 1997.

Unidade 7

Regressão

Objetivo de aprendizagem:

Nesta unidade, você vai aprender sobre as técnicas de regressão linear para tratar de problemas cuja variável dependente não tem valores expressos em categorias, mas sim em valores contínuos. E também irá estudar sobre as métricas de avaliação desse tipo de modelo.

Tópicos de estudo:

- Regressão linear;
- Métricas de regressão;
- Outros tipos de regressores;
- Ensemble.

Iniciando os estudos:

Você estudou até aqui métodos simbólicos de aprendizado de máquina supervisionado que procuram explicar, por árvores e regras de decisão, como a variável dependente se relaciona com as variáveis independentes. Assim, há em mãos um problema de classificação no qual a variável rótulo pode ser predita pelo valor das demais variáveis que compõem o conjunto de dados.

Nesta unidade, irei abordar outro problema de aprendizado supervisionado, mas, ao invés do rótulo ser categórico como você viu anteriormente, digamos que agora ele é um valor numérico contínuo que pode ser inteiro ou decimal. Essa é uma característica básica de problemas de regressão: variável dependente numérica.

Você irá estudar também outros tipos de regressão (não lineares) e as métricas de desempenho. Por último, também terá contato com um tópico sobre ensemble, que é uma forma de combinar vários classificadores com o objetivo de melhorar o desempenho.

Bons estudos!

1 REGRESSÃO LINEAR

Em aprendizado de máquina supervisionado, os métodos de regressão representam uma classe de problemas em que se calcula o valor de uma variável numérica com base nos valores de outras variáveis.

Diferente das árvores de decisão, que particionam o espaço amostral em hiperplanos para dividir os exemplos em classes, na regressão linear o valor da classe é obtido pela interpolação de uma reta que prediz esse valor com base nos valores das outras variáveis (FACELI et al., 2011).

Em um exemplo matemático de regressão linear bastante simples, com apenas uma variável dependente e uma independente, a relação entre elas pode ser expressa pela equação $y = b + xw + \epsilon$. O que se quer calcular são os valores b e w , que minimizam o quadrado da norma do vetor ϵ , isto é, minimiza-se a soma dos quadrados dos resíduos (CHARNET et al., 2008).

Basicamente, é esse mesmo tipo de equação linear que o Excel usa quando você faz um gráfico e manda criar uma linha de tendência, conforme a figura 1 abaixo. Se você optar por uma linha de tendência linear, e mostrar o gráfico da equação, é justamente o formato da fórmula do parágrafo anterior. Sobre o R^2 vamos aprender mais à frente.

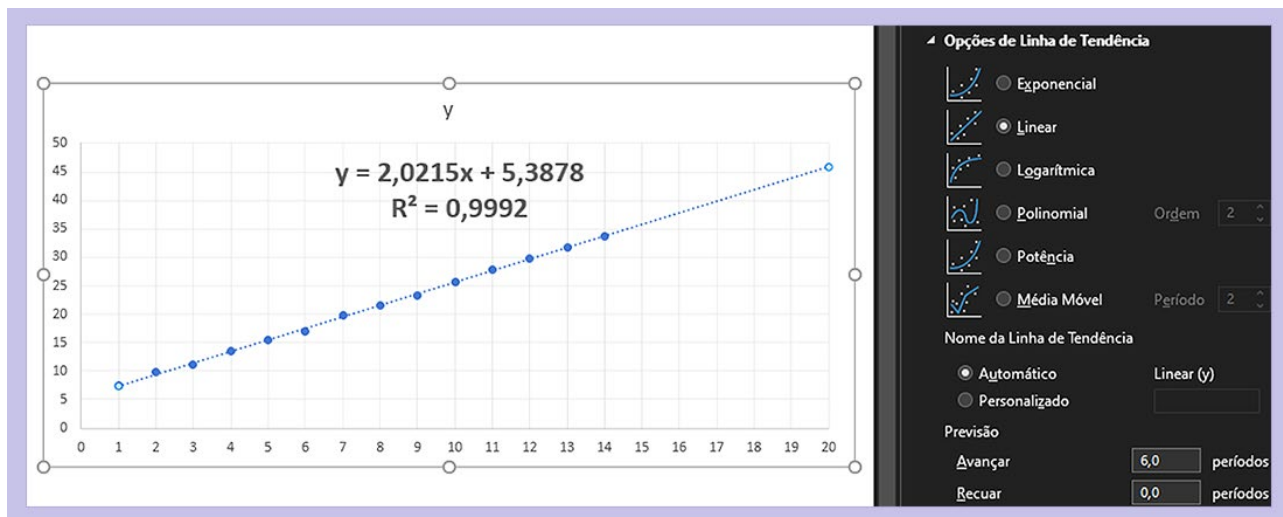


Figura 1 - Exemplo de equação linear no Excel.
Fonte: elaborado pelo autor.

Aprofunde-se

Os conceitos por trás da regressão linear, ajustando uma linha aos dados com mínimos quadrados e R^2 , são bastante simples, então vamos direto ao assunto! Confira o vídeo para descobrir como. Ele está em inglês, mas com geração automática de legenda em português.

Título: Regressão Linear

Acesso em: 15/08/2020.

Disponível em: https://youtu.be/nk2CQITm_eo

Utilizando o Python e a **biblioteca Scikit-learn**, você pode praticar como construir uma regressão linear e entender a influência das variáveis na predição dos valores, bem como entender se o regressor tem uma boa margem de erro por meio de suas métricas. **Vamos lá ao nosso Jupyter Notebook!**

Vamos utilizar um conjunto de dados da UCI Dataset Repository chamado de **Bike Sharing Dataset**. Os sistemas de compartilhamento de bicicletas são a nova geração de aluguel de bicicletas tradicionais, em que todo o processo de adesão, aluguel e devolução se tornou automático. Por meio desses sistemas, o usuário pode facilmente alugar uma bicicleta, em uma determinada posição, e retornar em outra.

Nesse conjunto de dados, iremos utilizar as informações disponíveis para montar um modelo de regressão linear para estimar a quantidade de bicicletas alugadas por dia, com base nas datas e nos dados meteorológicos. O processo será iniciado com a leitura dos dados do arquivo **'day.csv'**, que está no formato **CSV**, para o DataFrame. Esse conjunto de dados possui os dados agrupados por dia sobre o aluguel das bicicletas. Nos vídeos vou utilizar outro conjunto com os empréstimos por hora para maiores insights.

```

> MI
df = pd.read_csv('day.csv')
df.head(10)

```

	instant	dteday	season	yr	mnth	holiday	weekday	workingday	weathersit	temp	atemp	hum	windspeed	casual	registered	cnt
0	1	2011-01-01	1	0	1	0	6	0	2	0.344167	0.363625	0.805833	0.160446	331	654	985
1	2	2011-01-02	1	0	1	0	0	0	2	0.363478	0.353739	0.696087	0.248539	131	670	801
2	3	2011-01-03	1	0	1	0	1	1	1	0.196364	0.189405	0.437273	0.248309	120	1229	1349
3	4	2011-01-04	1	0	1	0	2	1	1	0.200000	0.212122	0.590435	0.160296	108	1454	1562
4	5	2011-01-05	1	0	1	0	3	1	1	0.226957	0.229270	0.436957	0.186900	82	1518	1600
5	6	2011-01-06	1	0	1	0	4	1	1	0.204348	0.233209	0.518261	0.089565	88	1518	1606
6	7	2011-01-07	1	0	1	0	5	1	2	0.196522	0.208839	0.498696	0.168726	148	1362	1510
7	8	2011-01-08	1	0	1	0	6	0	2	0.165000	0.162254	0.535833	0.266804	68	891	959
8	9	2011-01-09	1	0	1	0	0	0	1	0.138333	0.116175	0.434167	0.361950	54	768	822
9	10	2011-01-10	1	0	1	0	1	1	1	0.150833	0.150888	0.482917	0.223267	41	1280	1321

Figura 2 - Importando os dados do arquivo **'day.csv'**.

Fonte: elaborado pelo autor.

Na figura 3 abaixo, está sendo importado um método de regressão linear chamado de **OLS** (Ordinary Least Squares) para a construção do modelo. Esse método procura minimizar a soma dos erros quadrados de forma a construir uma reta que melhor explique a relação existente entre a variável dependente e as variáveis independentes. Esse tipo de problema faz muito mais sentido como investigação, porque se procura obter essa relação, de forma que, em alguns casos, não se sabe se ela existe de fato ou é especulativa.

```

> MI
import statsmodels.formula.api as sm

> MI
model = sm.ols(formula='cnt ~ season + yr + mnth + holiday + weekday + workingday + weathersit + temp + atemp + hum + windspeed', data=df).fit()

```

Figura 3 - Criando um modelo de regressão linear com OLS.

Fonte: elaborado pelo autor.

Uma das funcionalidades do OLS é que é possível definir em quais variáveis você imagina que exista a relação. Para isso, você define o parâmetro **'formula'** e, como argumento, você coloca **'y ~ x1 + x2 + ... xn'**, onde **y** é a variável dependente que pode ter seus valores linearmente relacionados com as demais variáveis **x1 + x2 + ... xn**. O argumento **data** é de onde provém os dados dentro do DataFrame.

Outra forma de criar um modelo de regressão linear é utilizando a biblioteca **sk-learn** e o objeto **LinearRegression** (MCKINNEY, 2018). No exemplo da figura 4 abaixo, estou separando os dados independentes e os dependentes entre **X** e **y**, já que nesse regressor as informações vão separadas. Para evitar um overfitting e uma interpretação incorreta, estou removendo as colunas **'dteday'**, pois o regressor não entende data nesse formato. Também removi **'instant'**, por ser um mero contador de registros. Por fim, eliminei **'casual'** e **'registered'**, porque são subtotais do valor final **'cnt'**. Mas claro que você poderia ter optado por buscar um modelo de regressão linear apenas para as bicicletas alugadas de forma **'casual'**.

```

> MI
from sklearn.linear_model import LinearRegression
cols = df.columns
X = df[cols[:-1]]
y = df[cols[-1:]]
X.drop(columns=['instant', 'dteday', 'casual', 'registered'], inplace=True)
model2 = LinearRegression()
model2.fit(X,y)

```

Figura 4 - Criando um modelo de regressão linear com a sk-learn.

Fonte: elaborado pelo autor.

Reflita

A regressão linear é um recurso para se criar um modelo que reflita a ligação existente entre o que se quer prever e as demais variáveis. Supondo que você tenha os dados distribuídos no formato de uma parábola, a regressão linear ainda poderia ser um bom estimador do valor da variável dependente?

Assista

Accesse na plataforma o vídeo: Regressão linear

Importante: para compreender melhor este vídeo, você deverá acessar os arquivos **hour.csv**, **hour_cat.csv** e **Jupyter_notebook_un_VIDEOS.ipynb** disponíveis na plataforma.

2 MÉTRICAS DE REGRESSÃO

Uma vez criado o modelo, você já deve estar curioso sobre como efetivamente saberá se ele é bom ou não, ou seja, quais as medidas que podem te levar a crer que o modelo resultante da regressão linear é efetivo. Essa pergunta sempre tem que estar na sua mente como cientista de dados ou como um arquiteto de outros modelos de Inteligência Artificial (FACELI *et al.*, 2011).

A partir do modelo criado pelo OLS, você pode executar o método **summary()**, que retorna muitas informações sobre o modelo. Veja na figura 5, mais adiante, que há o resultado do modelo que acabou de ser criado sobre os dados do aluguel de bicicletas. Nele, foram utilizadas praticamente todas as variáveis disponíveis para estabelecer uma relação do número de bicicletas emprestadas diariamente.

A primeira informação é o **'R-squared'**, que você pode interpretar como sendo a exatidão do modelo. Ele é um número adimensional, então você não pode lê-lo como sendo um percentual ou algo semelhante. Em relação aos resultados, quanto mais próximo de 1.0 for o valor de **'R-squared'**, significa que melhor o modelo se ajusta aos dados informados. No caso, são os dados de treinamento ainda. Então, vale uma validação depois usando cross validation para avaliar se não houver overfitting (GÉRON, 2019).

Seguindo a análise, na coluna **P > |t|** há os chamados de p-values. Pela análise desses valores, consegue-se determinar a relevância das variáveis na criação do modelo de regressão. Valores de **pvalue > 0.05** significam que a variável não tem relevância para a construção do modelo. E você pode notar isso suprimindo a variável da **'formula'** e refazendo o modelo, de forma que **R-squared** pouco sofrerá na alteração de valor. Para as demais variáveis com **pvalue < 0.05**, quanto mais próximo de 0 maior a sua relevância.

Aprofunde-se

Uma das medidas mais utilizadas para avaliar a performance de um regressor é R-squared ou R^2 . É uma medida importante derivada diretamente da obtenção do modelo em relação aos dados. Confira o vídeo para descobrir como! Ele está em inglês, mas com geração automática de legenda em português.

Título: Medida R^2 (R-squared) explicada

Acesso em: 15/08/2020.

Disponível em: <https://youtu.be/2AQKmw14mHM>

O valor do **'intercept'** é a constante linear da equação e pode ser considerado como o valor base relativo à variável dependente. O valor do **'intercept'** é onde a equação corta o eixo y quando $x = 0$. Ou seja, no exemplo, é possível dizer que são alugadas diariamente, ao menos, 1469 bicicletas. O que vem adicional a esse valor é influência das demais variáveis do modelo.

Os valores dos coeficientes **'coef'** também informam a variação das quantidades de bicicletas alugadas diariamente. Por exemplo, a variável **'holyday'** assume somente os valores 0 e 1, significando dias úteis e feriados, respectivamente. Note que a variação em 1 nessa variável influencia em -519 bicicletas alugadas, ou seja, nos feriados se alugam menos 519 bicicletas em média naquela data. Na variável **'temp'** há um efeito positivo, de forma que a variação de 1 ponto na temperatura reflete em mais 2029 bicicletas alugadas.

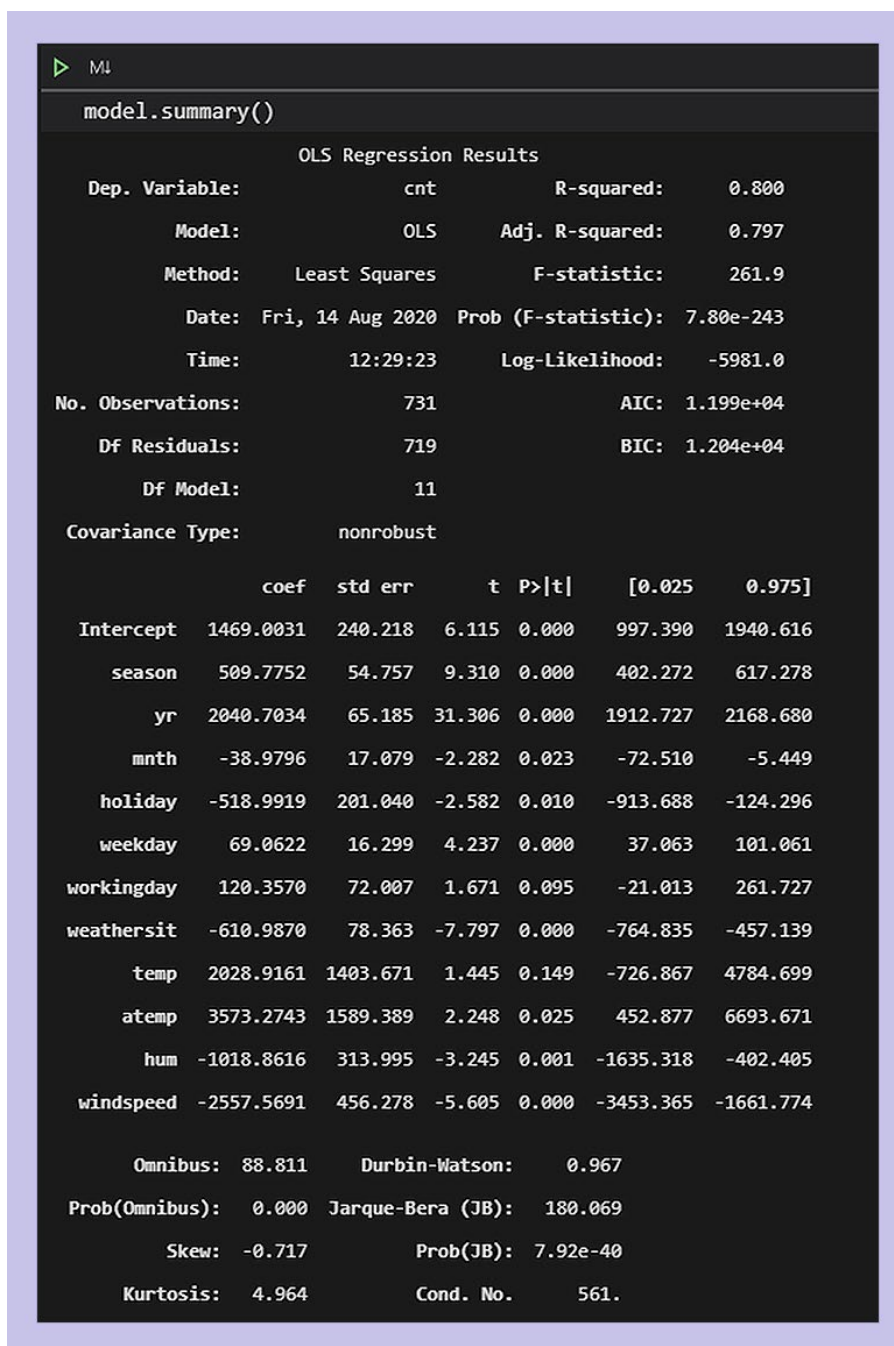


Figura 5 - Métricas de regressão.
 Fonte: elaborado pelo autor.

Assista

Acesse na plataforma o vídeo: Métricas em regressão linear

Importante: para compreender melhor este vídeo, você deverá acessar os arquivos

`Jupyter_notebook_un_VIDEOS.ipynb`, `hour.csv` e `hour_cat.csv` disponíveis na plataforma.

3 OUTROS TIPOS DE REGRESSORES

Existem outros tipos de regressores disponíveis ao uso. Para conjuntos de dados com variáveis categóricas, pode-se utilizar a regressão logística. Ela utiliza outro algoritmo de ajuste que leva em consideração a função **log()** sobre os atributos, por isso leva esse nome (CHARNET *et al.*, 2008).

Aprofunde-se

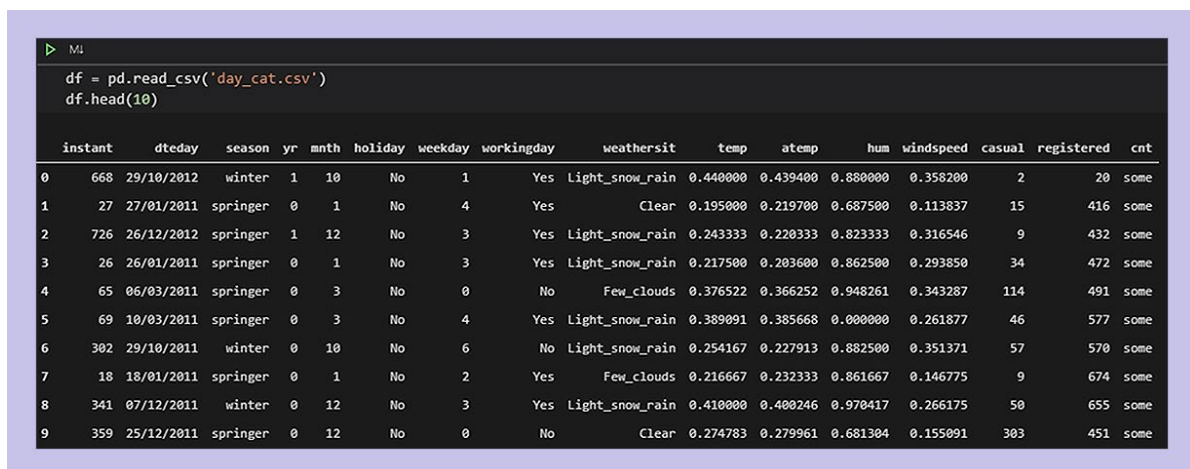
A regressão logística é uma técnica estatística tradicional que também é muito popular como uma ferramenta de aprendizado de máquina. Neste StatQuest, as principais ideias são examinadas para que você possa entender o que é e como ela é usada. Confira o vídeo para descobrir como! Ele está em inglês, mas com geração automática de legenda em português.

Título: Entendendo a regressão logística

Acesso em: 15/08/2020.

Disponível em: <https://youtu.be/yYKR4sgzI8>

Para exemplificar o uso da regressão logística, eu reformulei o conjunto de dados do aluguel de bicicletas para conter dados categóricos. Veja na figura 6 logo abaixo como ficou. As informações das estações do ano em **'season'** eram numéricas (índices somente) e agora são as próprias estações. Os atributos **'holiday'** e **'workingday'** assumiam apenas valores de 0 ou 1 para indicar o tipo do dia. O atributo **'weathersit'** continha a situação do tempo e era um índice para os valores agora apresentados. E, finalmente, a variável dependente **'cnt'** foi segmentada em quatro grupos de valores de aluguel de bicicletas por dia: 0 a 2000, **'some'**; 2000 a 4000, **'good'**; 4000 a 6000, **'many'**; e 6000 a +8000, **'alot'**.



	instant	dteday	season	yr	mnth	holiday	weekday	workingday	weathersit	temp	atemp	hum	windspeed	casual	registered	cnt
0	668	29/10/2012	winter	1	10	No	1	Yes	Light_snow_rain	0.440000	0.439400	0.880000	0.358200	2	20	some
1	27	27/01/2011	springer	0	1	No	4	Yes	Clear	0.195000	0.219700	0.687500	0.113837	15	416	some
2	726	26/12/2012	springer	1	12	No	3	Yes	Light_snow_rain	0.243333	0.220333	0.823333	0.316546	9	432	some
3	26	26/01/2011	springer	0	1	No	3	Yes	Light_snow_rain	0.217500	0.203600	0.862500	0.293850	34	472	some
4	65	06/03/2011	springer	0	3	No	0	No	Few_clouds	0.376522	0.366252	0.948261	0.343287	114	491	some
5	69	10/03/2011	springer	0	3	No	4	Yes	Light_snow_rain	0.389091	0.385668	0.000000	0.261877	46	577	some
6	302	29/10/2011	winter	0	10	No	6	No	Light_snow_rain	0.254167	0.227913	0.882500	0.351371	57	570	some
7	18	18/01/2011	springer	0	1	No	2	Yes	Few_clouds	0.216667	0.232333	0.861667	0.146775	9	674	some
8	341	07/12/2011	winter	0	12	No	3	Yes	Light_snow_rain	0.410000	0.400246	0.970417	0.266175	50	655	some
9	359	25/12/2011	springer	0	12	No	0	No	Clear	0.274783	0.279961	0.681304	0.155091	303	451	some

Figura 6 - Conjunto de dados com valores categóricos.
Fonte: elaborado pelo autor.

A regressão logística suporta somente valores inteiros ou numéricos e, por isso, é necessário transformar os valores categóricos em numéricos. Então, no código Python exibido na figura 7, os atributos nomeados na lista **'cols'** são identificados no DataFrame como categóricos e depois é utilizado o método **cat.codes** para redefinir os valores internos do DataFrame.

```

> M4

cols = ['season', 'holiday', 'workingday', 'weathersit', 'cnt']
for col in cols:
    unval = list(df[col].unique())
    df[col] = df[col].astype(pd.api.types.CategoricalDtype(categories=unval))
    df[col] = df[col].cat.codes

df.head(10)

```

Figura 7 - Redefinindo os atributos categóricos.
Fonte: elaborado pelo autor.

Veja na figura 8 como fica o DataFrame redefinido com os atributos contendo categorias numéricas.

	instant	dteday	season	yr	mnth	holiday	weekday	workingday	weathersit	temp	atemp	hum	windspeed	casual	registered	cnt
0	668	29/10/2012	0	1	10	0	1	0	0	0.440000	0.439400	0.880000	0.358200	2	20	0
1	27	27/01/2011	1	0	1	0	4	0	1	0.195000	0.219700	0.687500	0.113837	15	416	0
2	726	26/12/2012	1	1	12	0	3	0	0	0.243333	0.220333	0.823333	0.316546	9	432	0
3	26	26/01/2011	1	0	1	0	3	0	0	0.217500	0.203600	0.862500	0.293850	34	472	0
4	65	06/03/2011	1	0	3	0	0	1	2	0.376522	0.366252	0.948261	0.343287	114	491	0
5	69	10/03/2011	1	0	3	0	4	0	0	0.389091	0.385668	0.000000	0.261877	46	577	0
6	302	29/10/2011	0	0	10	0	6	1	0	0.254167	0.227913	0.882500	0.351371	57	570	0
7	18	18/01/2011	1	0	1	0	2	0	2	0.216667	0.232333	0.861667	0.146775	9	674	0
8	341	07/12/2011	0	0	12	0	3	0	0	0.410000	0.400246	0.970417	0.266175	50	655	0
9	359	25/12/2011	1	0	12	0	0	1	1	0.274783	0.279961	0.681304	0.155091	303	451	0

Figura 8 - DataFrame redefinido.
Fonte: elaborado pelo autor.

Para utilizar a regressão logística em Python, você deve importar o objeto **LogisticRegression** da biblioteca **sk-learn** (GÉRON, 2019). Também precisará ter os dados divididos em DataFrames separados para as variáveis independentes e a variável dependente, como na figura 9 abaixo.

Note que, mais uma vez, estou removendo as colunas que não fazem sentido serem utilizadas no modelo de regressão, como já explicado no tópico anterior. Ao criar o classificador **'clf'**, já aproveito para aferir a acurácia.

Sim! Para esse objeto temos a acurácia. Você, inclusive, poderá encontrar um objeto parecido com esse para executar a regressão linear tradicional, embora o OLS seja mais rico em informações (MCKINNEY, 2018).



```

> M1
from sklearn.linear_model import LogisticRegression
cols = df.columns
X = df[cols[:-1]]
y = df[cols[-1:]]
X.drop(columns=['instant', 'dteday', 'casual', 'registered'], inplace=True)
clf = LogisticRegression(random_state=0).fit(X, y)
print("Acurácia: ",clf.score(X,y)*100)

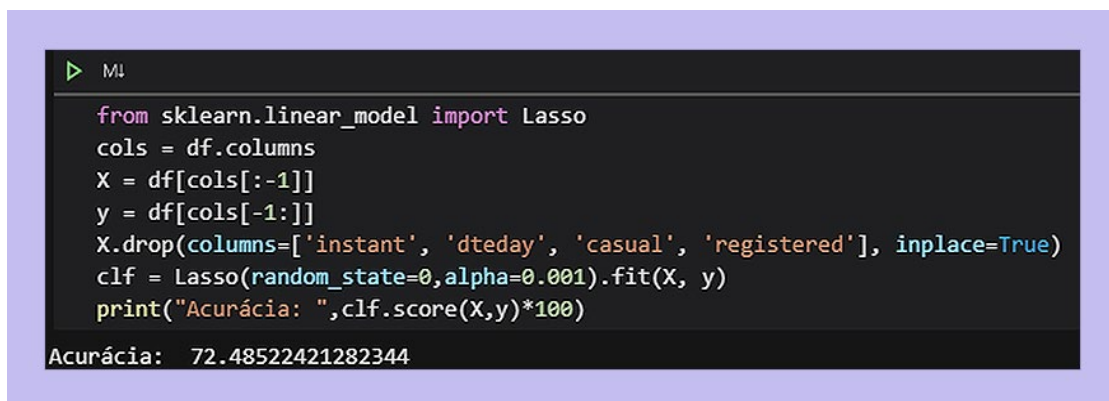
Acurácia: 75.78659370725033

```

Figura 9 - Regressão logística.
Fonte: elaborado pelo autor.

Outra forma de fazer a regressão linear é utilizando a regressão Lasso. Ela implementa um componente de regularização na equação linear, de forma que a função possa ficar mais bem ajustada aos dados, inclusive fazendo a eliminação de variáveis correlacionadas para poder simplificar o modelo.

A forma de construção do classificador é muito parecida com a anterior, exceto que existe um parâmetro chamado de **'alpha'**, em que você controla o nível de regularização, como exemplificado na figura 10.



```

> M1
from sklearn.linear_model import Lasso
cols = df.columns
X = df[cols[:-1]]
y = df[cols[-1:]]
X.drop(columns=['instant', 'dteday', 'casual', 'registered'], inplace=True)
clf = Lasso(random_state=0,alpha=0.001).fit(X, y)
print("Acurácia: ",clf.score(X,y)*100)

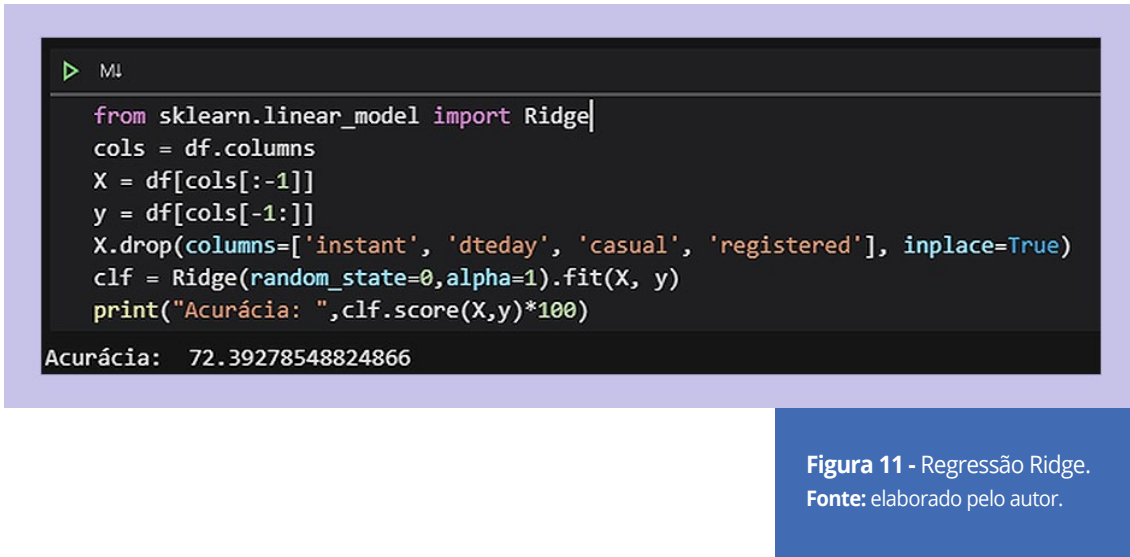
Acurácia: 72.48522421282344

```

Figura 10 - Regressão Lasso.
Fonte: elaborado pelo autor.

Por fim, mais um método bastante utilizado para a regressão linear é a regressão Ridge (GÉRON, 2019). Ela é um tipo de regressão que implementa um componente de regularização na equação linear, de forma que a função possa ficar mais bem ajustada aos dados.

A forma de construção do classificador é muito parecida com a anterior, embora não faça a eliminação das variáveis correlacionadas. Também existe um parâmetro chamado '**alpha**', em que você controla o nível de regularização, como exemplificado na figura 11.



Assista

Acesse na plataforma o vídeo: Outras formas de regressão

Importante: para compreender melhor este vídeo, você deverá acessar os arquivos **hour.csv**, **hour_cat.csv** e **Jupyter_notebook_un_VIDEOS.ipynb** disponíveis na plataforma.

4 ENSEMBLE

Agora que você já viu vários tipos de algoritmos para a classificação, como o **KNN**, as árvores de decisão, a regressão linear e a regressão logística, é hora de entender como funciona um ensemble. Ele é uma técnica para reunir um conjunto de classificadores com o objetivo de melhorar o poder de predição em termos de acurácia e precisão (FACELI *et al.*, 2011).

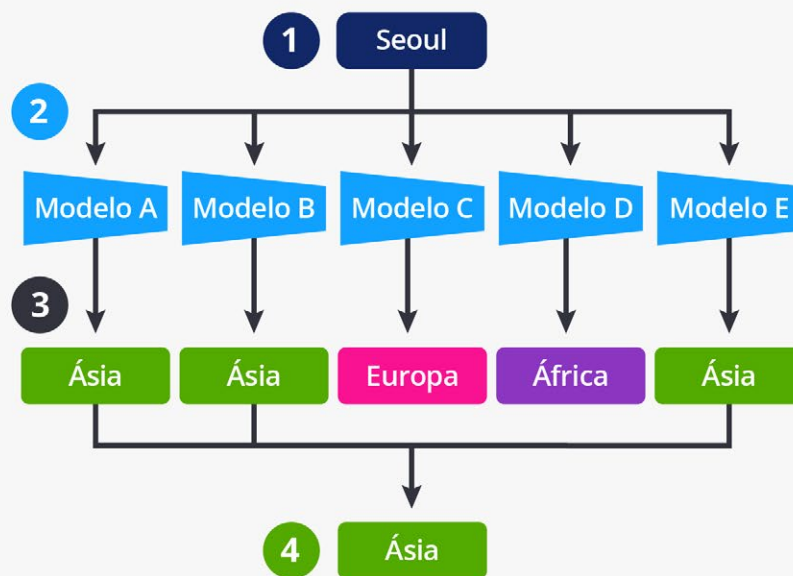
Um ensemble pode ser constituído de classificadores provenientes do mesmo indutor (**RandomForest** e **GradientBoosting**), em que, nesse caso, se constroem diversas árvores de decisão de profundidades e especialidades diferentes com vistas a cumprir o papel antes informado: melhorar a classificação (MCKINNEY, 2018).

Também é possível utilizar outras técnicas, como o Voting Classifier. Nesse tipo de ensemble, vários classificadores literalmente votam a qual classe o exemplo pertence e a classe com maior número de votos passa a ser a classe predita (FACELI *et al.*, 2011; GÉRON, 2019).

Como um ensemble de classificadores “vota” na escolha da classe de um exemplo

As ferramentas de Ciência de dados, no que tange aos algoritmos, critérios de avaliação e setup dos hiperparâmetros, resultam em inúmeras possibilidades de escolha. **Entender como esse fluxo funciona na busca dos melhores resultados faz de você um cientista de dados.**

Simple-voting-based ensemble



- 1** Considere a existência de uma nova informação, que é uma cidade em que não se sabe a qual continente pertence.
- 2** Os modelos de classificação (A até E), de acordo com seus modelos e dados de treinamento, **irão produzir com predição as informações a qual continente a cidade supostamente pertence.**
- 3** Cada um dos classificadores, ao prever a qual continente a cidade pertence, a saída de cada um dos classificadores é considerada um voto, **já que se tem potencialmente valores distintos de saída ou diferentes votos.**
- 4** Nesse caso, a técnica é de voto por maioria, em que a saída com maior representatividade passa a ser o valor predito. Como há três classificadores predizendo (votando) que a cidade pertence à Ásia, então essa passa a ser a classe final predita.

ED+ Content Hub © 2020

Infográfico 1 - Ensemble de classificadores.

Fonte: adaptado de <https://bit.ly/3kZe6A7> Acesso em: 31 ago. 2020.

Conceitualmente, para pronto uso, a ideia de ensemble é bastante simples. Assim como tenho feito em outros tópicos, a ideia é apresentar o conceito e o seu uso em Python. Não estou me preocupando em tratamento de dados ausentes, split, validação etc. Quero somente apresentar a você uma sintaxe mais limpa de outros conceitos.

Aprofunde-se

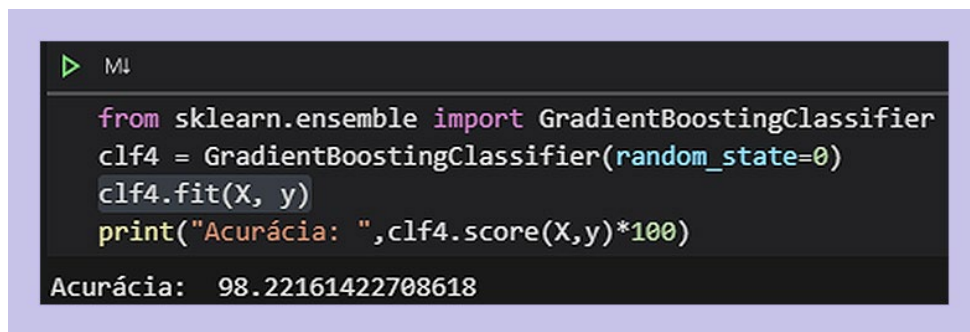
Às vezes, entender como funciona um ensemble e o esquema de votação, bem como as razões e as vantagens de seu uso, nem sempre é simples. Neste vídeo, tudo é colocado de forma simples e objetiva. Confira para descobrir como! O vídeo está em inglês, mas com geração automática de legenda em português.

Título: Ensembles para classificação

Acesso em: 15/08/2020.

Disponível em: <https://youtu.be/Un9zObFjBH0>

A figura 12 é uma forma de criar um objeto para o ensemble **GradientBoostingClassifier**. Tenho mantido o parâmetro **random_state=0**, de forma que você possa replicar os experimentos e obter o mesmo resultado. Também veja que a acurácia aqui tem o viés de estar sendo calculada utilizando o classificador sobre os próprios dados de treinamento. Não se pode afirmar nada referente à acurácia sobre os exemplos de teste e validação. **Assista aos vídeos!**



```

▶ M1
from sklearn.ensemble import GradientBoostingClassifier
clf4 = GradientBoostingClassifier(random_state=0)
clf4.fit(X, y)
print("Acurácia: ", clf4.score(X, y) * 100)

Acurácia: 98.22161422708618

```

Figura 12 - Ensemble – Gradient Boosting.

Fonte: elaborado pelo autor.

Já na figura 13, há uma forma de se criar um objeto para o ensemble **RandomForestClassifier**. Interessante notar que no exemplo utilizado a acurácia é de 100% e isso somente nos diz que, nesse caso, o classificador consegue prever todos os exemplos utilizados para induzir o classificador. Se o mesmo classificador for utilizado para exemplos desconhecidos, e obtiver o mesmo desempenho, excelente! Senão, pode ser um exemplo clássico de overfitting.

```

> ML

from sklearn.ensemble import RandomForestClassifier
clf5 = RandomForestClassifier(random_state=0)
clf5.fit(X, y)
print("Acurácia: ",clf5.score(X,y)*100)

Acurácia: 100.0

```

Figura 13 - Ensemble – Random Forest.
Fonte: elaborado pelo autor.

Os dois exemplos a seguir, ilustrados pelas figuras 14 e 15, são a respeito do ensemble por votação. Você vai aprender sobre as duas possibilidades.

A primeira possibilidade, representada na figura 14, é a criação do objeto **VotingClassifier** com os principais parâmetros. Você precisa informar uma lista de tuplas em **'estimators'** com um apelido para o classificador e o classificador propriamente criado. Estou utilizando os classificadores já criados e exemplificados anteriormente. Com o parâmetro **'voting=hard'**, significa que a votação será por maioria simples. Ou seja, estão sendo utilizados três classificadores (regressão logística, Gradient Boosting e Random Forest) e a classe que receber dois votos será a classe predita. Se você utilizar um número par de classificadores, o estimador irá selecionar como critério de desempate a primeira classe votada.

```

> ML

from sklearn.ensemble import VotingClassifier
eclf = VotingClassifier(estimators=[('LogReg', clf1), \
    ('GBoost', clf4), ('RForest', clf5)], voting='hard')
eclf.fit(X, y)
print("Acurácia: ",eclf.score(X,y)*100)

Acurácia: 98.49521203830369

```

Figura 14 - Ensemble – Voting Hard.
Fonte: elaborado pelo autor.

Já na figura 15, há o mesmo objeto de votação, que é o **VotingClassifier**. Entretanto, nesse caso, está sendo utilizado o argumento **'voting=soft'**. Nesse esquema de votação, você pode atribuir pesos aos classificadores ou omitir esses pesos. Os pesos são atribuídos à acurácia de cada classificador. No exemplo, a regressão logística tem peso 5 e os demais classificadores peso 1. Dessa forma, o voto da regressão logística vale cinco vezes mais do que o voto dos demais classificadores.

```

> ML

eclfs = VotingClassifier(estimators=[('LogReg', clf1), ('GBoost', clf4), \
    ('RForest', clf5)], voting='soft', weights=[5, 1, 1])
eclfs.fit(X, y)
for clf, label in zip([clf1, clf4, clf5, eclfs], ['Logistic Regression', \
    'Gradient Boosting', 'Random Forest', 'Ensemble']):
    print("Acurácia: %0.2f%% \t [%s]" % (clf.score(X,y)*100, label))

Acurácia: 75.79%      [Logistic Regression]
Acurácia: 98.22%      [Gradient Boosting]
Acurácia: 100.00%     [Random Forest]
Acurácia: 89.60%      [Ensemble]

```

Figura 15 - Ensemble – Voting Soft.
Fonte: elaborado pelo autor.

Analisando a acurácia de cada classificador e do estimador, utilizando **'voting=soft'** e com os pesos mencionados, o ensemble teve uma performance média pior que o Gradient Boosting e o Random Forest, claramente influenciada pela performance menor e de mais peso advinda da regressão logística.

Refleta

Se tiver um conjunto de classificadores em que você não saiba estipular os pesos de cada um para criar um ensemble do tipo **VotingClassifier 'soft'**, como seria possível utilizar a otimização de hiperparâmetros com **GridSearch** para se obter os melhores pesos?

Importante frisar que os dados são para esse exemplo e cenário. Não se pode afirmar nada sobre um classificador ou ensemble ser melhor que o outro apenas com os resultados obtidos. Os critérios para se afirmar que um classificador é melhor que o outro, em geral, envolvem muitos experimentos com conjuntos de dados diferentes e variação nos hiperparâmetros.

Assista

Acesse na plataforma o vídeo: Regressão: ensemble

Importante: para compreender melhor este vídeo, você deverá acessar os arquivos **Jupyter_notebook_un_VIDEOS.ipynb**, **hour.csv** e **hour_cat.csv** disponíveis na plataforma.

CONSIDERAÇÕES FINAIS

Nesta unidade, você aprendeu conceitos importantes, como a regressão linear, as métricas de regressão e outros regressores. Também estudou como criar um ensemble de classificadores.

A sua caixa de ferramentas em machine learning e ciência de dados já está ficando abarrotada de possibilidades. Revisitar os conceitos e as práticas dessa unidade, combinando técnicas e hiperparâmetros, é fundamental para o desenvolvimento de suas competências. Aproveite os conjuntos de dados abertos, monte projetos e abasteça o seu portfólio. Bons estudos!

GLOSSÁRIO

CSV (Comma-Separated Values): é um tipo de arquivo muito disseminado, pois pode ser carregado e lido em diversos sistemas operacionais e aplicações. Leva esse nome porque os valores são separados por vírgulas.

KNN (K-Vizinhos mais Próximos): é um algoritmo de aprendizado supervisionado em que a classe é predita com base na classe dos demais exemplos da vizinhança do exemplo, cuja classe é desconhecida.

OLS (Ordinary Least Squares): é um método estatístico para a regressão linear, baseado no cálculo dos mínimos quadrados ordinários, o qual procura estimar os parâmetros desconhecidos para formar um modelo.

REFERÊNCIAS

CHARNET, Reinaldo *et al.* **Análise de Modelos de Regressão Linear com Aplicações**. 2. ed. Campinas: Editora da Unicamp, 2008. 368 p.

FACELI, Katti *et al.* **Inteligência Artificial - Uma Abordagem de Aprendizado de Máquina**. 1. ed. Rio de Janeiro: LTC, 2011. 394 p.

GÉRON, Aurélien. **Mãos à Obra: Aprendizado de Máquina com Scikit-Learn & TensorFlow**. 1. ed. Rio de Janeiro: Alta Books, 2019. 576 p.

MCKINNEY, Wes. **Python para Análise de Dados: Tratamento de Dados com Pandas, NumPy e IPython**. 1. ed. São Paulo: Novatec, 2018. 616 p.

Unidade 8

Outras aplicações de machine learning

Objetivo de aprendizagem:

Conhecer outras aplicações de machine learning, quais métodos podem ser utilizados nessas aplicações e trechos de código para solucioná-las.

Tópicos de estudo:

- Métodos de agrupamento;
- Métodos de associação;
- Métodos de aprendizado por reforço;
- Métodos para dados não estruturados.

Iniciando os estudos:

Lendo a introdução dessa unidade, tenho certeza que já se passaram muitas horas de estudo. Espero que a jornada esteja lhe acrescentando muitas novas competências. Os assuntos de Inteligência Artificial, machine learning e Data Science são muito vastos. Seguramente, você poderia escolher algum tópico dentre eles e estudar por uma vida sem ainda esgotá-los. Nessa unidade, você vai estudar métodos de aprendizado um tanto quanto pouco explorados ainda, mas que têm muita relevância no cenário da Ciência de dados.

Vamos estudar quatro tópicos especiais relativos aos métodos de agrupamento, também conhecidos como clusterização. Outro tópico é sobre o aprendizado por associação, o qual foi uma das primeiras aplicações reais de aprendizado de máquina. Em seguida, vamos retomar um assunto que foi apenas citado anteriormente, mas que agora iremos estudar mais a fundo fazendo um exemplo, o aprendizado por reforço. E, por último, mas não menos importante, o aprendizado com dados não estruturados (imagem, texto e áudio). Lembre-se de acessar os arquivos Jupyter Notebook em sua plataforma para compreender melhor os vídeos dessa unidade.

Bons estudos!

1 MÉTODOS DE AGRUPAMENTO

Os métodos de aprendizado que temos visto – como árvore de decisão, KNN, regressão logística, Random Forest, entre outros – são todos da família do **aprendizado de máquina supervisionado**. Recapitulando, portanto, nos algoritmos mencionados, temos essa finalidade de se estabelecer uma relação entre a variável classe chamada variável dependente e as demais variáveis do conjunto de dados chamadas **variáveis independentes**.

Entretanto, quando os dados não são previamente classificados, podemos lançar mão de uma outra família de algoritmos chamados de aprendizado de máquina não supervisionado. Métodos de aprendizado não supervisionado desconsideram a necessidade de os dados estarem rotulados, de forma que exista uma relação previamente estabelecida entre uma variável dependente e outras independentes. Mesmo que o conjunto de dados tenha essa característica, ainda é possível aplicar um algoritmo de aprendizado de máquina não supervisionado com o objetivo de explorar outras características dos dados (AGGARWAL, 2013).

Uma forma de aprendizado não supervisionado é o **agrupamento**. Entretanto, será mais comum você encontrar esses algoritmos como **clusterização** ou simplesmente **clustering**. Os algoritmos de clusterização agrupam os dados do conjunto de dados em grupos cujas características os tornam semelhantes. Os grupos não têm rótulos previamente, embora o especialista do domínio possa identificar que determinados grupos possam ser um novo tipo de rótulo ou até mesmo pertencer a classes já conhecidas. O importante a se frisar é que os algoritmos de clusterização apenas agrupam os dados por similaridade (AGGARWAL, 2013).

Um algoritmo bastante conhecido dessa família é o **k-means**. Não confunda com o KNN que é um algoritmo de aprendizado supervisionado. Embora ambos compartilhem alguma similaridade, a verdade é que são de famílias de aprendizado distintas.

No algoritmo k-means, existe um hiperparâmetro k que é o número de cluster que se deseja agrupar os dados. Assim, se você considerar $k = 3$, o algoritmo irá agrupar os dados em três grupos utilizando uma medida de similaridade. A medida mais conhecida, nesse caso, é a distância euclidiana.

Aprofunde-se

O algoritmo de clusterização k-means é muito utilizado pela sua simplicidade de entendimento e implementação. Confira o vídeo para descobrir como.

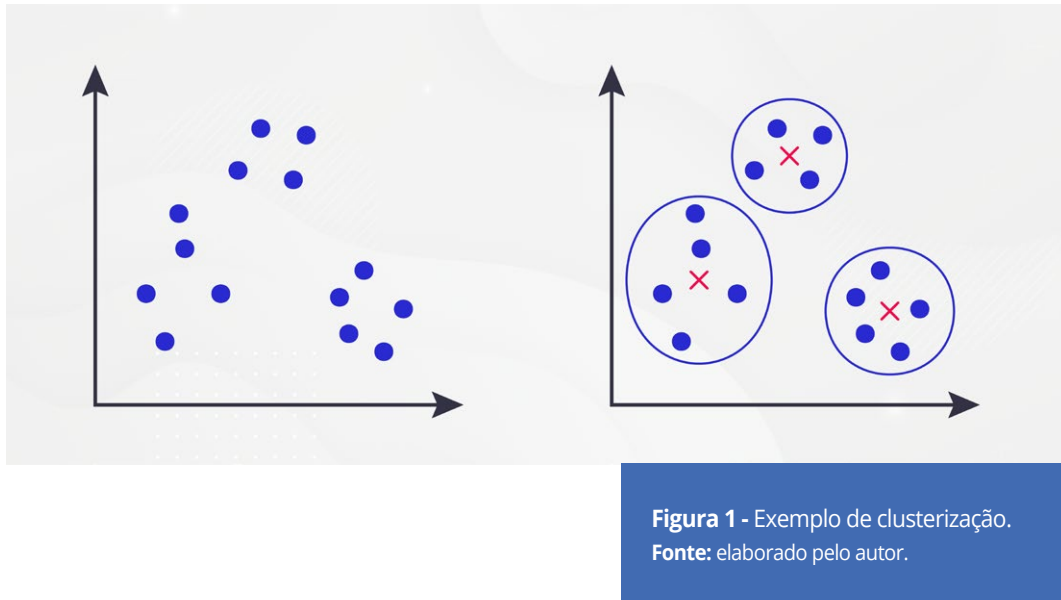
Ele está em inglês, mas com geração automática de legenda em português.

Título: K-means - Clusterização

Acesso em: 06/09/2020.

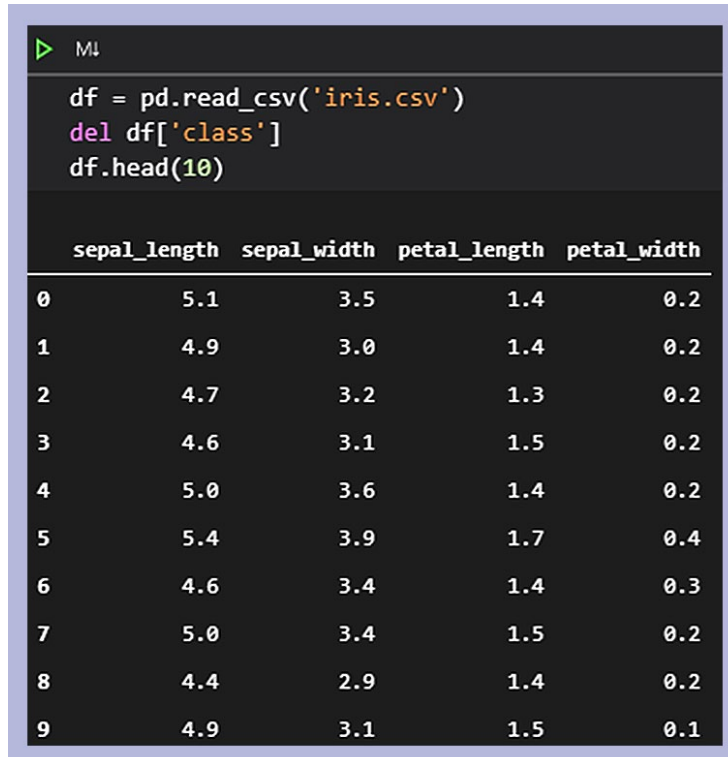
Disponível em: <https://youtu.be/4b5d3muPQmA>

Veja na figura 1 uma ilustração da clusterização considerando um conjunto de exemplos com somente duas variáveis. Note que as cores dos exemplos, o que denotaria possivelmente um rótulo, são idênticas para todos e não se alteram após o processo de agrupamento.



No exemplo foi considerado $k = 3$ e, por isso, são três clusters. O novo elemento denotado por um x vermelho ao centro dos clusters é o centróide. O centróide é um novo ponto criado a partir de uma métrica envolvendo os valores das variáveis e serve para indicar, como o nome diz, o centro do cluster. A métrica mais utilizada é a distância euclidiana.

Em **Python** já temos o privilégio de ter o k-means implementado dentro da biblioteca sk-learn. A título de exemplo, vamos utilizar o conjunto de dados **Iris**, já conhecido nosso, mas com uma alteração: iremos suprimir a coluna classe para ver a capacidade do algoritmo de agrupamento em dividir os exemplos por similaridade. Veja um exemplo do conjunto de dados na figura 2.

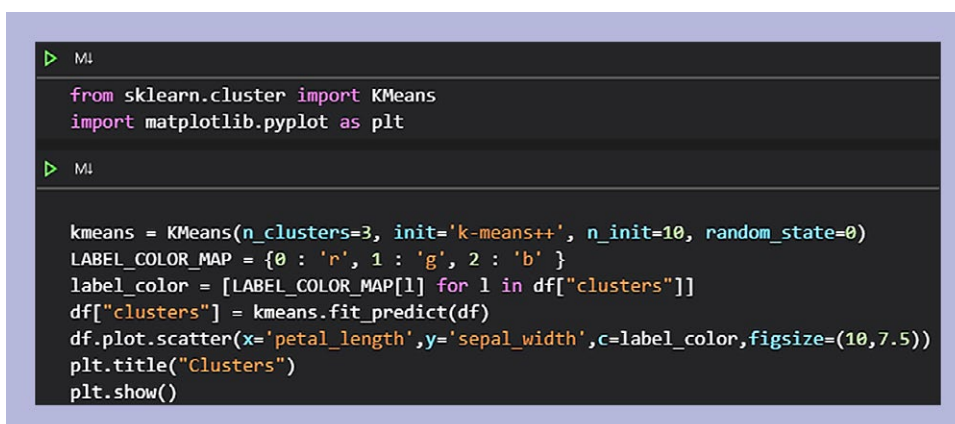


```
df = pd.read_csv('iris.csv')
del df['class']
df.head(10)
```

	sepal_length	sepal_width	petal_length	petal_width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
5	5.4	3.9	1.7	0.4
6	4.6	3.4	1.4	0.3
7	5.0	3.4	1.5	0.2
8	4.4	2.9	1.4	0.2
9	4.9	3.1	1.5	0.1

Figura 2 - Conjunto de dados Iris sem o atributo classe.
Fonte: elaborado pelo autor.

Existem vários algoritmos de clusterização e, para utilizar o k-means, você faz uma importação direta da biblioteca sk-learn (GÉRON, 2019). Acompanhe na figura 3. O algoritmo foi inicializado com **n_clusters = 3** informando que desejamos procurar por 3 clusters. Também deixamos explícito que queremos o algoritmo de clusterização baseado no **'k-means++'** que, na verdade, é o padrão. No parâmetro **'n_init'**, temos o número de vezes que o algoritmo irá executar com diferentes centroides para se obter o melhor resultado.



```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

kmeans = KMeans(n_clusters=3, init='k-means++', n_init=10, random_state=0)
LABEL_COLOR_MAP = {0 : 'r', 1 : 'g', 2 : 'b' }
label_color = [LABEL_COLOR_MAP[l] for l in df["clusters"]]
df["clusters"] = kmeans.fit_predict(df)
df.plot.scatter(x='petal_length', y='sepal_width', c=label_color, figsize=(10,7.5))
plt.title("Clusters")
plt.show()
```

Figura 3 - Código exemplo - K-means.
Fonte: Elaborado pelo autor.

As linhas que seguem na figura 3 são para criar uma nova coluna no conjunto de dados com o número do cluster obtido e associado a cada exemplo. Depois, cada valor numérico, no caso de 0, 1 ou 2, por serem três clusters, são associados a cores diferentes a fim de se produzir o resultado do gráfico 1. Foi plotado um gráfico de dispersão considerando duas variáveis apenas e as cores dos exemplos são os clusters calculados. Veja como temos uma boa separação dos exemplos.

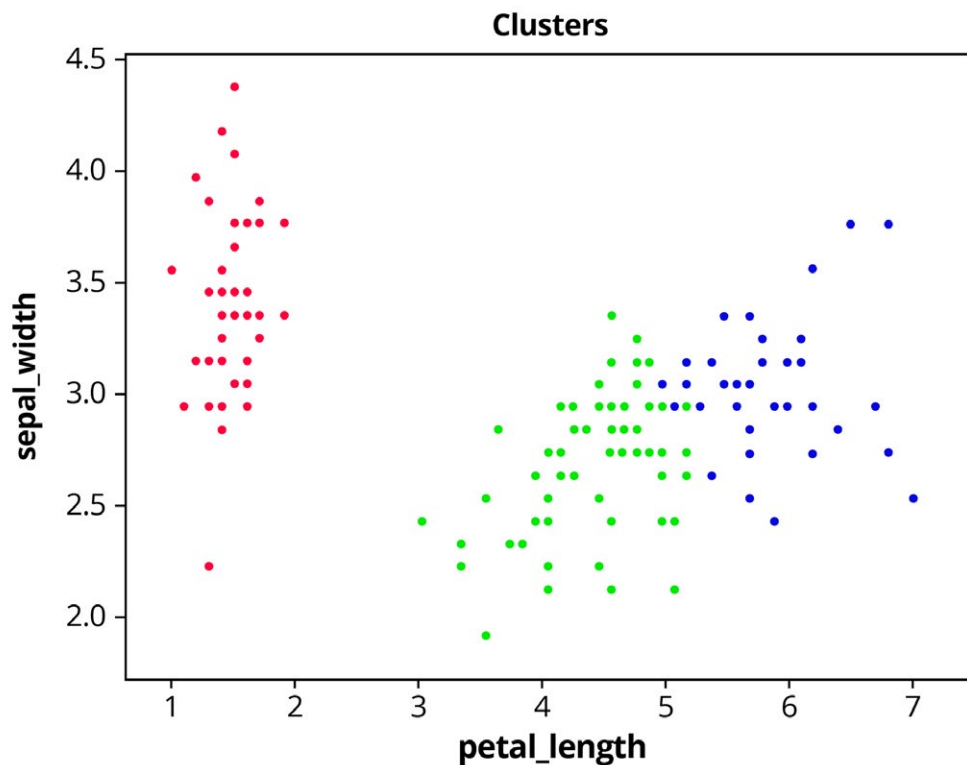


Gráfico 1 - Clusters obtidos pelo k-means.
Fonte: elaborado pelo autor.

Você pode ficar surpreso ao comparar o gráfico 1 com o (quase) mesmo gráfico do gráfico 2. Parecem idênticos, embora, com um olhar mais atento, poderá notar que existem pontos coloridos de forma diferente ao comparar as figuras. Acontece que o gráfico 1 é o resultado do k-means enquanto o gráfico 2 é o conjunto de dados original com as classes.

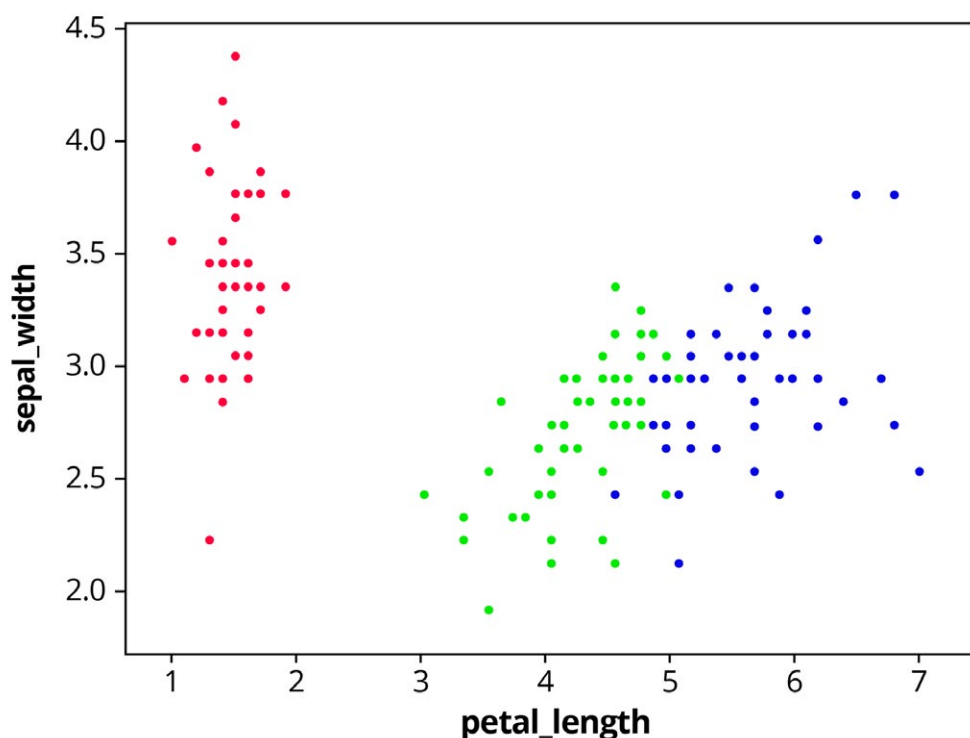


Gráfico 2 - Classes originais do exemplo Iris.
Fonte: elaborado pelo autor.

Assista

Acesse na plataforma o vídeo: Agrupamento (clusterização)

Importante: para melhor compreensão desse vídeo, você deverá acessar o arquivo **iris.csv** disponível em sua plataforma.

2 MÉTODOS DE ASSOCIAÇÃO

Os métodos de associação são conhecidos, na literatura, como sendo um dos primeiros casos de estudo envolvendo dados e computação. Na época, os termos **Big Data** e **Data Science** ainda não haviam sido cunhados. O estudo de comportamentos de compra em um supermercado, usando basicamente as informações do ticket de compras, ficou conhecido como Análise da Cesta de Supermercado, uma tradução livre para Market Basket Analysis. Nesse cenário, existe uma história envolvendo possivelmente uma rede famosa de supermercados, cujo nome ficou conhecido como o caso “cerveja-fraldas”.

Conta a história que, ao fazer o cálculo da correlação de venda de vários produtos, notou-se que às sextas-feiras a correlação entre venda de fraldas e cervejas era algo que chamava a atenção. Uma investigação mais profunda levou a descobrir que os homens passavam na sexta-feira para comprar fraldas para os filhos ainda de poucos meses de vida e, diante da perspectiva de pouca diversão no final de semana, compravam para si alguns fardos de cerveja para curtir sozinhos (AGGARWAL, 2013).

Fato é que, depois dessa história, muitos supermercados, como vemos hoje, passaram a agrupar os produtos muito mais separados pela finalidade do que pela categoria. Por isso é que temos produtos como carvão, espetos de churrasco e bebidas próximo ao açougue. Também podemos citar o corredor dos molhos prontos e massas junto com queijo ralado. Certamente você deve se lembrar de algo assim em algum supermercado que frequenta.

As **regras de associação** são normalmente escritas como **antecedente → consequente**. Não confunda com regras provenientes de algoritmos de classificação. No caso das regras de associação, podemos ter, por exemplo, {fralda}→{cerveja} ou até mesmo {fralda, leite}→{cerveja, amendoim}.

As regras de associação são calculadas com base na correlação existente entre o(s) antecedente(s) e o(s) consequente(s). Ao calcular, pode-se considerar períodos para entender a força da correlação, bem como, no caso dos produtos de supermercado, entender a correlação dos produtos por pessoa, dias, horários, períodos etc.

Ao aplicar algoritmos de associação, precisamos ter em mente três medidas relativas às regras obtidas para entendermos o que está sendo calculado e extraído dos dados: **suporte**, **confiança** e **lift** (AGGARWAL, 2013).

O **suporte** é a frequência relativa com que as regras aparecem. Em muitos casos, você pode querer buscar um alto suporte para ter certeza de que é um relacionamento útil. No entanto, pode haver casos em que um baixo suporte seja útil se você estiver tentando encontrar relacionamentos que são novidade.

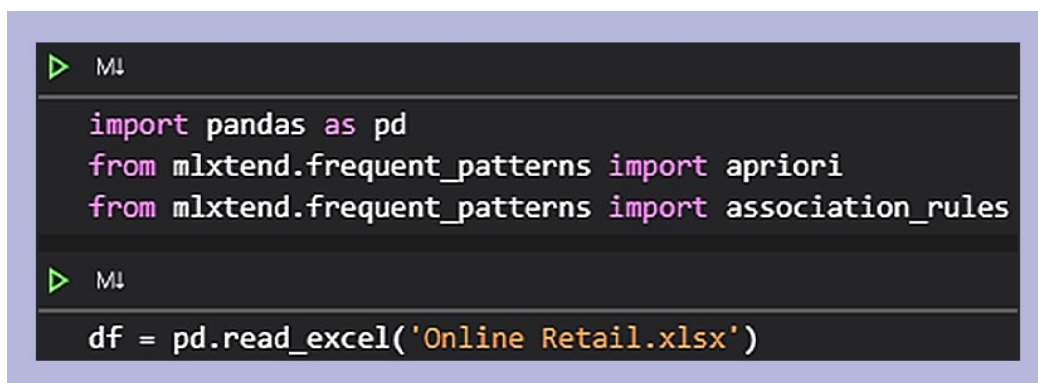
A **confiança** é uma medida da confiabilidade da regra. Por exemplo, uma confiança de 0,5 significaria que, em 50% dos casos em que fraldas foram compradas, a compra também incluiu cerveja. Para recomendação de produto, uma confiança de 50% pode ser perfeitamente aceitável, mas, em uma situação médica, esse nível pode não ser alto o suficiente.

O **lift** (aumento) é a proporção do suporte observado em relação ao esperado se as duas regras fossem independentes. A regra básica é que um valor de aumento próximo a um significa que as regras são completamente independentes. Valores de aumento maiores que um são geralmente mais “interessantes” e podem ser indicativos de um padrão de regra útil.

Para ilustrar o uso das regras de associação, vamos considerar um conjunto de dados formado dos seguintes atributos:

- **InvoiceNo:** número da compra. Se o número começar com a letra 'C' significa que a compra foi cancelada.
- **StockCode:** código do produto.
- **Description:** descrição do produto.
- **Quantity:** quantidade daquele item adquirido em cada compra.
- **InvoiceDate:** data e horário da compra.
- **UnitPrice:** preço unitário do produto.
- **CustomerID:** número do cliente (código do cliente).
- **Country:** país em que o cliente reside.

As funcionalidades para utilizar os algoritmos **apriori** e **association_rules** são fornecidas pela biblioteca **mlxtended** presentes na figura 4 (GÉRON, 2019). Provavelmente você precisará instalar ambas antes de fazer os testes. Tudo vai depender do seu ambiente de desenvolvimento. Certifique-se que ela esteja instalada antes de prosseguir. Também iremos utilizar um conjunto de dados bem representativo chamado '**Online Retail.csv**' do qual falaremos adiante.



```
▶ M!

import pandas as pd
from mlxtend.frequent_patterns import apriori
from mlxtend.frequent_patterns import association_rules

▶ M!

df = pd.read_excel('Online Retail.xlsx')
```

Figura 4 - Carregando bibliotecas **apriori e **association_rules**.**
Fonte: elaborado pelo autor.

O conjunto de dados utilizado no exemplo pode ser parcialmente visto na figura 5. É um conjunto contendo 8 atributos e 541.909 linhas. Veja que cada linha é de uma venda e possui os produtos e suas quantidades vendidas. Temos também a informação do cliente, como seu código e o país de origem da compra.

```
MI
df.head(10)
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850.0	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
5	536365	22752	SET 7 BABUSHKA NESTING BOXES	2	2010-12-01 08:26:00	7.65	17850.0	United Kingdom
6	536365	21730	GLASS STAR FROSTED T-LIGHT HOLDER	6	2010-12-01 08:26:00	4.25	17850.0	United Kingdom
7	536366	22633	HAND WARMER UNION JACK	6	2010-12-01 08:28:00	1.85	17850.0	United Kingdom
8	536366	22632	HAND WARMER RED POLKA DOT	6	2010-12-01 08:28:00	1.85	17850.0	United Kingdom
9	536367	84879	ASSORTED COLOUR BIRD ORNAMENT	32	2010-12-01 08:34:00	1.69	13047.0	United Kingdom

Figura 5 - Conjunto de dados - 'Online Retail'.

Fonte: elaborado pelo autor.

Aprofunde-se

O algoritmo apriori é um algoritmo clássico para descobrir conjuntos de itens frequentes e regras de associação relevantes. Ele é projetado para operar em um banco de dados contendo uma grande quantidade de transações – por exemplo, itens comprados pelos clientes em uma loja. Confira o vídeo para descobrir os detalhes! Ele está em inglês, mas com geração automática de legenda em português.

Título: Algoritmo apriori (aprendizagem por associação)

Acesso em: 06/09/2020.

Disponível em: https://youtu.be/WGIMIS_Ydk

O próximo passo para se obter as regras de associação é calcular as frequências de cada item do conjunto de dados. Isso pode ser feito utilizando a biblioteca apriori conforme demonstrado na figura 6.

```

> MI

# Construindo a frequência dos itens
frequent_itemsets = apriori(basket_sets, min_support=0.07, use_colnames=True)

> MI

frequent_itemsets.head()

```

	support	itemsets
0	0.071429	(4 TRADITIONAL SPINNING TOPS)
1	0.096939	(ALARM CLOCK BAKELIKE GREEN)
2	0.102041	(ALARM CLOCK BAKELIKE PINK)
3	0.094388	(ALARM CLOCK BAKELIKE RED)
4	0.081633	(BAKING SET 9 PIECE RETROSPOT)

Figura 6 - Cálculo das frequências relativas.

Fonte: elaborado pelo autor.

Uma vez calculadas, as frequências servem de entrada para a obtenção das regras de associação. Veja na figura 7 um extrato das regras de associação obtidas. Note as medidas associadas ao antecedente e ao consequente de cada regra.

```

> MI

# Criando as regras de associação
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1)
rules.head(10)

```

	antecedents	consequents	antecedent support	consequent support	support	confidence
0	(ALARM CLOCK BAKELIKE GREEN)	(ALARM CLOCK BAKELIKE PINK)	0.096939	0.102041	0.073980	0.763
1	(ALARM CLOCK BAKELIKE PINK)	(ALARM CLOCK BAKELIKE GREEN)	0.102041	0.096939	0.073980	0.725
2	(ALARM CLOCK BAKELIKE RED)	(ALARM CLOCK BAKELIKE GREEN)	0.094388	0.096939	0.079082	0.837
3	(ALARM CLOCK BAKELIKE GREEN)	(ALARM CLOCK BAKELIKE RED)	0.096939	0.094388	0.079082	0.815
4	(ALARM CLOCK BAKELIKE RED)	(ALARM CLOCK BAKELIKE PINK)	0.094388	0.102041	0.073980	0.783
5	(ALARM CLOCK BAKELIKE PINK)	(ALARM CLOCK BAKELIKE RED)	0.102041	0.094388	0.073980	0.725
6	(DOLLY GIRL LUNCH BOX)	(SPACEBOY LUNCH BOX)	0.099490	0.125000	0.071429	0.717
7	(SPACEBOY LUNCH BOX)	(DOLLY GIRL LUNCH BOX)	0.125000	0.099490	0.071429	0.571
8	(PLASTERS IN TIN CIRCUS PARADE)	(PLASTERS IN TIN SPACEBOY)	0.168367	0.137755	0.089286	0.530
9	(PLASTERS IN TIN SPACEBOY)	(PLASTERS IN TIN CIRCUS PARADE)	0.137755	0.168367	0.089286	0.648

Figura 7 - Obtenção das regras de associação

Fonte: elaborado pelo autor.

O resultado das regras que foi atribuído à **'rules'** é um DataFrame. Então você pode filtrar as informações, agrupar, plotar, tudo o mais que é possível fazer com um DataFrame e as funcionalidades do Pandas. Por exemplo, você pode simplesmente querer entender as melhores regras, ou seja, as que têm o suporte e a confiança acima de algum limiar. Por exemplo: **rules[(rules['support'] >= 0.1) & (rules['confidence'] >= 0.8)]**. Isso vai lhe retornar as regras de associação presentes na figura 8. Pode parecer meio óbvio, mas temos a prova, por meio de regras de associação, que mostram a existência de uma alta correlação entre a venda de guardanapos e pratos descartáveis de um tipo peculiar, no caso: retrô, vermelhos e com bolinhas brancas.

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	leverage	conviction
17	(SET/6 RED SPOTTY PAPER PLATES)	(SET/20 RED RETROSPOT PAPER NAPKINS)	0.127551	0.132653	0.102041	0.800000	6.030769	0.085121	4.336735
18	(SET/6 RED SPOTTY PAPER PLATES)	(SET/6 RED SPOTTY PAPER CUPS)	0.127551	0.137755	0.122449	0.960000	6.968889	0.104878	21.556122
19	(SET/6 RED SPOTTY PAPER CUPS)	(SET/6 RED SPOTTY PAPER PLATES)	0.137755	0.127551	0.122449	0.888889	6.968889	0.104878	7.852041

Figura 8 - Melhores regras de associação.
Fonte: elaborado pelo autor.

Assista

Acesse na plataforma o vídeo: Regras de associação

Importante: para melhor compreensão desse vídeo, você deverá acessar o arquivo **Online Retail.xlsx** disponível em sua plataforma.

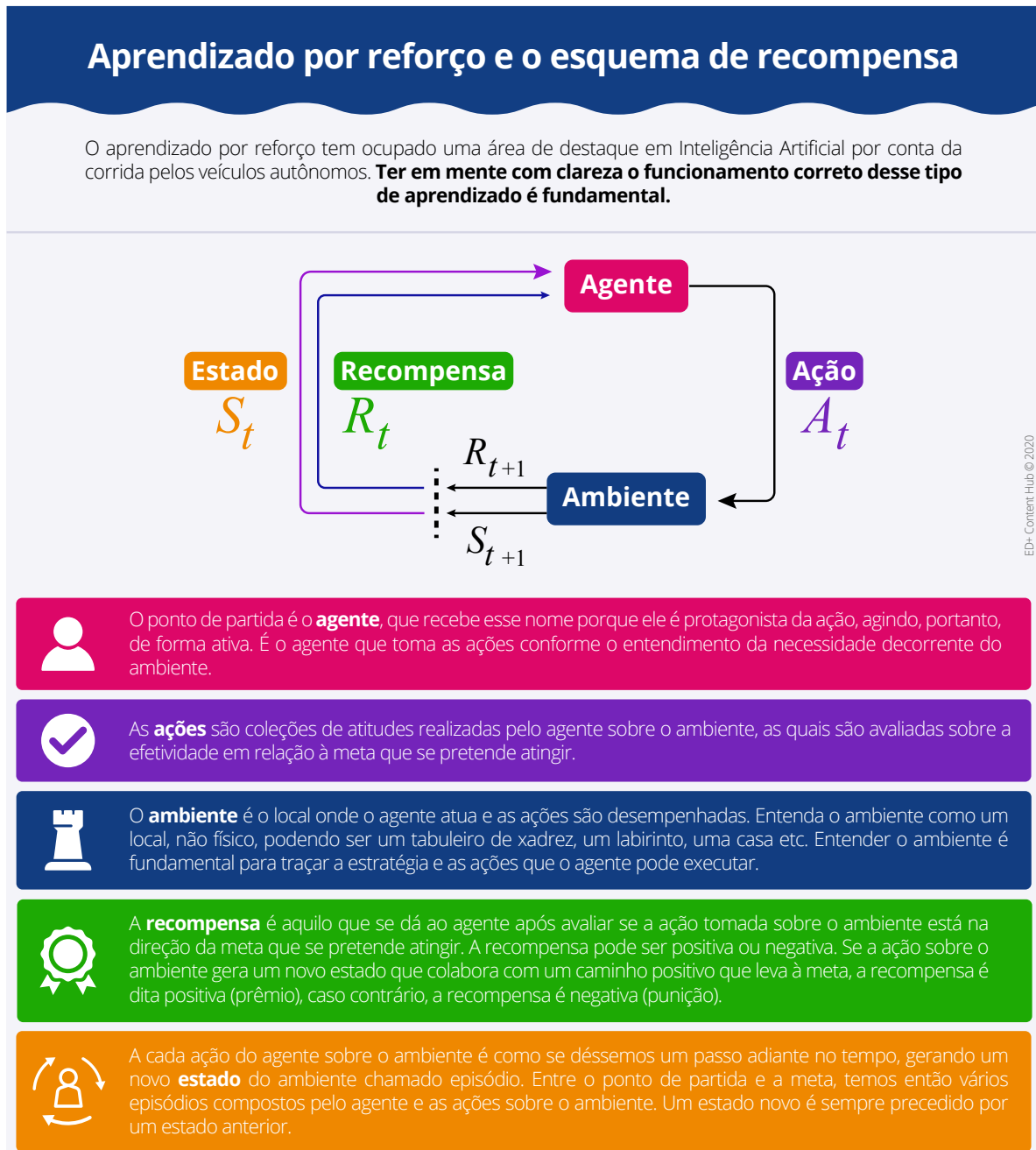
3 MÉTODOS DE APRENDIZADO POR REFORÇO

Como temos visto nos paradigmas de aprendizado supervisionado e não supervisionado, a questão envolvida sempre está relacionada a **ter um rótulo ou não** nas informações. Esse rótulo, quando existente, faz o papel do supervisor, ensinando o algoritmo que, para determinadas variáveis independentes, existe uma possível associação entre elas com possíveis valores que levam à inferência do valor da variável dependente (rótulo).

No **aprendizado por reforço**, temos um outro nível de supervisão baseado em recompensas e punições. O exemplo clássico é o treinamento de cães. Você já deve ter visto ao menos em algum vídeo uma pessoa estimulando que o cão faça determinada ação (saltar, deitar-se, correr etc.) e, quando o animal responde corretamente, a pessoa discretamente dá algo que o cão goste de comer (banana, salsicha etc.) (SUTTON, 2018).

O exemplo é pertinente porque o aprendizado por reforço é útil em sistemas e ambientes onde não se sabe ao certo o que deve ser feito ou não existe uma forma de comunicar por completo qual a ação correta. O que se faz então é dar ao agente que desempenha a ação uma recompensa ou punição, dependendo do que foi feito estar dentro ou fora do esperado, respectivamente.

No aprendizado por reforço a ideia é que exista um **agente** em um ambiente. Esse agente pode ser um robô em uma sala, mas também pode ser um programa de computador imerso em uma situação computacional. Uma vez exposto ao ambiente, o agente assume um estado inicial, sendo que o agente é o protagonista de ações que levam de um estado a outro. Após a transição de estados, o agente recebe uma recompensa ou punição dependendo se o novo estado satisfaz a expectativa do observador. Para um novo estado dentro da expectativa do observador, é dado ao agente uma recompensa. Por outro lado, para um novo estado fora da expectativa do observador, é dado ao agente uma punição.



Infográfico 1 - Aprendizado por reforço e o esquema de recompensa.

Fonte: adaptado de KDnuggets.

Você deve estar pensando que terá que acariciar seu computador ou chutá-lo dependendo se ele executar ou não o que você esperava. Mas calma! Não é bem assim. Na computação, as recompensas e punições estão relacionadas ao mundo dos números sendo traduzidas, portanto, em valores que podem variar de magnitude e/ou sinal para representá-los.

Entendido sobre aprendizado por reforço, vamos falar do método mais conhecido, chamado **Q-learning**, que é o tipo de aprendizado de máquina livre de modelagem formal prévia, no sentido de que o "agente" de Inteligência Artificial não precisa saber ou ter um modelo do ambiente em que estará. O mesmo algoritmo pode ser usado em uma variedade de ambientes (SUTTON, 2018).

Para um determinado ambiente, tudo é dividido em "estados" e "ações". Os estados são observações e amostras que extraímos do ambiente, e as ações são as escolhas que o agente fez com base na observação.

Então vamos utilizar um problema que envolve descobrir uma possível rota entre pontos. Teremos 8 pontos que representam os vértices de um grafo com suas coordenadas. Para isso, utilizaremos a biblioteca `networkx`. Verifique se ela está instalada, senão execute o comando `'pip install networkx'` a partir do prompt de comando do seu sistema operacional. Repita o processo para a biblioteca `'pylab'`. Na figura 9, temos a definição dos vértices do grafo em `points_list` e, usando a biblioteca `networkx`, conseguimos plotar os pontos (GÉRON, 2019).

```

> MI
import numpy as np
import pylab as plt
import networkx as nx

# map cell to cell, add circular cell to goal point
points_list = [(0,1), (1,5), (5,6), (5,4), (1,2), (2,3), (2,7)]

goal = 7

G=nx.Graph()
G.add_edges_from(points_list)
pos = nx.spring_layout(G)
nx.draw_networkx_nodes(G,pos)
nx.draw_networkx_edges(G,pos)
nx.draw_networkx_labels(G,pos)
plt.show()

```

Figura 9 - Definição dos pontos e plot na tela.
Fonte: elaborado pelo autor.

A figura 10 é o resultado dos pontos plotadores no formato de um grafo. Os pontos numerados em azul são os vértices e as arestas são as distâncias e conexões existentes entre cada ponto.

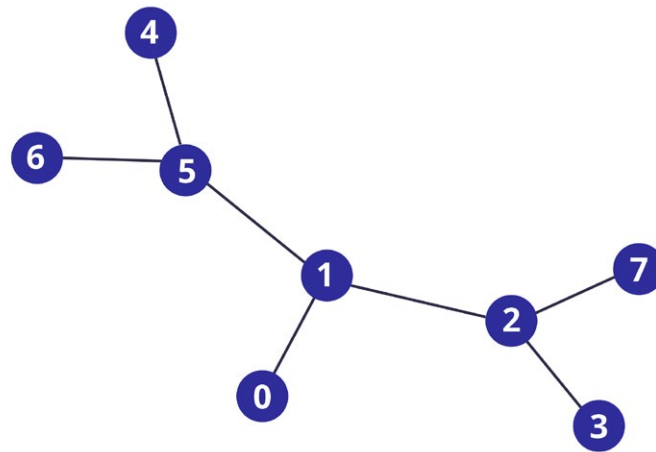


Figura 10 - Grafo dos 8 pontos.
Fonte: elaborado pelo autor.

Como temos 8 pontos, então a partir de cada ponto podemos ter a decisão de permanecer no próprio ponto ou se deslocar para outro ponto. Por isso, precisamos definir a matriz formada por números negativos de forma que inicialmente nenhuma aresta (caminho) exista ligando os vértices (pontos). Veja o código Python e o resultado conforme figura 11.

```

> MI
# how many points in graph? x points
MATRIX_SIZE = 8

# create matrix x*y
R = np.matrix(np.ones(shape=(MATRIX_SIZE, MATRIX_SIZE)))
R *= -1
R

matrix([[ -1.,  -1.,  -1.,  -1.,  -1.,  -1.,  -1.,  -1.],
        [ -1.,  -1.,  -1.,  -1.,  -1.,  -1.,  -1.,  -1.],
        [ -1.,  -1.,  -1.,  -1.,  -1.,  -1.,  -1.,  -1.],
        [ -1.,  -1.,  -1.,  -1.,  -1.,  -1.,  -1.,  -1.],
        [ -1.,  -1.,  -1.,  -1.,  -1.,  -1.,  -1.,  -1.],
        [ -1.,  -1.,  -1.,  -1.,  -1.,  -1.,  -1.,  -1.],
        [ -1.,  -1.,  -1.,  -1.,  -1.,  -1.,  -1.,  -1.],
        [ -1.,  -1.,  -1.,  -1.,  -1.,  -1.,  -1.,  -1.]])

```

Figura 11 - Inicialização da Matriz de estados.
Fonte: elaborado pelo autor.

O próximo passo, conforme a figura 12, é identificar os pontos que possuem conexão e inicializar seus vértices com zero. Os pontos que levam ao ponto destino (**goal**) terão vértice iniciado com valor 100.

```

> Ml
# assign zeros to paths and 100 to goal-reaching point
for point in points_list:
    print(point)
    if point[1] == goal:
        R[point] = 100
    else:
        R[point] = 0

    if point[0] == goal:
        R[point[:-1]] = 100
    else:
        # reverse of point
        R[point[::-1]] = 0

# add goal point round trip
R[goal,goal] = 100

```

Figura 12 - Melhores regras de associação.
Fonte: elaborado pelo autor.

O resultado da operação anterior pode ser visualizado na figura 13. Os vértices que possuem o valor 100 são os que levam do ponto 2 ao 7 e o ponto 7 em si mesmo, denotados pelas linhas 3 e 8, respectivamente, visto o primeiro ponto ser o ponto zero.

```

matrix([[ -1.,   0.,  -1.,  -1.,  -1.,  -1.,  -1.,  -1.],
        [  0.,  -1.,   0.,  -1.,  -1.,   0.,  -1.,  -1.],
        [-1.,   0.,  -1.,   0.,  -1.,  -1.,  -1., 100.],
        [-1.,  -1.,   0.,  -1.,  -1.,  -1.,  -1.,  -1.],
        [-1.,  -1.,  -1.,  -1.,  -1.,   0.,  -1.,  -1.],
        [-1.,   0.,  -1.,  -1.,   0.,  -1.,   0.,  -1.],
        [-1.,  -1.,  -1.,  -1.,  -1.,   0.,  -1.,  -1.],
        [-1.,  -1.,   0.,  -1.,  -1.,  -1.,  -1., 100.]])

```

Figura 13 - Matriz de estados representando os pontos e os vértices.
Fonte: elaborado pelo autor.

A partir da matriz anterior, precisamos entender como o aprendizado por reforço funciona. Já entendemos que, para cada novo estado, existe um reforço que é definido por meio de uma recompensa ou penalização, a depender da escolha a partir do estado atual. O valor de cada vértice será calculado utilizando o método Q-learning dado pela equação a seguir.

$$NewQ(s,a)=Q(s,a)+ \alpha[R(s,a)+ \gamma \max_{a'} Q'(s',a')-Q(s,a)]$$

A equação **NewQ(s,a)** é o valor atualizado do vértice, que será calculado como sendo uma atualização do valor atual dado por **Q(s,a)** adicionado do valor ponderado por **α**, que é a taxa de aprendizado. Entre colchetes, temos **R(s,a)**, que é o valor da recompensa atrelado ao vértice, o qual será somado a uma nova ponderação **γ**, que é o termo de desconto a ser calculado com base em todos os possíveis estados e recompensas dado por **Q'(s',a')**. O termo de desconto é o impacto que as ações futuras trazem para a ação atual de forma a corrigir a busca pelos vértices corretos (SUTTON, 2018).

Aprofunde-se

Neste vídeo, vamos apresentar a ideia de Q-learning com iteração de valor, que é uma técnica de aprendizagem por reforço usada para aprender a política ideal no processo decisório. Confira o vídeo para descobrir como. Ele está em inglês, mas com geração automática de legenda em português.

Título: Q-learning explicado - Uma técnica de aprendizado por reforço

Acesso em: 06/09/2020.

Disponível em: <https://youtu.be/qhRNvCVJJaA>

Entendido o funcionamento do método Q-learning e os passos anteriores de inicialização dos vértices e do problema, podemos avançar com a construção do código. Na figura 14, temos a definição das funções para cálculo das ações disponíveis a partir de um determinado estado, **available_actions(state)**. Na função **sample_next_action(available_actions_range)**, temos a seleção aleatória da próxima ação a ser tomada com base no estado atual. Isso serve para conferir ao algoritmo um caráter estocástico de forma a explorar todos os pontos possíveis, mas sempre priorizando os pontos mais próximos da meta. Por fim, temos a definição da função **update(current_state, action, gamma)** para atualizar os valores das arestas entre os vértices utilizando o método Q-learning.

```

> ML

Q = np.matrix(np.zeros([MATRIX_SIZE,MATRIX_SIZE]))

# learning parameter
gamma = 0.8

initial_state = 1

def available_actions(state):
    current_state_row = R[state,]
    av_act = np.where(current_state_row >= 0)[1]
    return av_act

available_act = available_actions(initial_state)

def sample_next_action(available_actions_range):
    next_action = int(np.random.choice(available_act,1))
    return next_action

action = sample_next_action(available_act)

def update(current_state, action, gamma):

    max_index = np.where(Q[action,] == np.max(Q[action,]))[1]

    if max_index.shape[0] > 1:
        max_index = int(np.random.choice(max_index, size = 1))
    else:
        max_index = int(max_index)
    max_value = Q[action, max_index]

    Q[current_state, action] = R[current_state, action] + gamma * max_value
    #print('max_value', R[current_state, action] + gamma * max_value)

    if (np.max(Q) > 0):
        return(np.sum(Q/np.max(Q)*100))
    else:
        return (0)

update(initial_state, action, gamma)

```

Figura 14 - Método Q-learning.
Fonte: elaborado pelo autor.

Vamos agora ao treinamento da malha de estados utilizando as funções anteriores. Iremos executar 700 iterações. Observe na figura 15 os passos para termos um algoritmo de aprendizado por reforço funcionando com o método Q-learning. Para cada cenário (entre os 700), será obtido um estado atual, as possíveis ações (em que direção se mover), um sorteio de qual ação tomar e a atualização dos valores da matriz com base no método Q-learning. **Ao final, temos a matriz com os valores ajustados.** Já é possível identificar a partir do ponto zero qual seria o caminho provável até o ponto sete.

```

▶ M!

# Training
scores = []
for i in range(700):
    current_state = np.random.randint(0, int(Q.shape[0]))
    available_act = available_actions(current_state)
    action = sample_next_action(available_act)
    score = update(current_state, action, gamma)
    scores.append(score)
    #print ('Score:', str(score))

print("Trained Q matrix:")
print(Q/np.max(Q)*100)

Trained Q matrix:
[[ 0.  64.  0.  0.  0.  0.  0.  0. ]
 [ 51.2 0.  80.  0.  0.  51.2 0.  0. ]
 [ 0.  64.  0.  64.  0.  0.  0. 100. ]
 [ 0.  0.  80.  0.  0.  0.  0.  0. ]
 [ 0.  0.  0.  0.  0.  51.2 0.  0. ]
 [ 0.  64.  0.  0.  40.96 0.  40.96 0. ]
 [ 0.  0.  0.  0.  0.  51.2 0.  0. ]
 [ 0.  0.  80.  0.  0.  0.  0. 100. ]]

```

Figura 15 - Matriz de treinamento.
Fonte: elaborado pelo autor.

O próximo passo é testar se o aprendizado por reforço deu certo do ponto de vista de criar os reforços positivos nos pontos onde existam conexões. Isso pode ser feito de acordo com o exposto na figura 16. Inicialmente, parte-se do estado inicial sendo igual a zero (**current_state = 0**) e enquanto o estado atual (**current_state**) não for o estado sete, ou seja, o ponto meta de chegada, o algoritmo irá ficar nesse laço em busca do caminho entre o ponto zero e o ponto sete. Ao final, temos que o melhor caminho entre o ponto de partida e o de chegada (meta) é o caminho que passa pelos pontos [0, 1, 2, 7]. Retorne à figura 10 para verificar e conferir.

```

> MI

# Testing
current_state = 0
steps = [current_state]

while current_state != 7:

    next_step_index = np.where(Q[current_state,] == np.max(Q[current_state,]))[1]

    if next_step_index.shape[0] > 1:
        next_step_index = int(np.random.choice(next_step_index, size = 1))
    else:
        next_step_index = int(next_step_index)

    steps.append(next_step_index)
    current_state = next_step_index

print("Most efficient path:")
print(steps)

plt.plot(scores)
plt.show()

Most efficient path:
[0, 1, 2, 7]

```

Figura 16 - Teste do aprendizado por reforço.

Fonte: elaborado pelo autor.

O algoritmo de teste do aprendizado por reforço gerou 700 episódios, embora, analisando o gráfico 3, seja possível determinar que, por volta do episódio 400, o algoritmo já tenha convergido, ou seja, o melhor caminho já havia sido determinado por volta do episódio 400. O que temos no gráfico, portanto, é uma distribuição dos episódios e o valor do **Q** acumulado em cada um deles. Quanto maior o valor de **Q**, maior a recompensa no aprendizado por reforço.

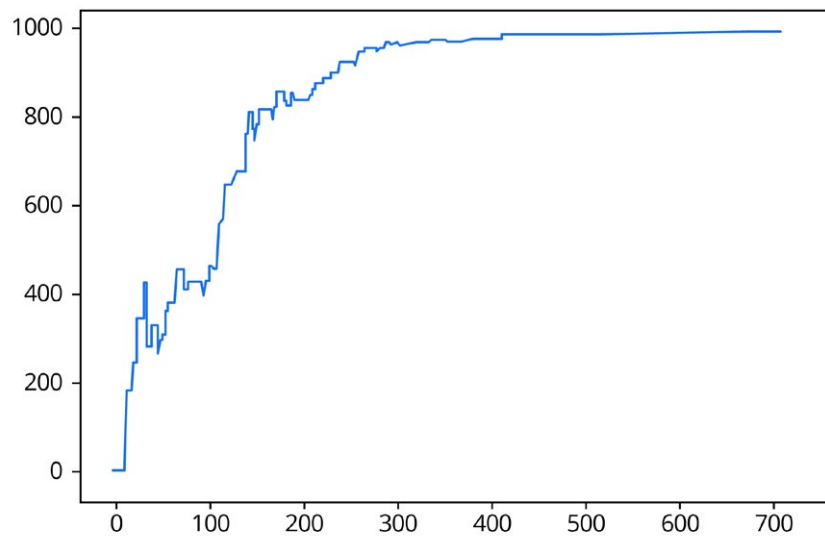


Gráfico 3 - Gráfico de conversão do algoritmo Q-learning.
Fonte: elaborado pelo autor.

Assista

Acesse na plataforma o vídeo: Aprendizado por reforço

4 MÉTODOS PARA DADOS NÃO ESTRUTURADOS

Em Ciência de dados, nem tudo são dados estruturados. O formato de dados estruturados é tudo o que um cientista de dados mais, digamos, tradicional deseja, já que as informações têm um formato que remete a um contexto. No caso, tabular os dados é no formato linha x coluna, no qual os registros de informações estão em cada linha e na coluna temos o nome do campo. Também temos dados estruturados em arquivos XML, JSON, entre outros. Quando falamos em vídeo, áudio e textos, temos neles a principal fonte de dados não estruturados.

4.1 ÁUDIO

Observe, na figura 17, algumas diferentes formas de ondas de sons urbanos. Esses dados são provenientes do projeto **Urban Sound Datasets** disponível em <https://urbansounddataset.weebly.com/>. Você

vai encontrar milhares de sons e ruídos urbanos gravados em altíssima qualidade de som (8K) para projetos de machine learning e Data Science.

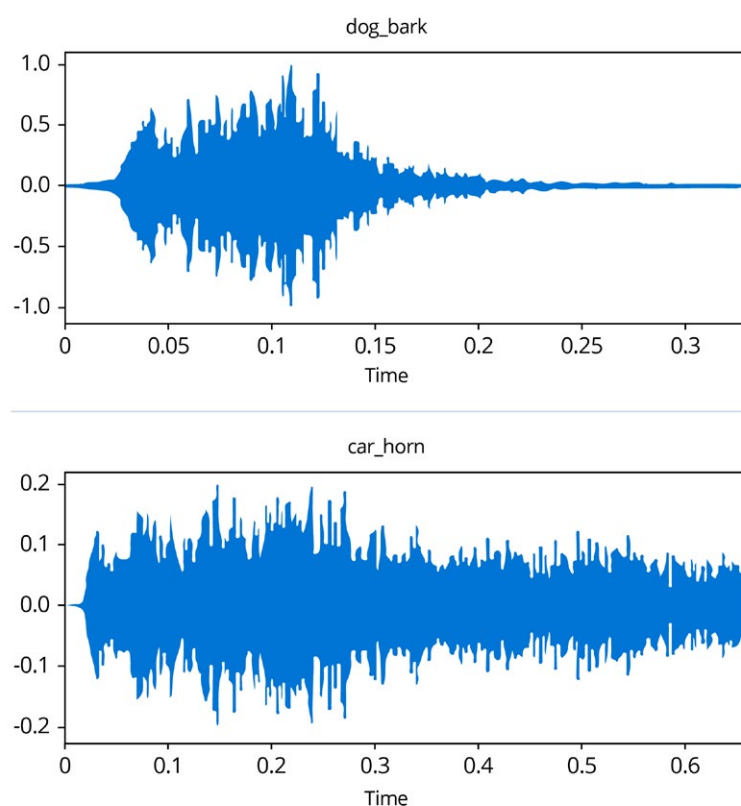


Figura 17 - Diferentes formatos de ondas sonoras.
Fonte: elaborado pelo autor.

Ao ter como objetivo classificar arquivos de áudio, a tarefa inicial é a extração de características (chamado de **extração de features**) dos arquivos de dados tal que, com as características extraídas, seja possível classificar e/ou agrupar os sons.

A aplicação de algoritmos de machine learning em arquivos de áudio envolve muitos passos de análise e transformação. Primeiramente, você precisa ter uma vasta coleção de áudios classificados como é o caso do exemplo em uso. Depois, você tem que garantir que todos os arquivos estão dentro de um mesmo espectro de frequência com qualidade de som mono. Arquivos estéreos tem o som codificado em duas faixas e, ao fazer a análise, você vai querer que o algoritmo entenda o som como um todo e não somente parte de uma faixa.

Tudo isso você pode fazer com a **biblioteca librosa**, a qual recomendo que você se aprofunde mais e está disponível no site oficial em <https://librosa.org/>. Usando essa biblioteca, você também vai ter que estudar sobre **MFCC (Mel-Frequency Cepstral Coefficients)**, que é uma forma de extração de características de

arquivos de áudio. Uma vez extraídas as características, você pode usar algoritmos de machine learning para fazer classificação, se for o caso.

Para problemas desse tipo, vem sendo muito utilizada as redes neurais convolucionais, mais conhecidas como **deep learning** (MICHELUCCI, 2019). São redes neurais com incrível capacidade de acurácia e precisão, mas sem o efeito indesejável do overfitting. Para aprender mais sobre isso, estude sobre as bibliotecas tensorflow e keras (GÉRON, 2019).

O conjunto de dados **UrbanSound8K** é formado por 8.732 classificados com sons tipicamente urbanos: latido de cão, criança brincando, buzina de carro, ar-condicionado, música de rua, tiro de arma de fogo, sirene, motor de carro, britadeira e furadeira. São 6,6GB de arquivos.

Ao aplicar os algoritmos mencionados e para 100 ciclos da rede neural convolucional, obteve-se 93% de acurácia no conjunto de treinamento e 88% de acurácia no conjunto de testes. Resultados excelentes se você for pensar em ter algum tipo de sistema urbano para reconhecer sons de armas de fogo e já alimentar as informações de abertura de chamados para o sistema da polícia. Bem interessante!

Aprofunde-se

Neste vídeo, você confere uma explicação amigável de como o computador reconhece imagens, com base em redes neurais convolucionais. Toda a matemática necessária é saber somar e subtrair 1's. Ele está em inglês, mas com geração automática de legenda em português.

Título: Uma introdução amigável para redes neurais convolucionais e reconhecimento de imagem

Acesso em: 06/09/2020.

Disponível em: <https://youtu.be/2-Ol7ZB0MmU>

4.2 IMAGEM

Com a assustadora popularização dos chamados **deep fakes**, o tratamento e criação de imagens e vídeos com uso de Inteligência Artificial tem ganho muito espaço na mídia técnica e geral.

Trabalhar com vídeo e imagens tem praticamente o mesmo problema, já que os vídeos são compostos por inúmeros frames de imagens. Claro que tratar um vídeo é mais dispendioso do ponto de vista computacional, pois um segundo de vídeo contém dezenas de imagens.

O processamento e classificação de imagens tem muitas aplicações interessantes, como a detecção de doenças com base em imagens de ressonância magnética, análise de sentimentos de pessoas olhando uma vitrine, classificação de imagens no ambiente para uso com veículos autônomos etc.

Um possível caminho para iniciar seus estudos é entender como funciona a extração de características de imagens. Recomendo que você estude a partir da **biblioteca OpenCV (Open Computer Vision)**, disponível no site oficial em <https://docs.opencv.org/master/index.html>. Essa biblioteca já prova funcionalidade para você analisar, redimensionar, transformar e obter características de imagens. Veja na figura 18 um exemplo de uma célula atípica (tumor) e um algoritmo de extração de características como formato, excentricidade, cor, proximidade e dimensões.

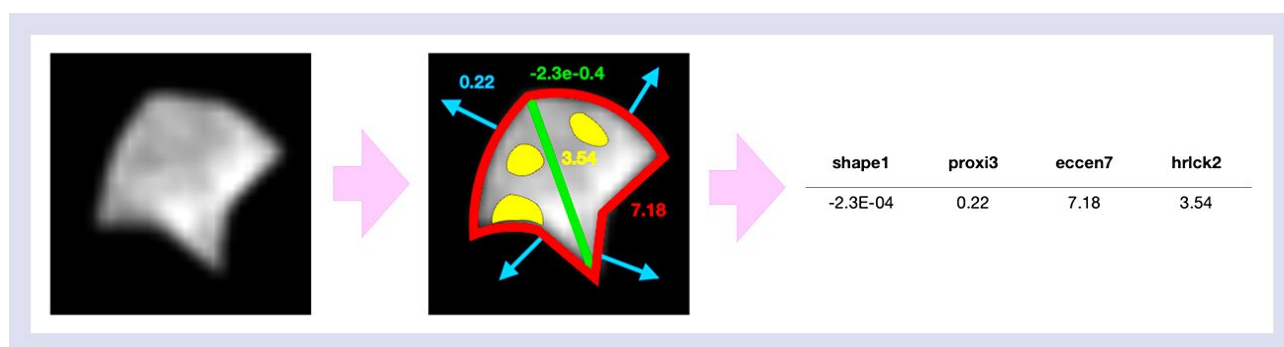


Figura 18 - Extração de características de imagens.
Fonte: elaborado pelo autor.

Uma vez extraídas essas características, podemos recorrer mais uma vez às redes neurais convolucionais (MICHELUCCI, 2019). Por isso a importância de você estudar as bibliotecas tensorflow e keras (GÉRON, 2019).

Assim como nos dados estruturados, a fonte das informações desempenha papel primordial. No caso da análise e classificação de imagens de tumor, por exemplo, foi feito um tratamento prévio de forma que todas as imagens fossem centralizadas e de mesma dimensão. Como a atipia está relacionada muito mais ao formato, as imagens foram previamente transformadas em preto e branco. Isso ajuda inclusive no tamanho dos arquivos. Veja na figura 19 um exemplo do que foi explanado.

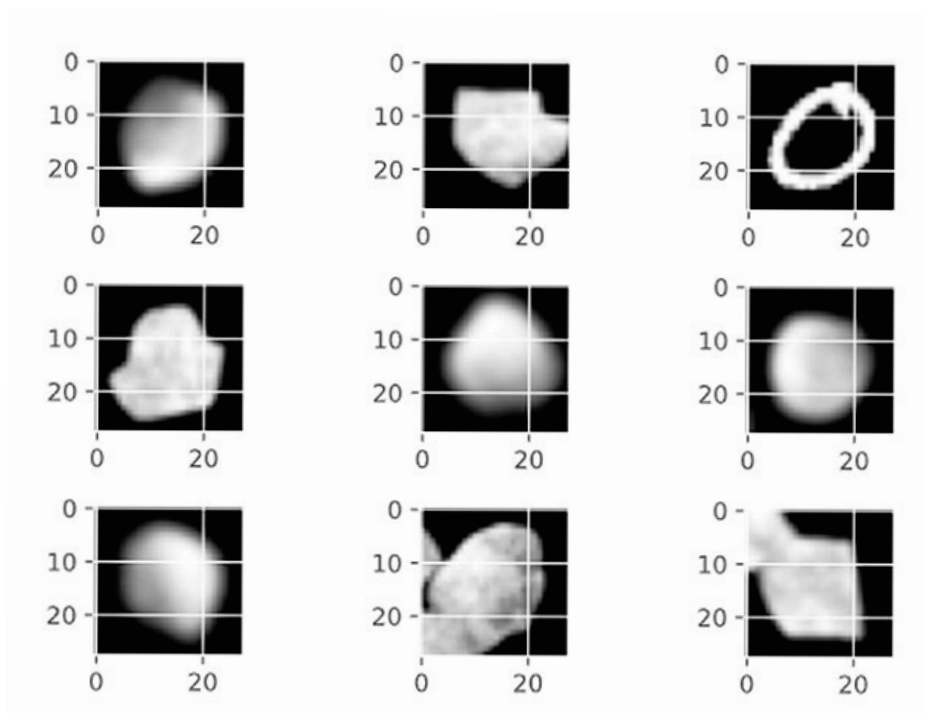


Figura 19 - Imagens de células normais e atípicas.
Fonte: elaborado pelo autor.

Com a extração de atributos, é interessante que você pode utilizar outros modelos de classificação que vimos também, como Random Forest, KNN, regressão logística etc. Isso é interessante porque você consegue induzir diferentes classificadores e avaliar a performance utilizando validação cruzada, por exemplo.

O conjunto de dados citado como exemplo possui 1.200 imagens que tiveram suas características extraídas e codificadas em 280 colunas, contendo cor, dimensão, rotação, tamanho, proximidade e excentricidade. O número de características é resultante da análise tridimensional da célula, que é uma estrutura bidimensional apenas em imagem. Por isso, ao analisar de diversas perspectivas diferentes, para esse caso, chegou-se a esse número de características.

Ao aplicar **deep learning**, mais uma vez o resultado foi surpreendente com 97% de acurácia dos dados de treinamento e 92% de precisão nos dados de teste.

4.3 TEXTO

O tratamento, processamento, interpretação, entendimento e geração de dados dentro do mundo da Inteligência Artificial tem nome: **NLP** - Natural Language Processing, ou, em português, Processamento de Linguagem Natural. Posso dizer que é um mundo à parte também. Já é um ramo mais maduro porque vem se desenvolvendo há anos junto com algoritmos de machine learning (MISHRA; KUMAR, 2012).

Algumas das histórias de sucesso mais famosas envolvendo NLP vêm do Google, onde ela é usada para dar respostas muito boas a pesquisas vagas ou com erros ortográficos na internet, traduções automáticas bastante compreensíveis de texto simples e legendas geradas automaticamente para a maioria dos vídeos do YouTube.

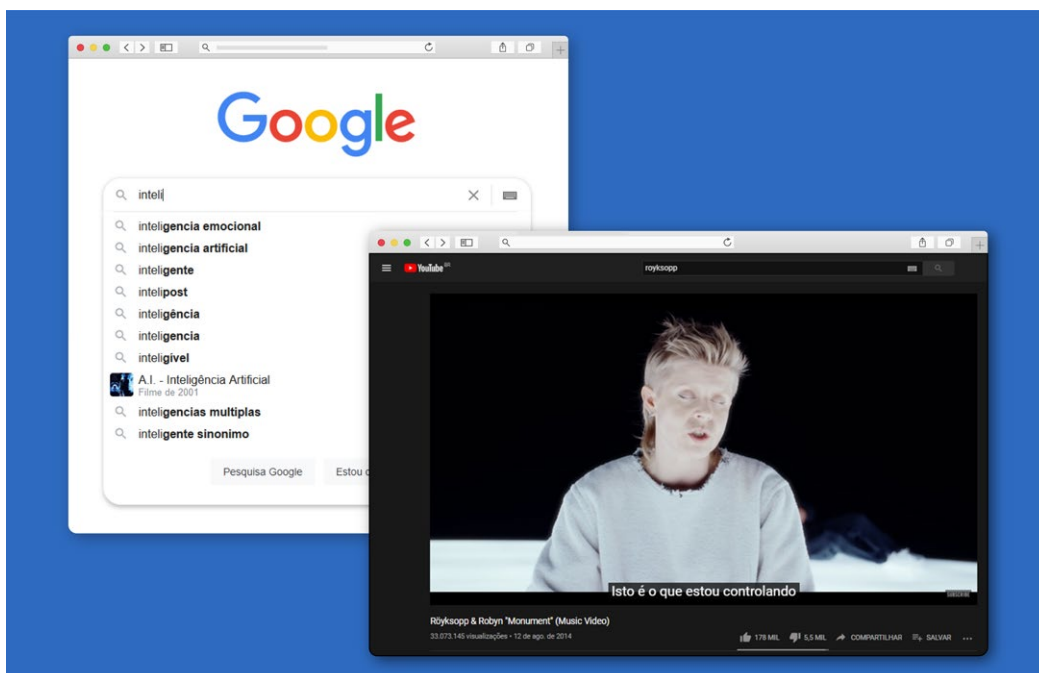


Figura 20 - Exemplo das NPL em funcionamento: busca no Google e legenda de em vídeos do YouTube.

Fonte: adaptados de <https://bit.ly/38C67Fz> e <https://youtu.be/6c-RbGZBnBI>

Acesso em: 12 nov. 2020.

A NLP tem uma gama tão diversa de usos que pode ser difícil encontrar uma definição sucinta para ela. Em geral, os computadores preferem dados numéricos e linguagens formais, como matemática e linguagens de programação, enquanto os humanos preferem se comunicar nas chamadas "linguagens naturais": inglês, espanhol e assim por diante. Sempre que tentamos construir uma ponte entre o computador e os humanos, a NLP entra em jogo para construir essa interface de comunicação (MISHRA; KUMAR, 2012). Alguns exemplos de aplicação de NLP:

- Para ajudar os computadores a entender os humanos falando (pense em Siri), usamos NLP e processamento de áudio.

- Para traduzir automaticamente entre diferentes idiomas naturais (pense no Google Translate).
- Para aplicar automaticamente rótulos ou categorias a textos naturais (pense em detectores de spam que mantêm e-mails indesejados fora de sua caixa de entrada).
- Para que os computadores leiam o texto em voz alta para uso (pense no Kindle transformando e-books em audiolivros), novamente usamos NLP com processamento de áudio.

Muitas pessoas pensam que a NLP é apenas aprendizado de máquina aplicado a palavras. No entanto, isso está longe de ser verdade. A NLP tem duas características que a tornam intratável para aplicações comuns de aprendizado de máquina:

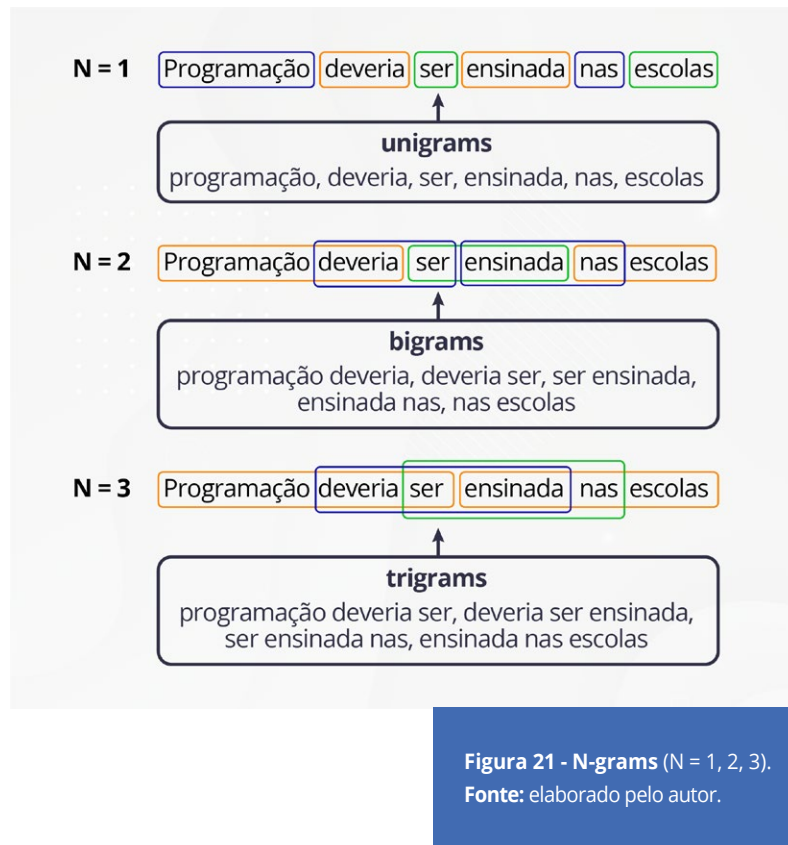
1. **Altíssima dimensionalidade:** considere, por exemplo, uma versão em português da Bíblia com mais de 3,5 milhões de caracteres. Podemos ver isso como um longo vetor de strings assumindo valores em um espaço de 3 milhões de dimensões e, em seguida, aplicar métodos de aprendizado de máquina aqui?
2. **Texto depende do contexto:** considere por exemplo a palavra “meia”. Pode ser a peça do vestuário. Pode se referir a seis quando nos referimos à meia dúzia ou ao número seis propriamente. Pode se referir a 30 minutos quando nos referimos há uma hora e meia. Na entrada do teatro pode se referir à metade do valor.

Mais uma vez, graças à comunidade Python que sempre tem soluções para “quase” tudo e não para de crescer em quantidade e qualidade, temos a biblioteca **NLTK** – Natural Language Toolkit.

NLTK é uma ótima biblioteca para aprender sobre NLP em Python (BIRD, 2009). Ela implementa quase todos os algoritmos padrões usados em NLP em Python puro e é muito legível. Ela tem uma excelente documentação e um livro que a acompanha, e muitas vezes implementa várias alternativas para o mesmo algoritmo para que possamos compará-los.

Para aprender sobre NLP usando a biblioteca NTKL, você terá que entender o conceito de **token** (símbolo), que é o processo de particionamento do texto em partes que remetem a uma sentença. Você pode até pensar que é fácil, bastando procurar por pontos finais. Mas não é tão simples, afinal pontos finais também podem ser utilizados em números como separação de milhares e marcação de ordem. Mas, mais uma vez, NLTK provê uma solução para isso (BIRD, 2009).

Outra questão a ter em mente é o **contexto das palavras**, mesmo aquelas que não têm sentido dubio como a palavra “meia”. Por exemplo, um texto com ocorrências da palavra “bom” poderia levar você a crer que ele se refere a algo que seja bom. Mas se o adjetivo for precedido da expressão “não é tão”, o entendimento já se altera. Então outro ponto a se considerar é a criação das tuplas de forma a manter grupos de palavras juntas para que o seu significado original não se perca. Isso é chamado de **n-grams**, onde **n** é o número de palavras a ser mantidas juntas. Não é um simples particionamento do texto original e grupos, mas a manutenção da união das palavras de forma repetida para que não se perca a semântica, conforme ilustrado na figura 20.



Ao estudar esse tópico, você provavelmente vai ler e ouvir falar das **stop words**, que é o conjunto de palavras mais usuais do vocabulário e que não conferem semântica ao texto, como os pronomes e preposições. Se formos analisar o texto pela frequência dos **n-grams**, essas palavras vão poluir demais a análise e, por isso, sempre são descartadas antes de iniciar a análise (BIRD, 2009).

Feito o levantamento dos **n-grams** e sua frequência, você pode analisar a lista ordenada dos **n-grams** e começar a tirar algumas conclusões. Por exemplo, ao analisar as avaliações dos clientes de uma rede de fast food que fornece lanches, você deparou com o trigram “pior lanche hambúrguer” com ocorrência de 250 vezes, já subtraídas as stop words. Acredito que você entendendo tudo que falamos até agora iria se levantar e falar com o proprietário da rede de lanches.

Você também pode utilizar os **n-grams** para, por exemplo, classificar músicas em gêneros musicais apenas com base na letra delas.

Assista

Acesse na plataforma o vídeo: Processamento de Linguagem Natural

Importante: para melhor compreensão desse vídeo, você deverá acessar o arquivo **sdata.csv** disponível em sua plataforma.

CONSIDERAÇÕES FINAIS

Nesta unidade, você aprendeu conceitos importantes sobre métodos de agrupamento, mais popularmente conhecidos como clusterização. Aprendemos sobre o pai dos métodos de análise inteligentes de dados, os métodos de associação, mais precisamente vimos sobre regras de associação. Diferentemente dos outros paradigmas que vínhamos estudando, mergulhamos no mundo do aprendizado por reforço em uma aplicação de descoberta de rota entre cidades. Por último, mas não menos importante, abordamos os métodos para dados não estruturados. Claro que esse é um tópico bem abrangente e, por isso, no tutorial escolhi tratar apenas de NLP para trazer mais riqueza. O que posso desejar para você agora é sucesso nessa jornada. Continue os estudos, faça cases e participe de grupos de Ciência de dados. São muitas oportunidades de trabalho nessa área, muitas mesmo. Até breve!

GLOSSÁRIO

NLP (Natural Language Processing): conjunto de técnicas e algoritmos de Inteligência Artificial utilizado para o processamento, transformação e entendimento de textos.

NLTK (Natural Language Toolkit): é uma biblioteca de Python que implementa diversas funcionalidades para o tratamento de linguagem natural.

REFERÊNCIAS

AGGARWAL, Charu C. REDDY, Chandan K. **Data Clustering: Algorithms and Applications**. 1. ed. CRC Press, 2013. 652 p.

BIRD, Steven; KLEIN, Ewan; LOPER, Edward. **Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit**. 1. ed. O'Reilly Media, 2009. 803 p.

GÉRON, Aurélien. **Hands-on machine learning with scikit-learn, tensorflow and keras: concepts, tools, and techniques to build intelligent systems**. 2. ed. O'Reilly Media, 2019. 600 p.

MICHELUCCI, Umberto. **Advanced Applied Deep Learning: convolutional Neural Networks and Object Detection**. 1. ed. Apress, 2019. 304 p.

MISHRA, Brojo Kishore; KUMAR, Raghvendra. **Natural Language Processing in Artificial Intelligence**. International Publishing Hous, 2012.

SUTTON, Richard S.; BARTO, Andrew G. **Reinforcement Learning: An Introduction**. 2. ed. MIT Press, 2018. 552 p.

